Python 程序设计

实验 1–15 全解析学习讲义

题目 → 超详细解析 → 专题拓展 → 参考资料

实验 1	实验 2	实验 3	实验 4	实验 5
实验 6	实验 7	实验 8	实验 9	实验 10
实验 11	实验 12	实验 13	实验 14	实验 15

覆盖内容 运算符与内置函数・列表/集合/字典・选择与循环・函数与递归
类与运算符重载・字符串・随机模拟・pillow 图像・pandas/matplotlib

配套代码 15 个实验的完整可运行 .py 文件（另附压缩包）

运行环境 Python 3.13 ・numpy ・pandas ・matplotlib ・pillow（均已实际运行验证）

2026 年 11 月

目录

编者说明 iv

第一部分　全部原始实验题目 1

实验一　Python 开发与实验环境使用 1

实验二　Python 基本运算 1

实验三　熟悉列表和集合——求素数 2

实验四　熟悉使用 filter() 函数 2

实验五　使用蒙特·卡罗方法计算圆周率 2

实验六　猜数游戏 3

实验七　使用枚举法验证 6174 猜想 3

实验八　小明爬楼梯 3

实验九　抓狐狸游戏 4

实验十　自定义类模拟三维向量及其运算 4

实验十一　蒙蒂霍尔悖论游戏 4

实验十二　凯撒加密算法原理与实现 4

实验十三　打字练习成绩评定 5

实验十四　生成棋盘纹理图片 5

实验十五　数据分析与可视化综合实验 5

第二部分　全部答案与超详细解析 7

【实验一】Python 开发与实验环境使用 7

【实验二】Python 基本运算 10

【实验三】熟悉列表和集合——求素数 15

【实验四】熟悉使用 filter() 函数 17

【实验五】使用蒙特·卡罗方法计算圆周率 19

【实验六】猜数游戏 21

【实验七】使用枚举法验证 6174 猜想 23

【实验八】小明爬楼梯 25

【实验九】抓狐狸游戏 28

【实验十】自定义类模拟三维向量及其运算 31

【实验十一】蒙蒂霍尔悖论游戏 34

【实验十二】凯撒加密算法原理与实现 37

【实验十三】打字练习成绩评定 39

【实验十四】生成棋盘纹理图片 41

【实验十五】数据分析与可视化综合实验 43

第三部分　专题拓展：举一反三、知识体系、高校公开题与科学史 48

开篇　题目分类与筛选 48

第一章　本次实验的知识体系总览 49

第二章　重点题深度回顾 50

第三章　重点题举一反三 50

第四章　易错题专题 54

第五章　易错题举一反三 55

第六章　核心知识系统回顾 56

第七章　学科科普：我们学的“程序设计”究竟是什么 59

第八章　科学史：这些题目背后的人与事 59

第九章　关联学科知识网络与跨课程迁移 61

第十章　国内外高校公开课程题典型题专题 62

第十一章　生活中的实际应用 67

第十二章　学习总结 **70**

第四部分　参考资料与权威来源 **71**

编者说明

1. **结构**：全书严格按“第一部分全部原始题目 → 第二部分全部答案与解析 → 第三部分专题拓展 → 第四部分参考资料”排列。第一部分只有题目本身。
2. **题号**：原始图片中“熟悉使用 filter() 函数”一题的标题也写作“实验三”，按其在图片序列中的位置（位于实验三与实验五之间）以及 15 个实验的总数判断，应为**实验四**，本讲义统一按实验四处理。
3. **错别字校正**（均依据上下文与学科逻辑判断，求解后已反向验证）：实验三目的中“filer()” → “filter()”；实验四中“测试整数 n 是否为叔叔” → “素数”，“若干随机证书” → “若干随机整数”，“列表 1st” → “列表 lst”；实验七中“各位书字” → “各位数字”，“数量使用内置函数” → “熟练使用内置函数”；实验十五中“反应” → “反映”。
4. **删除内容**：原实验一中“实验平台用法”（平台网址、登录、导出 html 与提交实验报告的步骤）属于教学平台与提交说明，按讲义规范删除；原实验一中三张截图在原页面中已无法显示，题目中以文字说明其作用。
5. **代码**：第二部分中的全部代码均与随附压缩包中的 `.py` 文件逐字一致（排版时直接从源文件读入），并已在 Python 3.13 中实际运行。含随机数的程序，讲义中的输出仅为某一次运行的示例。

第一部分　全部原始实验题目

实验一　Python 开发与实验环境使用

实验一　Python 开发与实验环境使用

实验目的

1. 熟练掌握 Python 解释器安装与基本用法。
2. 熟练掌握使用 pip 命令安装 Python 扩展库。
3. 熟悉掌握实验平台基本用法。

实验内容

1. 安装 Python 解释器。
2. 安装 Python 扩展库。

实验步骤（单机用法）

1. 打开 Python 官方网站 http://www.python.org。
2. 下载 Python 3.5.x 或 Python 3.6.x 或 Python 3.7.x 的最新版，至少安装其中两个。
3. 在开始菜单中找到成功安装的 IDLE，输入一段测试代码，确保 IDLE 运行正常（原题配有 IDLE 截图）。
4. 在资源管理器中进入 Python 安装目录的 `scripts` 子目录，然后按下 Shift 键，在空白处单击鼠标右键，在弹出来的菜单中选择“在此处打开命令窗口”进入命令提示符环境（原题配有截图）。
5. 使用 pip 命令在线安装 Python 扩展库 numpy、pandas、scipy、matplotlib、jieba、openpyxl、pillow（原题以安装 openpyxl 的命令截图为例）。
6. 如果遇到安装不成功的扩展库，使用浏览器打开下面的网址下载 whl 文件进行离线安装：https://www.lfd.uci.edu/~gohlke/pythonlibs/
7. 在 IDLE 中使用 import 导入安装好的扩展库，验证是否安装成功（原题配有截图）。

代码与 Markdown 示例：在 Jupyter 环境中运行代码实例

```
# 代码实例，可直接运行
print('hello world!')
```

并用 Markdown 写出一级标题、二级标题、三级标题与加粗文字。

实验二　Python 基本运算

实验二　Python 基本运算

实验目的：1. 熟练运用 Python 运算符；2. 熟练运用 Python 内置函数；3. 理解浮点数运算的误差。

实验内容

1. 编写程序，输入任意大的自然数，输出各位数字之和。
2. 编写程序，输入两个集合 setA 和 setB，分别输出它们的交集、并集和差集 setA−setB。
3. 编写程序，输入一个自然数，输出它的二进制、八进制、十六进制表示形式。
4. 阅读并调试下面的代码，分析代码功能，发现并解决代码中的错误。

```python
def cni(n,i):
    minNI = min(i, n-i)
    result = 1
    for j in range(0, minNI):
        result = result * (n-j) // (minNI-j)
    return result
```

实验三　熟悉列表和集合——求素数

实验三　熟悉列表和集合——求素数

实验目的

1. 使用列表实现筛选法求素数：(1) 理解筛选法求解素数的原理；(2) 理解列表切片操作；(3) 熟练运用内置函数 `enumerate()`；(4) 熟练运用内置函数 `filter()`；(5) 理解序列解包工作原理；(6) 初步了解选择结构和循环结构。
2. 使用集合实现筛选法求素数：(1) 理解求解素数的筛选法原理；(2) 理解 Python 集合对象的 `discard()` 方法；(3) 熟练运用列表推导式；(4) 理解 for 循环工作原理。

实验内容：输入一个大于 2 的自然数，输出小于该数字的所有素数组成的列表/集合。

实验四　熟悉使用 filter() 函数

实验四　熟悉使用 filter() 函数

实验目的：1. 熟练运用 Python 运算符；2. 列表推导式语法和执行过程；3. 运用列表推导式解决实际问题；4. 函数的定义与使用；5. 使用内置函数 `filter()`；6. 掌握 lambda 表达式作为函数参数的用法。

实验内容：首先，使用列表推导式和标准库 random 生成一个包含 50 个介于 1–100 的随机整数的列表，然后编写函数 `def isPrime(n)` 来测试整数 n 是否为素数，接下来使用内置函数 `filter()` 把函数 `isPrime()` 作用到包含若干随机整数的列表 `lst` 上，最后程序输出一个列表，其中包含列表 `lst` 中不是素数的那些整数。

实验五　使用蒙特·卡罗方法计算圆周率

实验五　使用蒙特·卡罗方法计算圆周率

实验目的：1. 理解蒙特·卡罗方法原理；2. 理解 for 循环本质与工作原理；3. 了解 random 模块中常用函数。

实验内容：蒙特·卡罗方法是一种通过概率来得到问题近似解的方法，在很多领域都有重要的应用，其中就包括圆周率近似值的计算问题。假设有一块边长为 2 的正方形木板，上面画一个单位圆，然后随意往木板上扔飞镖，落点坐标 (x, y) 必然在木板上（更多的时候是落在单位圆内），如果扔的次数足够多，那么落在单位圆内的次数除以总次数再乘以 4，这个数字会无限逼近圆周率的值。这就是蒙特·卡罗发明的用于计算圆周率近似值的方法，如图1所示。

编写程序，模拟蒙特·卡罗计算圆周率近似值的方法，输入掷飞镖次数，然后输出圆周率近似值。

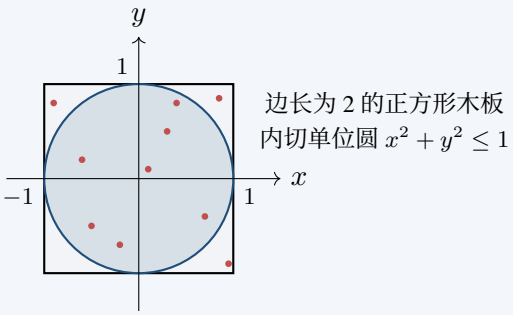


图 1　原题示意图（按题意重绘）：随机投掷的飞镖落点

实验六　猜数游戏

实验六　猜数游戏

实验目的：1. 熟练运用选择结构与循环结构解决实际问题；2. 注意选择结构嵌套时代码的缩进与对齐；3. 理解带 else 子句的循环结构执行流程；4. 理解条件表达式 `value1 if condition else value2` 的用法；5. 理解使用异常处理结构约束用户输入的用法；6. 理解带 else 子句的异常处理结构的执行流程。

实验内容：编写程序模拟猜数游戏。程序运行时，系统生成一个随机数，然后提示用户进行猜测，并根据用户输入进行必要的提示（猜对了、太大了、太小了），如果猜对则提前结束程序，如果次数用完仍没有猜对，提示游戏结束并给出正确答案。

实验七　使用枚举法验证 6174 猜想

实验七　使用枚举法验证 6174 猜想

实验目的：1. 了解 6174 猜想的内容；2. 使用选择结构和循环结构；3. 了解标准库 itertools 中的 `combinations()` 函数的用法；4. 熟练使用字符串 `join()` 方法；5. 熟练使用内置函数 `int()`、`str()`、`sorted()`。

实验内容：1955 年，卡普耶卡对四位数字进行研究，发现一个规律：对任意各位数字不相同的 4 位数，使用各位数字能组成的最大数减去能组成的最小数，对得到的差重复这个操作，最终会得到 6174 这个数字，并且这个操作最多不超过 7 次。（编程验证该猜想。）

实验八　小明爬楼梯

实验八　小明爬楼梯

实验目的：1. 理解并熟练使用序列解包；2. 理解递归函数工作原理；3. 能够编写递归函数代码解决实际问题；4. 理解 Python 字典的用法。

实验内容：假设一段楼梯共 15 个台阶，小明一步最多能上 3 个台阶。编写程序计算小明上这段楼梯一共有多少种方法。要求给出递推法和递归法两种代码。

实验九　抓狐狸游戏

实验九　抓狐狸游戏

实验目的：1. 培养分析问题并对其进行建模的能力；2. 熟练使用列表解决实际问题；3. 熟练运用选择结构和循环结构解决实际问题；4. 理解带 else 子句的循环结构执行流程；5. 理解使用异常处理结构约束用户输入的用法。

实验内容：编写程序，模拟抓狐狸小游戏。假设一共有一排 5 个洞口，小狐狸最开始的时候在其中一个洞口，然后玩家随机打开一个洞口，如果里面有狐狸就抓到了。如果洞口里没有狐狸就第二天再来抓，但是第二天狐狸会在玩家来抓之前跳到隔壁洞口里。

实验十　自定义类模拟三维向量及其运算

实验十　自定义类模拟三维向量及其运算

实验目的：1. 了解如何定义一个类；2. 了解如何定义类的私有数据成员和成员方法；3. 了解如何使用自定义类实例化对象。

实验内容：定义一个三维向量类，并定义相应的特殊方法实现两个该类对象之间的加、减运算（要求支持运算符 +、-），实现该类对象与标量的乘、除运算（要求支持运算符 *、/），以及向量长度的计算（要求使用属性实现）。

实验十一　蒙蒂霍尔悖论游戏

实验十一　蒙蒂霍尔悖论游戏

实验目的：1. 了解蒙蒂霍尔悖论内容；2. 了解游戏规则；3. 熟练运用字典方法和集合运算；4. 了解断言语句 assert 的用法；5. 熟练运用循环结构。

实验内容：假设你正参加一个有奖游戏节目，并且有 3 道门可选：其中一个后面是汽车，另外两个后面是山羊。你选择一个门，比如说 1 号门，主持人当然知道每个门后面是什么并且打开了另一个门，比如说 3 号门，后面是一只山羊。这时，主持人会问你“你想改选 2 号门吗?”，然后根据你的选择确定最终要打开的门，并确定你获得山羊（输）或者汽车（赢）。编写程序，模拟上面的游戏。

实验十二　凯撒加密算法原理与实现

实验十二　凯撒加密算法原理与实现

实验目的：1. 了解 Python 标准库 string；2. 理解凯撒加密算法原理；3. 理解切片操作；4. 熟练运用字符串对象的方法。

实验内容：编写程序，要求输入一个字符串，然后输入一个整数作为凯撒加密算法的密钥，然后输出该字符串加密后的结果。

实验十三　打字练习成绩评定

实验十三　打字练习成绩评定

实验目的：1. 熟练运用内置函数 `zip()`、`sum()`、`round()`、`isinstance()`；2. 熟练运用生成器表达式。

实验内容：编写程序，模拟打字练习程序的成绩评定。假设 origin 为原始内容，userInput 为用户练习时输入的内容，要求用户输入的内容长度不能大于原始内容的长度，如果对应位置的字符一致则认为正确，否则判定输入错误。最后成绩为：正确的字符数量/原始字符串长度，按百分制输出，要求保留 2 位小数。

实验十四　生成棋盘纹理图片

实验十四　生成棋盘纹理图片

实验目的：1. 熟悉 Python 扩展库 pillow 的安装方法；2. 熟悉 Python 扩展库 pillow 的简单使用；3. 理解棋盘网格纹理的生成原理。

实验内容：编写程序，绘制棋盘网格，要求棋盘的宽度和高度、交替的两种颜色以及网格数量都可以通过参数指定，并且两种颜色交替出现，水平方向和垂直方向上的网格数量相同。

实验十五　数据分析与可视化综合实验

实验十五　数据分析与可视化综合实验

实验目的：1. 熟悉 Python 标准库 csv 的用法；2. 熟悉 CSV 和 TXT 文件操作；3. 熟练安装扩展库 numpy、pandas、matplotlib；4. 熟悉使用扩展库 pandas 进行数据分析的基本操作；5. 熟悉使用扩展库 matplotlib 进行数据可视化的基本操作。

实验内容

(1) 运行下面的程序，在当前文件夹中生成饭店营业额模拟数据文件 `data.csv`。

```
import csv
import random
import datetime

fn = 'data.csv'

with open(fn, 'w') as fp:
    # 创建 csv 文件写入对象
    wr = csv.writer(fp)
    # 写入表头
    wr.writerow(['日期', '销量'])

    # 生成模拟数据
    startDate = datetime.date(2017, 1, 1)

    # 生成 365 个模拟数据，可以根据需要进行调整
    for i in range(365):
        # 生成一个模拟数据，写入 csv 文件
        amount = 300 + i*5 + random.randrange(100)
```

```
        wr.writerow([str(startDate), amount])
        # 下一天
        startDate = startDate + datetime.timedelta(days=1)
```

(2) 然后完成下面的任务：

1) 使用 pandas 读取文件 `data.csv` 中的数据，创建 DataFrame 对象，并删除其中所有缺失值；
2) 使用 matplotlib 生成折线图，反映该饭店每天的营业额情况，并把图形保存为本地文件 `first.jpg`；
3) 按月份进行统计，使用 matplotlib 绘制柱状图显示每个月份的营业额，并把图形保存为本地文件 `second.jpg`；
4) 按月份进行统计，找出相邻两个月最大涨幅，并把涨幅最大的月份写入文件 `maxMonth.txt`；
5) 按季度统计该饭店 2017 年的营业额数据，使用 matplotlib 生成饼状图显示 2017 年 4 个季度的营业额分布情况，并把图形保存为本地文件 `third.jpg`。

第二部分　全部答案与超详细解析

本部分按原实验编号逐题对应。每题依次给出：考查知识点、已知条件、解题目标、解题思路、公式与定理说明、完整代码与推导、最终答案、结果检验；适宜时给出多种解法与易错点。代码行号与随附压缩包中的源文件一致。

【实验一】Python 开发与实验环境使用

1. 考查知识点　解释器（interpreter）与 IDLE；pip 包管理器与 PyPI（Python 包索引）；wheel（`.whl`）离线安装；`import` 语句；“包名”与“模块名”的区别；Jupyter 与 Markdown 基本语法。

2. 已知条件　题目要求安装 Python 解释器（原题指定 3.5/3.6/3.7 系列）和 7 个扩展库：numpy、pandas、scipy、matplotlib、jieba、openpyxl、pillow，并用 import 验证。

3. 解题目标　得到一个“能运行 `print('hello world!')`、能用 pip 装库、能 import 这 7 个库”的开发环境，并会写基本 Markdown。

4. 解题思路　先弄清楚 Python 程序是怎样跑起来的（图2）：源代码交给解释器执行；扩展库是别人写好的代码，由 pip 从 PyPI 下载后放进**当前解释器**的 `site-packages` 目录，之后才能被 `import`。因此安装与验证的关键只有一句话：**用哪个解释器运行程序，就要用同一个解释器的 pip 安装库**。

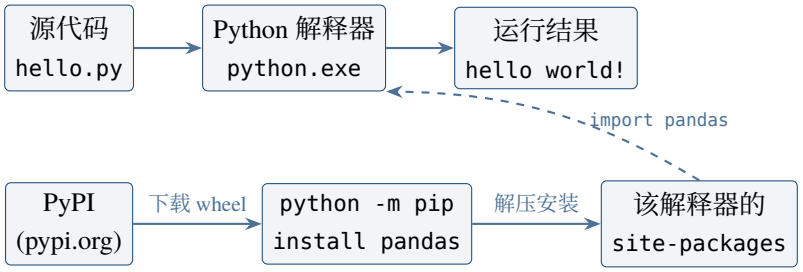


图 2　Python 程序的运行与扩展库的安装路径。看点：pip 把库装进“某一个”解释器的 site-packages，import 只在同一个解释器里找。

5. 关键命令说明

- `python -m pip install 包名`：`-m pip` 表示“用这个 python 运行 pip 模块”，可保证库装进同一个解释器，比直接敲 `pip` 更稳妥。
- Windows 下装了多个版本时，用启动器 `py -3.12 -m pip install numpy` 指定版本；`py -0` 可列出已安装的版本。
- 国内网络较慢时可临时指定镜像（如清华大学开源软件镜像站）：

```
python -m pip install numpy -i https://pypi.tuna.tsinghua.edu.cn/simple
```

- 题目第 4 步“进入 scripts 目录打开命令窗口”，目的就是让 `pip.exe` 可被找到；如果安装时勾选了 *Add python.exe to PATH*，在任意目录都可以使用。

6. 对原题版本要求的更新说明　原题要求的 Python 3.5/3.6/3.7 均已停止维护：3.5 于 2020 年 9 月、3.6 于 2021 年 12 月、3.7 于 2023 年 6 月 27 日到达生命周期终点（end-of-life），不再有任何安全修复；按 Python 官方开发者指南，3.10 已于 2026 年 10 月停止维护。今天做本实验，建议安装两个仍受支持的版本（如 3.12 与 3.13）来完成“至少安装两个版本”的要求。原题第 6 步给出的 Christoph Gohlke 非官方 Windows wheel 网站，已因其实验室经费终止于 2022 年 6 月底停止服务；现在主流库都在 PyPI 上直接提供 Windows wheel，遇到装不上的情况，优先升级 pip（`python -m pip install`

`-U pip`）或换用 conda。

7. 完整代码　下面的程序一次完成“运行 hello world + 报告版本 + 逐个 import 7 个库”：

```python
# -*- coding: utf-8 -*-
"""
实验一　Python 开发与实验环境使用
功能：
  1. 运行教材示例 print('hello world!')，确认解释器正常；
  2. 打印当前 Python 版本、解释器路径、pip 版本；
  3. 逐个 import 实验要求安装的 7 个扩展库，报告是否安装成功及版本号。
未安装的库会给出对应的 pip 安装命令（注意：import 名与 pip 包名不一定相同）。
"""
import importlib
import sys
import platform

# 代码实例，可直接运行
print('hello world!')

print('Python 版本  :', platform.python_version())
print('解释器路径   :', sys.executable)

try:
    import pip
    print('pip 版本     :', pip.__version__)
except ImportError:
    print('pip 未安装，可运行: python -m ensurepip --upgrade')

# (import 时使用的模块名, pip 安装时使用的包名)
LIBS = [
    ('numpy', 'numpy'),
    ('pandas', 'pandas'),
    ('scipy', 'scipy'),
    ('matplotlib', 'matplotlib'),
    ('jieba', 'jieba'),
    ('openpyxl', 'openpyxl'),
    ('PIL', 'pillow'),        # pillow 安装后用 import PIL 导入
]

print('-' * 48)
missing = []
for module_name, pip_name in LIBS:
    try:
        mod = importlib.import_module(module_name)
        version = getattr(mod, '__version__', '版本号未知')
        print(f'[成功] {pip_name:<11} import {module_name:<11} 版本 {version}')
    except ImportError:
        print(f'[失败] {pip_name:<11} 尚未安装')
        missing.append(pip_name)

```

```
48 print('-' * 48)
49 if missing:
50     print('请在命令行执行下面的命令安装缺失的库：')
51     print(f'"{sys.executable}" -m pip install ' + ' '.join(missing))
52 else:
53     print('全部扩展库均已安装成功！')
```

第 27–35 行的列表把**安装名**与**导入名**成对存放——pillow 安装时叫 `pillow`，导入时却是 `PIL`；第 41 行 `importlib.import_module()` 用字符串导入模块，失败时抛出 `ImportError`，被第 44 行捕获。

8. 运行结果（本讲义环境）

```
hello world!
Python 版本  : 3.13.18
[成功] numpy       import numpy       版本 2.5.3
[成功] pandas      import pandas      版本 3.0.5
[成功] scipy       import scipy       版本 1.18.1
[成功] matplotlib  import matplotlib  版本 3.11.2
[成功] jieba       import jieba       版本 0.42.1
[成功] openpyxl    import openpyxl    版本 3.1.5
[成功] pillow      import PIL         版本 12.3.0
全部扩展库均已安装成功!
```

9. Markdown 部分

在 Jupyter 中把单元格类型切换为 Markdown（快捷键 `M`），输入：

```
# markdown 示例（一级标题）
## 二级标题
### 三级标题
** 加粗 **
```

井号个数等于标题级别，井号后要有一个空格；两对星号包围的文字加粗。按 Shift+Enter 渲染。

最终答案

安装受支持的 Python 版本 → 用 `python -m pip install numpy pandas scipy matplotlib jieba openpyxl pillow` 安装扩展库 → 运行上面的检查程序，全部显示“成功”即完成。

10. 结果检验

①在命令行运行 `python -c "import sys; print(sys.executable)"` 与在 IDLE 中运行同一句，两者路径必须相同；② `python -m pip show pandas` 显示的 `Location` 应位于该解释器目录下。

易错点

1. **在 Python 交互提示符 `>>>` 里输入 `pip install ...`**：会报 SyntaxError。pip 是命令行程序，要在命令提示符/终端里运行，不是 Python 语句。
2. **“装好了却 import 失败”**：最常见原因是装了两个 Python，pip 属于 A 而 IDLE 用的是 B。判断方法见“结果检验”①；改正方法是统一使用 **对应解释器 `-m pip`**。
3. **`import pillow` 报错**：pillow 的导入名是 `PIL`；同理 `pip install opencv-python` 后要 `import cv2`。包名与模块名可以不同。

【实验二】Python 基本运算

第 1 小题　任意大自然数的各位数字之和

考查知识点　字符串遍历、`map()`、`sum()`、整除 `//` 与取余 `%`、`divmod()`；Python 整数的任意精度。

已知与目标　输入一个位数不限的自然数 $n=\overline{d_{k-1}\cdots d_1 d_0}$，求 $S=\sum_{i=0}^{k-1} d_i$。

解题思路　“任意大”是题眼。Python 的 `int` 是任意精度整数，不会像 C 语言的 `int` 那样溢出，所以两条路都可行：**方法一**把输入直接当字符串，每个字符就是一位；**方法二**用十进制位值展开 $n=10q+r$，其中 $r=n \bmod 10$ 就是最低位，$q=\lfloor n/10 \rfloor$ 是去掉最低位后的数，反复进行直到 $n=0$。

公式说明　带余除法：对整数 $n\ge 0$，存在唯一的 q,r 使 $n=10q+r,\ 0\le r<10$。Python 中 `divmod(n, 10)` 一次返回 `(n // 10, n % 10)`，即 (q,r)。

```python
# -*- coding: utf-8 -*-
""" 实验二-1：输入任意大的自然数，输出各位数字之和。"""


def digit_sum_str(s):
    """ 方法一：把数字当字符串处理，map(int, s) 逐位转整数再求和。"""
    return sum(map(int, s))


def digit_sum_math(n):
    """ 方法二：纯算术，反复 divmod(n, 10) 取最低位。"""
    total = 0
    while n > 0:
        n, r = divmod(n, 10)
        total += r
    return total


if __name__ == '__main__':
    s = input('请输入一个自然数：').strip()
    if not s.isdigit():          # 约束输入：只能包含 0-9
        print('输入错误：必须是自然数（只含数字 0-9）')
    else:
        print('各位数字之和（字符串法）：', digit_sum_str(s))
        print('各位数字之和（算术法）  ：', digit_sum_math(int(s)))
```

推导追踪　以 $n=2026$ 为例：$2026=10\times 202+6$，$202=10\times 20+2$，$20=10\times 2+0$，$2=10\times 0+2$，余数依次为 $6,2,0,2$，和为 10。

最终答案

输入 `98765432109876543210`（20 位），两种方法都输出 **90**。

结果检验　该数由 $0\sim 9$ 各出现两次组成，和应为 $2\times(0+1+\cdots+9)=2\times 45=90$，与程序一致。

第 2 小题　集合的交、并、差

考查知识点　集合运算符 `&`、`|`、`-`；集合字面量；安全解析输入 `ast.literal_eval()`。

公式说明　$A\cap B=\{x\mid x\in A\text{ 且 }x\in B\}$, $A\cup B=\{x\mid x\in A\text{ 或 }x\in B\}$, $A-B=\{x\mid x\in A\text{ 且 }x\notin B\}$（图3）。

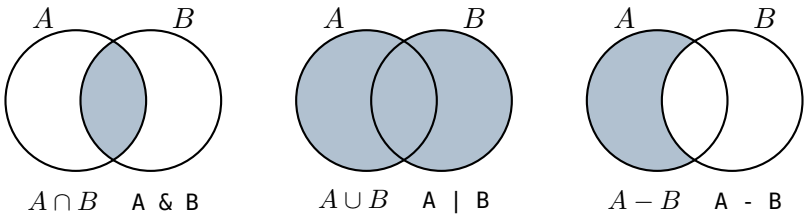


图 3　三种集合运算的维恩图（阴影为结果）。看点：差集 $A-B$ 只保留“属于 A 但不属于 B”的部分，因此 $A-B\neq B-A$。

```python
# -*- coding: utf-8 -*-
""" 实验二-2：输入两个集合 setA 和 setB，输出交集、并集和差集 setA - setB。
输入格式示例：{1, 2, 3, 4}  或  1,2,3,4
"""
import ast


def read_set(prompt):
    text = input(prompt).strip()
    if not (text.startswith('{') or text.startswith('[') or text.startswith('(')):
        text = '{' + text + '}'          # 允许直接输入 1,2,3
    if text in ('{}', '{ }'):
        return set()                     # 注意：{} 是空字典，空集合要用 set()
    # ast.literal_eval 只解析字面量，比 eval 安全
    return set(ast.literal_eval(text))


if __name__ == '__main__':
    setA = read_set('请输入集合 setA：')
    setB = read_set('请输入集合 setB：')
    print('交集 setA & setB =', setA & setB)
    print('并集 setA | setB =', setA | setB)
    print('差集 setA - setB =', setA - setB)
```

代码要点　第 12–13 行：`{}` 在 Python 中是**空字典**而不是空集合，空集合只能写 `set()`，这里特意处理。第 15 行用 `ast.literal_eval` 而不用 `eval`：前者只接受数字、字符串、元组、列表、集合等字面量，不会执行任意代码。

最终答案

输入 `{1,2,3,4}` 与 `3,4,5`，输出：交集 $\{3,4\}$，并集 $\{1,2,3,4,5\}$，差集 $\{1,2\}$。

结果检验　应满足 $|A\cup B|=|A|+|B|-|A\cap B|$：$5=4+3-2$ ✓；且 $(A-B)\cup(A\cap B)=A$ ✓。

第 3 小题　二进制、八进制、十六进制

考查知识点　内置函数 `bin()`、`oct()`、`hex()`、`format()`；进制转换“除基取余、逆序排列”。

原理　若 $n=\sum_i a_i b^i\ (0\le a_i<b)$，则 $n \bmod b = a_0$，$\lfloor n/b \rfloor = \sum_{i\ge 1} a_i b^{i-1}$。所以反复除以基数 b，余数从低位到高位依次得到各位数字，最后要**逆序**（图4）。

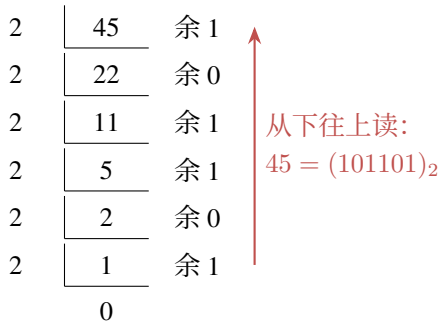


图 4　短除法把 45 化为二进制。看点：先得到的余数是最低位，所以要逆序读取。

```python
# -*- coding: utf-8 -*-
""" 实验二-3：输入一个自然数，输出它的二进制、八进制、十六进制表示。"""


def to_base(n, base):
    """ 手写的“除基取余、逆序排列”法，用来和内置函数互相验证。"""
    digits = '0123456789abcdef'
    if n == 0:
        return '0'
    out = []
    while n > 0:
        n, r = divmod(n, base)
        out.append(digits[r])
    return ''.join(reversed(out))


if __name__ == '__main__':
    n = int(input('请输入一个自然数：'))
    print('二进制  :', bin(n), '  不带前缀:', format(n, 'b'))
    print('八进制  :', oct(n), '  不带前缀:', format(n, 'o'))
    print('十六进制:', hex(n), '  不带前缀:', format(n, 'x'))
    # 自检：手写算法与内置函数结果一致
    assert to_base(n, 2) == format(n, 'b')
    assert to_base(n, 8) == format(n, 'o')
    assert to_base(n, 16) == format(n, 'x')
    print('手写进制转换与内置函数结果一致')
```

最终答案

输入 2026，输出 `0b11111101010`、`0o3752`、`0x7ea`；手写算法与内置函数一致。

结果检验　$3\times 8^3+7\times 8^2+5\times 8+2=1536+448+40+2=2026$ ✓；$7\times 256+14\times 16+10=1792+224+10=2026$ ✓。

第 4 小题　调试组合数函数 cni

考查知识点　组合数 $\binom{n}{i}$ 的乘积公式；整除 `//` 的截断；浮点数误差（实验目的 3）；循环不变式。

功能分析　函数名 cni 即“C(n, i)”。第 2 行用 $\binom{n}{i}=\binom{n}{n-i}$ 取较小者 $k=\min(i,n-i)$ 以减少循环次数；循环想计算

$$\binom{n}{k}=\frac{n(n-1)\cdots(n-k+1)}{k!}=\prod_{j=0}^{k-1}\frac{n-j}{\cdot}.$$

定位错误　原代码第 j 步除以 $(k-j)$，即分母按 $k,k-1,\ldots,1$ 的顺序出现。第一步就是 $\lfloor n/k\rfloor$，当 $k\nmid n$ 时被截断。以 `cni(5,2)` 逐步追踪：

j	原代码 `result*(n-j)//(k-j)`	改正后 `result*(n-j)//(j+1)`	改正后的含义
0	$1\times5\,//\,2=\mathbf{2}$（2.5 被截断）	$1\times5\,//\,1=5$	$\binom{5}{1}=5$
1	$2\times4\,//\,1=\mathbf{8}$	$5\times4\,//\,2=10$	$\binom{5}{2}=10$

为什么改成 $j+1$ 就一定整除　记第 j 步结束后的值为 r_j。若 $r_{j-1}=\binom{n}{j}$，则

$$r_{j-1}\,(n-j)=\binom{n}{j}(n-j)=\frac{n!}{j!\,(n-j-1)!}=(j+1)\binom{n}{j+1},$$

所以 $r_{j-1}(n-j)$ 恰好是 $(j+1)$ 的倍数，整除没有任何截断，并且 $r_j=\binom{n}{j+1}$。由 $r_{-1}=1=\binom{n}{0}$ 归纳，循环结束时 $r_{k-1}=\binom{n}{k}$。这就是“循环不变式”：**每一步结束时 result 都是一个组合数**。原代码没有这个性质，中间值 $n(n-1)\cdots/(k(k-1)\cdots)$ 一般不是整数。

```python
# -*- coding: utf-8 -*-
""" 实验二-4：阅读并调试组合数函数 cni(n, i)，发现并改正错误。

原代码：
    def cni(n,i):
        minNI = min(i, n-i)
        result = 1
        for j in range(0, minNI):
            result = result * (n-j) // (minNI-j)
        return result

功能：计算组合数 C(n, i) = n! / (i! (n-i)!)。
错误：分母用 (minNI-j) 按 k, k-1, ..., 1 的顺序去除，中间结果
      n(n-1).../(k(k-1)...) 不一定是整数，整除 // 会截断，结果偏小。
      例如 cni(5, 2) 得到 8，正确答案是 10。
改正：分母改为 (j+1)，按 1, 2, ..., k 的顺序除。第 j 步结束后
      result 恰好等于 C(n, j+1)，始终是整数，// 不会丢失精度。
"""
from math import comb   # Python 3.8+ 自带，仅用于验证


def cni_buggy(n, i):
    minNI = min(i, n - i)
    result = 1
    for j in range(0, minNI):
```

```
        result = result * (n - j) // (minNI - j)
    return result


def cni(n, i):
    """ 改正后的版本，并增加参数检查。"""
    if not (isinstance(n, int) and isinstance(i, int)) or n < 0 or i < 0 or i > n:
        raise ValueError('要求 n、i 为整数且 0 <= i <= n')
    minNI = min(i, n - i)            # 利用 C(n,i) = C(n,n-i) 减少循环次数
    result = 1
    for j in range(0, minNI):
        result = result * (n - j) // (j + 1)
    return result


if __name__ == '__main__':
    print('原函数 cni(5,2) =', cni_buggy(5, 2), '   （错误）')
    print('改正后 cni(5,2) =', cni(5, 2))
    # 找出原函数出错的最小反例并全面对拍
    wrong = [(n, i) for n in range(0, 30) for i in range(0, n + 1)
             if cni_buggy(n, i) != comb(n, i)]
    print('n<30 时原函数出错的 (n,i) 个数：', len(wrong), '，例如', wrong[:5])
    assert all(cni(n, i) == comb(n, i) for n in range(0, 200) for i in range(0, n + 1))
    print('改正后的 cni 在 0<=i<=n<200 范围内与 math.comb 完全一致')
    print('cni(100, 50) =', cni(100, 50))
```

最终答案

功能：计算组合数 $\binom{n}{i}$。**错误**：第 5 行分母 `(minNI-j)` 使中间结果非整数，`//` 截断导致结果偏小（如 `cni(5,2)` 得 8）。**改正**：把第 5 行改为 `result = result * (n-j) // (j+1)`，并补充参数检查。

结果检验　程序与标准库 `math.comb` 在 $0 \le i \le n < 200$ 全部范围内对拍一致；而原函数在 $n < 30$ 时就有 297 组出错，最小反例为 $(5, 2)$。

解法二：浮点除法为什么不行　有人会把 `//` 改成 `/`。小数据看似正确，但 Python 浮点数只有 53 位二进制有效数字（约 15–16 位十进制），实测 $\binom{100}{50}$ 用 `/` 计算再取整得 `100891344545564184479528910848`，而精确值为 `100891344545564193334812497256`，从第 17 位开始就错了。这正是实验目的“理解浮点数运算的误差”：**整数问题坚持整数运算**。例如 `0.1 + 0.2 == 0.3` 的结果是 **`False`**，因为 0.1 在二进制下是无限循环小数，只能近似存储。

易错点

1. **只改成 `/` 不改分母顺序**：大数时有浮点误差，结果还是 `float`，不符合“组合数是整数”的本质。
2. **先乘完再除**（`num //= den` 放到最后）：结果对，但中间数字比答案大得多，失去了“边乘边除”控制规模的好处。
3. **忽视参数合法性**：$i > n$ 时 `min(i, n-i)` 为负，`range` 为空，原函数返回 1，属于“静默错误”；改正版主动抛出 `ValueError`。

【实验三】熟悉列表和集合——求素数

1. 考查知识点　素数定义；埃拉托斯特尼筛法；列表切片赋值；`enumerate()`；`filter()` 与 lambda；集合 `discard()`；列表推导式。

2. 已知条件与目标　输入 $N > 2$，输出所有小于 N 的素数（$2 \le p < N$）。

3. 解题思路　筛法不是“逐个判断谁是素数”，而是“逐个划掉谁不是”：从最小的 2 开始，2 是素数，把 2 的倍数全部划掉；剩下的最小数 3 必是素数，再划掉 3 的倍数……一直到某个不超过 $\sqrt{N}$ 的位置停下，剩下的全是素数（图5）。

4. 关键定理　**若合数 c 满足 $c < N$，则它一定有一个素因子 $p \le \sqrt{c} < \sqrt{N}$。**证明：设 $c = ab$，$1 < a \le b$，则 $a^2 \le ab = c$，所以 $a \le \sqrt{c}$，而 a 的任一素因子 $p \le a$。因此只需用不超过 $\lfloor\sqrt{N}\rfloor$ 的素数去筛，所有合数都会被划掉。

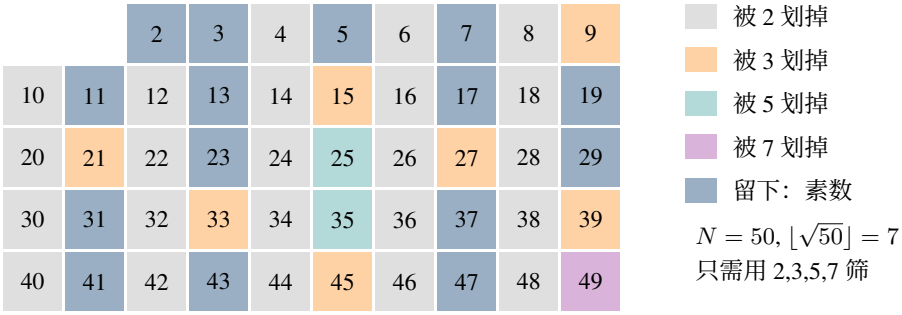


图 5　$N = 50$ 时的筛法过程。颜色表示该合数第一次被哪个素数划掉。看点：7 之后再无新的颜色——所有合数都已被不超过 $\sqrt{50}$ 的素数划掉。

解法一：列表 + filter + 切片赋值

```python
# -*- coding: utf-8 -*-
""" 实验三-1：使用列表实现筛选法（埃拉托斯特尼筛法）求小于 maxNumber 的所有素数。
涉及：列表切片赋值、enumerate()、filter()、lambda、选择与循环结构。
"""


def primes_by_list(maxNumber):
    lst = list(range(2, maxNumber))          # 候选数 2, 3, ..., maxNumber-1
    m = int(maxNumber ** 0.5)                # 只需用不超过 sqrt(maxNumber) 的数去筛
    for index, value in enumerate(lst):
        if value > m:                        # 之后剩下的数都已是素数
            break
        # 切片赋值：把 value 后面所有 value 的倍数筛掉
        lst[index + 1:] = filter(lambda x: x % value != 0, lst[index + 1:])
    return lst


if __name__ == '__main__':
    maxNumber = int(input('请输入一个大于 2 的自然数：'))
    if maxNumber <= 2:
        print('输入必须大于 2')
    else:
        print(primes_by_list(maxNumber))
```

逐行解释

- 第 8 行 `lst = list(range(2, maxNumber))`：候选数 $2, 3, \ldots, N-1$。
- 第 10 行 `for index, value in enumerate(lst)`：`enumerate` 同时给出下标和值，这里用到了**序列解包**——每次产生的元组 `(index, value)` 被拆给两个变量。
- 第 14 行是核心：`filter(lambda x: x % value != 0, lst[index+1:])` 从 value 之后的元素中只保留**不能被 value 整除**的，再通过**切片赋值** `lst[index+1:] = ...` 原地替换列表尾部。
- 第 11–12 行：一旦当前 value 超过 $m = \lfloor\sqrt{N}\rfloor$，由上面的定理，剩下的元素都是素数，直接 break。

一个容易被忽视的细节　循环在遍历 `lst` 的同时修改了 `lst`。这里之所以安全，是因为 `enumerate` 内部按“当前下标”去列表里取元素，而切片赋值只改动**当前位置之后**的部分，已经遍历过的位置不变，所以下一轮取到的正是筛过之后的下一个数——它一定是素数。

解法二：集合 + discard + 列表推导式

```
1  # -*- coding: utf-8 -*-
2  """ 实验三-2：使用集合实现筛选法求小于 maxNumber 的所有素数。
3  涉及：集合的 discard() 方法、列表推导式、for 循环。
4  """
5
6
7  def primes_by_set(maxNumber):
8      numbers = set(range(2, maxNumber))
9      m = int(maxNumber ** 0.5) + 1
10     # 列表推导式：先求出不超过 sqrt(maxNumber) 的素数（试除法）
11     primesLessThanM = [p for p in range(2, m)
12                        if 0 not in [p % d for d in range(2, int(p ** 0.5) + 1)]]
13     for p in primesLessThanM:
14         for i in range(2, maxNumber // p + 1):
15             numbers.discard(i * p)           # discard: 元素不存在时不报错（remove 会报
               ↪ KeyError）
16     return numbers
17
18
19 if __name__ == '__main__':
20     maxNumber = int(input('请输入一个大于 2 的自然数：'))
21     if maxNumber <= 2:
22         print('输入必须大于 2')
23     else:
24         result = primes_by_set(maxNumber)
25         print(result)
26         print('排序后：', sorted(result))     # 集合无序，展示时可排序
```

逐行解释　第 11–12 行用列表推导式加试除法先求出“小素数” $p \le \sqrt{N}$：`0 not in [p % d for d in ...]` 表示 p 不能被 $2 \sim \lfloor\sqrt{p}\rfloor$ 中任何数整除。第 13–15 行对每个小素数 p，把 $2p, 3p, \ldots$ 从集合中 `discard` 掉。`discard` 与 `remove` 的区别是：元素不存在时 `discard` 静默忽略，`remove` 抛出 `KeyError`——而 6 会被 2 和 3 各删一次，第二次删除时它已经不在集合里了，所以必须用 `discard`。

最终答案

输入 50，解法一输出 `[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]`，解法二输出同样 15 个元素组成的集合。

结果检验　小于 50 的素数个数 $\pi(49) = 15$，与已知素数表一致；两种解法结果相同；列表版输出天然有序，集合版需要 `sorted()` 才能有序显示。

两种解法比较　列表版依赖“遍历中修改列表”的技巧，逻辑紧凑；集合版先求小素数再删倍数，思路更直观，且集合删除平均 $O(1)$。若追求效率，可用布尔列表标记（见第三部分第十章 CMU 15-112 的写法），总复杂度为 $O(N \log\log N)$。

易错点

1. **把 1 当成素数或把 N 本身算进去**：题目要求“小于该数字”，所以是 `range(2, maxNumber)`。
2. **边界 m 少 1**：若写成 `range(2, int(maxNumber**0.5))`，当 $N = 50$ 时只用 2..6 筛，**7 被漏掉**，49 会被误判为素数。检查方法：取 N 略大于某素数平方（如 50），看 p^2 是否被筛掉。
3. **极大 N 时浮点开方误差**：`int(n**0.5)` 对很大的完全平方数可能差 1，严谨写法是 `math.isqrt(n)`（整数平方根）。

【实验四】熟悉使用 filter() 函数

1. 考查知识点　列表推导式；`random.randint`；函数定义；`filter(function, iterable)`；lambda 表达式；逻辑非 `not`。

2. 已知与目标　生成 50 个 $[1, 100]$ 内的随机整数构成 `lst`；用 `filter` 与 `isPrime` 得到 `lst` 中**不是**素数的元素（保持原顺序、保留重复）。

3. 解题思路　`filter(f, seq)` 只保留使 `f(x)` 为真的元素（图6）。我们要的是“非素数”，所以传入的判定函数要取反：`lambda x: not isPrime(x)`。

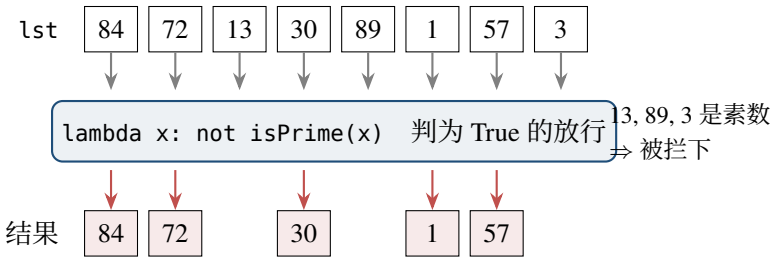


图 6　filter 的工作方式。看点：filter 不改变元素，只决定“放行还是拦下”，并保持原顺序；1 不是素数，所以 1 被放行。

4. 素数判定的依据　由实验三的定理，$n \geq 2$ 是素数当且仅当它不能被 $2 \sim \lfloor\sqrt{n}\rfloor$ 中的任何整数整除。代码中还先排除偶数，再只试奇数因子，试除次数约减半。

```python
# -*- coding: utf-8 -*-
""" 实验四（原图标题误写为“实验三”）：熟悉使用 filter() 函数。
用列表推导式和 random 生成 50 个 [1,100] 的随机整数列表 lst，
编写 isPrime(n) 判断素数，再用 filter() 输出 lst 中不是素数的那些整数。
"""
import random


def isPrime(n):
    """ 试除法：只需试除到 sqrt(n)。1 不是素数。"""
    if n < 2:
        return False
    if n in (2, 3):
        return True
    if n % 2 == 0:
        return False
    for i in range(3, int(n ** 0.5) + 1, 2):
        if n % i == 0:
            return False
    return True


if __name__ == '__main__':
    lst = [random.randint(1, 100) for _ in range(50)]   # randint 两端都包含
    print('随机列表 lst：', lst)
    # filter 只保留函数返回 True 的元素，所以要“取反”
    notPrimes = list(filter(lambda x: not isPrime(x), lst))
    print('不是素数的整数：', notPrimes)
    # 对照：素数部分 + 非素数部分 = 原列表长度
    primes = list(filter(isPrime, lst))
    assert len(primes) + len(notPrimes) == len(lst)
    print('其中素数有', len(primes), '个，非素数有', len(notPrimes), '个')
```

5. 一次运行示例

```
随机列表 lst： [84, 72, 63, 58, 13, 30, 89, 18, 50, 57, 85, 54, 57, 54, 69, 100, 57, 63, 3, 33,
              33, 53, 72, 84, 71, 80, 54, 88, 94, 91, 65, 14, 34, 68, 17, 33, 23, 15, 55, 17,
              37, 57, 44, 82, 48, 100, 63, 50, 99, 98]
不是素数的整数： [84, 72, 63, 58, 30, 18, 50, 57, 85, 54, 57, 54, 69, 100, 57, 63, 33, 33, 72, 84,
               80, 54, 88, 94, 91, 65, 14, 34, 68, 33, 15, 55, 57, 44, 82, 48, 100, 63, 50, 99,
               ↪ 98]
其中素数有 9 个，非素数有 41 个
```

最终答案

核心语句：`notPrimes = list(filter(lambda x: not isPrime(x), lst))`。输出随随机数而变，上面为一次实际运行。

6. 结果检验　①素数个数 + 非素数个数 $= 50$（代码第 31 行断言）；②抽查：$57 = 3\times19$、$91 = 7\times13$、$85 = 5\times17$ 都是合数，均出现在结果中；$13, 89, 3, 53, 71, 17, 23, 37$ 都是素数，均未出现。③数量级：$1 \sim 100$ 中有 25 个素数，随机抽 50 个时素数个数的期望是 $50 \times \frac{25}{100} = 12.5$，本次 9 个属于正常波动。

7. 一题多解

- 列表推导式：`[x for x in lst if not isPrime(x)]`，可读性更好。
- 标准库：`from itertools import filterfalse; list(filterfalse(isPrime, lst))`，直接保留“判定为假”的元素，省去 lambda。

三者结果完全相同。filter 体现的是“函数作为参数”的函数式思想；推导式更符合 Python 风格；`filterfalse` 最简洁但需要多记一个工具。

易错点

1. **认为 1 是素数**：按定义素数是大于 1 且只有 1 和自身两个因数的整数，`isPrime(1)` 必须为 False。
2. **直接 print(filter(...))**：Python 3 中 filter 返回**惰性迭代器**，打印出来是 `<filter object at ...>`，需要 `list()` 转换；而且迭代器只能遍历一次，第二次遍历为空。
3. **randint 与 randrange 的端点**：`randint(1, 100)` 两端都包含；`randrange(1, 100)` 不包含 100。

【实验五】使用蒙特·卡罗方法计算圆周率

1. 考查知识点　几何概率；大数定律；`random.uniform` / `random.random`；for 循环计数；估计误差。

2. 已知条件　正方形 $D=[-1,1]\times[-1,1]$，面积 4；单位圆 $C=\{x^2+y^2\le 1\}$，面积 π；飞镖落点在 D 内均匀分布；投掷次数 n 由用户输入。

3. 解题目标　用落入 C 的频率估计 π。

4. 解题思路　落点均匀分布时，“落在某区域内”的概率等于“该区域面积 / 总面积”。所以一次投掷落入圆内的概率是 $p=\pi/4$。投 n 次、命中 h 次，频率 h/n 会随 n 增大逼近 p，于是 $\hat{\pi}=4h/n$。

5. 公式与定理说明

- **几何概率**：$P(\text{落入}C)=\dfrac{S_C}{S_D}=\dfrac{\pi\cdot 1^2}{2^2}=\dfrac{\pi}{4}$。成立条件是落点在 D 上均匀分布，`random.unifo」rm(-1, 1)` 分别生成 x,y 正好满足。
- **大数定律**：设 $X_k=1$（第 k 次命中）或 0，它们独立同分布且 $EX_k=p$，则 $\dfrac{h}{n}=\dfrac{1}{n}\sum X_k\to p$（依概率收敛）。
- **误差大小**：$h\sim B(n,p)$，$\mathrm{Var}(\hat{\pi})=16\cdot\dfrac{p(1-p)}{n}$，故标准差

$$\sigma_{\hat{\pi}}=\frac{4\sqrt{p(1-p)}}{\sqrt{n}}=\frac{4\sqrt{0.7854\times 0.2146}}{\sqrt{n}}\approx\frac{1.642}{\sqrt{n}}.$$

  误差按 $1/\sqrt{n}$ 缩小：**想多得到一位有效数字，投掷次数要乘以 100**。

```python
# -*- coding: utf-8 -*-
""" 实验五：使用蒙特·卡罗方法计算圆周率。
在边长为 2 的正方形 [-1,1]x[-1,1] 内均匀随机投点，
落入单位圆 x^2+y^2<=1 的概率 = 圆面积/正方形面积 = pi/4，
所以 pi ≈ 4 * 圆内点数 / 总点数。
"""
import random


def estimate_pi(times):
```

```
11      hits = 0
12      for _ in range(times):
13          x = random.uniform(-1, 1)     # random.random() 得 [0,1)，uniform(-1,1) 得 [-1,1]
14          y = random.uniform(-1, 1)
15          if x * x + y * y <= 1:        # 比较平方，避免开方
16              hits += 1
17      return 4 * hits / times           # Python 3 中 / 是真除法
18
19
20  if __name__ == '__main__':
21      try:
22          times = int(input('请输入掷飞镖次数：'))
23          if times <= 0:
24              raise ValueError
25      except ValueError:
26          print('请输入正整数')
27      else:
28          print('圆周率近似值为：', estimate_pi(times))
```

6. 代码要点　第 15 行比较 $x^2+y^2\le 1$ 而不是 $\sqrt{x^2+y^2}\le 1$，两者等价且省去开方；第 17 行 `4 * hits / times` 中的 `/` 是真除法，结果为浮点数；第 21–28 行用 `try...except...else` 拒绝非正整数输入。

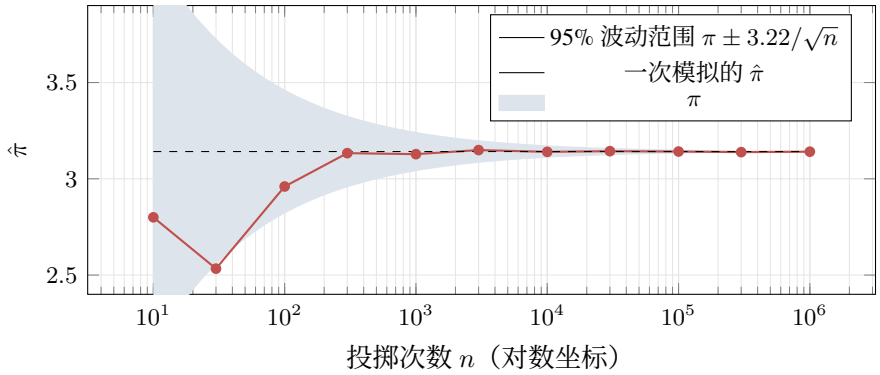


图 7　同一次模拟中 $\hat{\pi}$ 随投掷次数的变化（真实运行数据，随机种子 5）。看点：估计值始终在“喇叭口”内摆动，喇叭口宽度按 $1/\sqrt{n}$ 收窄。

最终答案

输入 100、10000、1000000，一次实际运行分别输出 3.24、3.114、3.142448。程序：$\hat{\pi}=4\times\dfrac{\text{圆内次数}}{\text{总次数}}$。

7. 结果检验　$n=10^6$ 时 $\sigma\approx 1.642/1000=0.0016$，95% 范围约 ± 0.0032；实测误差 $|3.142448-\pi|\approx 0.00086$，在范围内。$n=100$ 时 $\sigma\approx 0.16$，得到 3.24 也完全正常——并不是程序出错。

8. 解法二：只用四分之一圆　若用 `random.random()` 取 $x,y\in[0,1)$，则正方形面积 1、四分之一圆面积 $\pi/4$，比值相同，公式不变。**解法三（向量化）**：

```
import numpy as np
x, y = np.random.uniform(-1, 1, (2, 10**7))
print(4 * np.mean(x*x + y*y <= 1))
```

三种方法的数学本质完全相同；numpy 把循环交给 C 实现，10^7 次在一秒左右完成，而纯 Python 循环要慢数十倍。

易错点

1. **用 `random.randint(-1, 1)` 生成坐标**：只会得到 $-1, 0, 1$ 三个整数，落点只有 9 个格点，结果完全错误。必须用连续分布 `uniform` 或 `random`。
2. **期待“结果等于 3.14159”**：蒙特·卡罗给出的是带随机误差的估计；判断程序对不对，要看误差是否与 $1.64/\sqrt{n}$ 同数量级。
3. **整除写错**：若写成 `4 * hits // times`，结果只能是 2、3 这样的整数。

【实验六】猜数游戏

1. 考查知识点　选择结构嵌套与缩进；带 else 的 for 循环；条件表达式 `v1 if cond else v2`；`try...except...else`；`random.randint`。

2. 已知条件　随机数范围 $[1, \text{maxValue}]$，最多猜 maxTimes 次（程序取 10 和 3，均可改）；每次给出“太大了/太小了/猜对了”的提示。

3. 解题目标　猜对**提前结束**；次数用完仍未猜对，提示结束并公布答案。

4. 解题思路　“最多猜 k 次”是固定次数的重复，用 `for i in range(maxTimes)`；“猜对提前结束”用 `break`；“次数用完才执行的收尾”正好是 **for...else** 的语义——else 子句只在循环**没有被 break 打断、自然结束**时执行。输入非法时不能让程序崩溃，所以把 `int(input())` 放进 try。整体流程见图8。

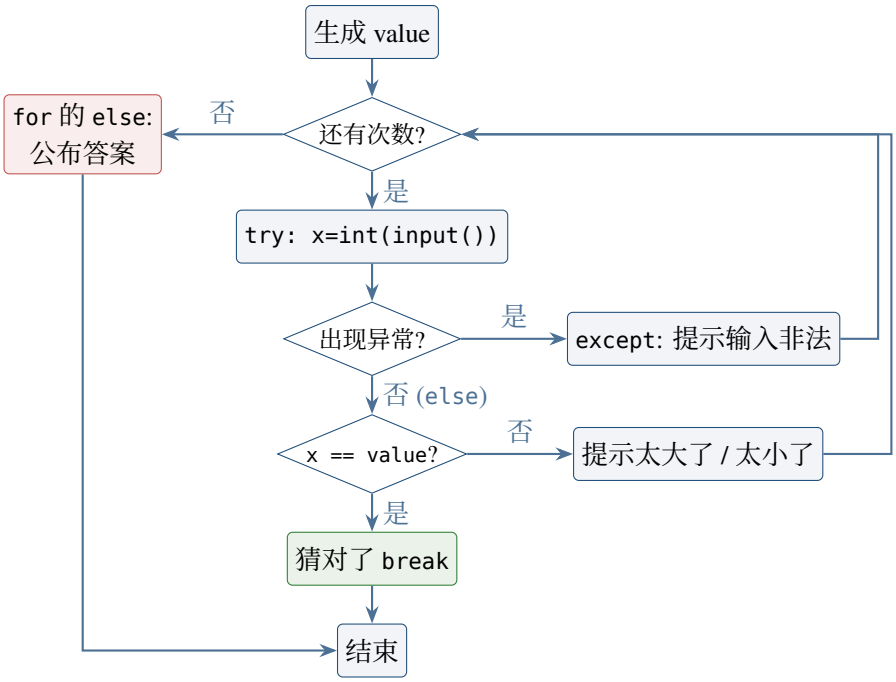


图 8　猜数游戏的控制流程。看点：只有“次数用完”这条路才会进入 for 的 else；猜对时 break 直接跳到结束，绕过 else。

```python
# -*- coding: utf-8 -*-
""" 实验六：猜数游戏。
系统生成随机数，用户最多猜 maxTimes 次，程序提示“太大了/太小了/猜对了”；
猜对提前结束，次数用完仍未猜对则提示游戏结束并给出正确答案。
涉及：for...else、条件表达式 value1 if cond else value2、
      try...except...else 约束用户输入。
"""
from random import randint


def guess_game(maxValue=10, maxTimes=3):
    value = randint(1, maxValue)              # 随机生成 [1, maxValue] 的整数
    for i in range(maxTimes):
        prompt = '开始猜吧：' if i == 0 else '再猜一次：'    # 条件表达式
        try:
            x = int(input(prompt))
        except ValueError:
            print('必须输入整数，而且必须介于 1 与', maxValue, '之间')
        else:                                 # 没有发生异常时执行
            if x == value:
                print('恭喜，猜对了！')
                break                         # break 后不会执行循环的 else
            elif x > value:
                print('太大了')
            else:
                print('太小了')
    else:                                     # 循环“自然结束”（次数用完）才执行
        print('次数用完，游戏结束。正确的数字是', value)


if __name__ == '__main__':
    guess_game(maxValue=10, maxTimes=3)
```

5. 语法要点逐条说明

- 第 14 行条件表达式：`'开始猜吧：' if i == 0 else '再猜一次：'`，先判断条件，真取左值、假取右值，是一个**表达式**，可以直接赋值。
- 第 15–19 行：`try` 中出现 `ValueError`（如输入 `abc`、`3.5`）时执行 except；**没有异常**时才执行 try 的 else。把比较大小的代码放在 else 中，可以保证 x 一定已经被成功赋值。
- 第 20–26 行是嵌套在 else 中的 if–elif–else，注意它比 try 多缩进一层。
- 第 27 行的 else 与第 13 行的 **`for` 对齐**，属于循环；若错误地与 if 对齐，含义就完全变了。

6. 一次运行追踪　（答案为 9，maxTimes=3）

```
开始猜吧：5        -> 太小了
再猜一次：abc      -> 必须输入整数，而且必须介于 1 与 10 之间（本次机会被消耗）
再猜一次：3        -> 太小了
次数用完，游戏结束。正确的数字是 9
```

最终答案

按上面的代码实现：for 循环控制次数，try...except...else 约束输入，if–elif–else 给出提示，break 提前结束，for...else 公布答案。

7. 结果检验　分别测试三条路径：①第一次就猜中（应输出“恭喜”且不公布答案）；②三次都猜错（应公布答案）；③夹杂非法输入（程序不崩溃）。三条路径都已实际运行通过。

8. 拓展思考：几次才“保证”猜中？　每次提示“太大/太小”能把候选区间砍掉一半左右，这就是**二分查找**。k 次猜测最多能区分 2^k-1 个数，所以范围 $1\sim N$ 至少需要 $\lceil\log_2(N+1)\rceil$ 次。$N=10$ 时需 4 次；本程序只给 3 次，说明即使策略最优也**不能保证**猜中——这是设计游戏难度时可以利用的结论。

易错点

1. **else 缩进错位**：把第 27 行的 else 缩进到与 if 对齐，就成了 if 的分支，每次猜错都会“公布答案”。判断规则：else 和谁对齐就属于谁。
2. **认为 break 后还会执行 else**：恰恰相反，for/while 的 else 只在没有 break 时执行。
3. **非法输入是否消耗次数**：本程序中会消耗（这是一种设计选择）。若不想消耗，可把输入放进内层 `while True` 循环，直到合法再 break（实验九的写法）。

【实验七】使用枚举法验证 6174 猜想

1. 考查知识点　卡普雷卡尔（Kaprekar）变换；`itertools.combinations`；`sorted`、`''.join`、`int`、`str`；while 循环与计数；枚举法。

2. 已知条件　对各位数字互不相同的四位数，记 $K(n)=$（各位数字降序排成的数）$-$（升序排成的数）。猜想：反复作用 K，至多 7 次得到 6174。

3. 解题目标　对**所有**满足条件的数验证：(a) 都能到达 6174；(b) 操作次数 ≤ 7。

4. 解题思路　**降低枚举量**：$K(n)$ 只与四个数字是哪几个有关，与它们原来的排列顺序无关（排序会抹掉顺序）。所以不必枚举 4536 个四位数，只需枚举从 $0\sim9$ 中选 4 个不同数字的**组合**，共 $\binom{10}{4}=210$ 种。`combinations('0123456789', 4)` 恰好生成它们。

5. 为什么 6174 是“终点”　$7641-1467=6174$，即 $K(6174)=6174$，它是 K 的**不动点**。另外，$K(n)$ 一定是 9 的倍数：设数字为 $a\ge b\ge c\ge d$，

$$K(n)=(1000a+100b+10c+d)-(1000d+100c+10b+a)=999(a-d)+90(b-c),$$

这也解释了为什么所有中间结果（3087、8352、6174……）的数字和都是 9 的倍数。

```python
# -*- coding: utf-8 -*-
""" 实验七：使用枚举法验证 6174 猜想（卡普雷卡尔常数）。
对任意各位数字不相同的 4 位数，用各位数字能组成的最大数减去最小数，
对差重复此操作，最终得到 6174，且操作次数不超过 7 次。

关键点：
  * 结果只取决于 4 个数字的“组合”，与排列顺序无关，所以用
    itertools.combinations('0123456789', 4) 枚举 C(10,4)=210 种数字组合即可；
  * 差可能不足 4 位（例如 999），必须用 zfill(4) 补成 '0999' 再继续，
    否则会把它当成 3 位数处理而得到错误结果。
"""
```

```python
from itertools import combinations


def kaprekar_steps(digits):
    """digits: 长度为 4 的数字字符串（或字符序列）。返回到达 6174 所需的操作次数。"""
    item = ''.join(digits)
    times = 0
    while True:
        big = int(''.join(sorted(item, reverse=True)))
        little = int(''.join(sorted(item)))
        difference = big - little
        times += 1
        if difference == 6174:
            return times
        if difference == 0:                    # 四位数字全相同时才会出现
            raise ValueError('四位数字全部相同，无法到达 6174')
        item = str(difference).zfill(4)        # 补足 4 位


if __name__ == '__main__':
    stat = {}
    for item in combinations('0123456789', 4):
        t = kaprekar_steps(item)
        stat[t] = stat.get(t, 0) + 1
    print('各位数字互不相同的 210 种数字组合，操作次数分布：', dict(sorted(stat.items())))
    print('最大操作次数：', max(stat))
    assert max(stat) <= 7
    print('验证通过：所有情况都能在 7 次以内得到 6174')

    # 拓展：枚举 1000~9999 中所有“不全相同”的 4 位数（允许部分数字重复）
    worst = max(kaprekar_steps(str(n)) for n in range(1000, 10000) if len(set(str(n))) >
    ↪  1)
    print('拓展：所有数字不全相同的 4 位数，最多需要', worst, '次')
    # 演示一个具体过程
    n, path = '3524', ['3524']
    while n != '6174':
        n = str(int(''.join(sorted(n, reverse=True))) - int(''.join(sorted(n)))).zfill(4)
        path.append(n)
    print('示例 3524 的过程：', ' -> '.join(path))
```

6. 代码要点　第 20–21 行：`sorted('3524')` 返回字符列表 `['2','3','4','5']`，`''.join` 拼回字符串，`int` 转为整数；`reverse=True` 得到降序。第 28 行 `zfill(4)` 在左侧补 0。第 26–27 行处理四位数字全相同（如 1111）时差为 0、永远到不了 6174 的情况，避免死循环。

7. 运行结果

```
各位数字互不相同的 210 种数字组合，操作次数分布： {1: 12, 2: 14, 3: 60, 4: 11, 5: 28, 6: 19, 7: 66}
最大操作次数： 7
验证通过：所有情况都能在 7 次以内得到 6174
拓展：所有数字不全相同的 4 位数，最多需要 7 次
示例 3524 的过程： 3524 -> 3087 -> 8352 -> 6174
```

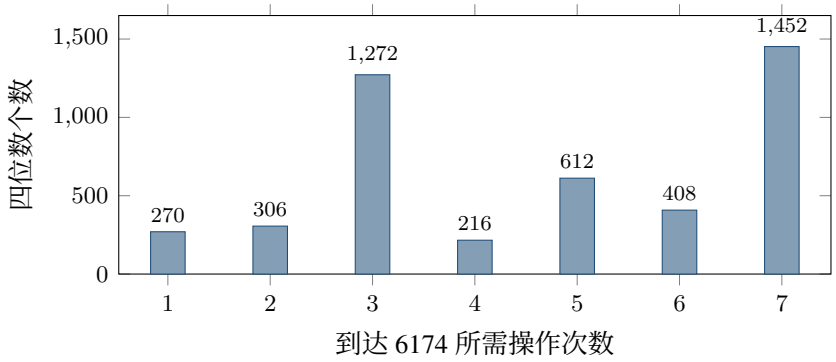


图 9　1000–9999 中各位数字互不相同的 4536 个四位数，按操作次数统计（程序枚举得到）。看点：没有任何一个数超过 7 次；需要恰好 7 次的最多，有 1452 个。

最终答案

枚举全部 210 种数字组合（等价于全部 4536 个各位不同的四位数），都能到达 6174，最大操作次数为 **7**，猜想在该范围内成立。

8. 结果检验　手算 3524：$5432-2345=3087$，$8730-0378=8352$，$8532-2358=6174$，共 3 次，与程序一致。另：图9中各柱之和 $270+306+1272+216+612+408+1452=4536=9\times8\times7\times9$ ✓（千位 9 种、其余位依次 9、8、7 种）。

9. 解法二：直接枚举 4536 个数　`for n in range(1000, 10000): if len(set(str(n))) == 4: ...`。`set(str(n))` 去重后长度为 4 即各位不同。结果相同但计算量约为解法一的 21 倍。两种方法比较：组合法利用了“结果只与数字集合有关”的**不变性**来缩小搜索空间，这是枚举法优化最常用的思想。

易错点

1. **差不足 4 位时不补零**：本题的 210 种起点在迭代中恰好从不出现小于 1000 的差（已枚举验证），但推广到“数字不全相同”的四位数时会出错：$2111-1112=999$，若当作三位数，下一步得 $999-999=0$；正确做法是视为 0999，$9990-0999=8991$，再经 3 步到达 6174。`zfill(4)` 让程序对推广情形也正确。
2. **对 1111 这类数死循环**：差为 0 后永远是 0，必须单独判断。
3. **计数从 0 还是从 1 开始**：本程序把“做一次减法”计为一次操作，6174 本身计 1 次（再做一次仍是 6174）。不同教材计法可能差 1，比较结论时要先统一口径。

【实验八】小明爬楼梯

1. 考查知识点　递推关系的建立；递推（迭代）与递归两种实现；序列解包；字典作缓存（记忆化）；递归的重复计算。

2. 已知条件　楼梯 $n=15$ 级，每步可上 1、2 或 3 级；不同的步伐序列算不同方法（如 1+2 与 2+1 不同）。

3. 解题目标　求方法总数 $f(15)$，并分别用递推法和递归法实现。

4. 解题思路（建立递推式）　按**最后一步**分类：最后一步上 1 级，之前要走完 $n-1$ 级；上 2 级，之前走完 $n-2$ 级；上 3 级，之前走完 $n-3$ 级。三类互不重叠且覆盖全部情况（加法原理），所以

$$f(n) = f(n-1) + f(n-2) + f(n-3) \quad (n \geq 4), \qquad f(1) = 1,\ f(2) = 2,\ f(3) = 4.$$

初值的来历：$f(1)$：{1}；$f(2)$：{1+1, 2}；$f(3)$：{1+1+1, 1+2, 2+1, 3}。（若约定 $f(0) = 1$ 表示“已站在顶上、什么都不做”这一种方法，则初值也可写成 $f(0) = 1, f(-1) = f(-2) = 0$。）

n	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$f(n)$	1	2	4	7	13	24	44	81	149	274	504	927	1705	3136	**5768**

```python
# -*- coding: utf-8 -*-
""" 实验八：小明爬楼梯。楼梯共 15 个台阶，一步最多上 3 个台阶，
求上楼方法总数。要求给出递推法和递归法两种代码。

递推关系：最后一步走 1、2 或 3 级，所以
    f(n) = f(n-1) + f(n-2) + f(n-3)   (n >= 4)
    f(1) = 1, f(2) = 2, f(3) = 4
"""
from functools import lru_cache


def climb_iterative(n):
    """ 递推法：只保留最近三项，用序列解包同步更新。"""
    a, b, c = 1, 2, 4                # f(1), f(2), f(3)
    if n <= 3:
        return (a, b, c)[n - 1]
    for _ in range(n - 3):
        a, b, c = b, c, a + b + c    # 序列解包：右边先全部算好再赋值
    return c


def climb_recursive(n):
    """ 递归法（朴素版）：直接翻译递推式，存在大量重复计算。"""
    if n == 1:
        return 1
    if n == 2:
        return 2
    if n == 3:
        return 4
    return climb_recursive(n - 1) + climb_recursive(n - 2) + climb_recursive(n - 3)


cache = {1: 1, 2: 2, 3: 4}           # 字典做“备忘录”


def climb_memo(n):
    """ 递归法（字典缓存版）：每个 n 只计算一次。"""
    if n not in cache:
```

```
39          cache[n] = climb_memo(n - 1) + climb_memo(n - 2) + climb_memo(n - 3)
40      return cache[n]
41
42
43  @lru_cache(maxsize=None)
44  def climb_lru(n):
45      """ 递归法（标准库缓存版）。"""
46      return {1: 1, 2: 2, 3: 4}[n] if n <= 3 else climb_lru(n - 1) + climb_lru(n - 2) +
        ↪  climb_lru(n - 3)
47
48
49  calls = 0
50
51
52  def _count_calls(n):
53      global calls
54      calls += 1
55      if n <= 3:
56          return (1, 2, 4)[n - 1]
57      return _count_calls(n - 1) + _count_calls(n - 2) + _count_calls(n - 3)
58
59
60  if __name__ == '__main__':
61      n = 15
62      print('递推法：', climb_iterative(n))
63      print('递归法：', climb_recursive(n))
64      print('字典缓存递归：', climb_memo(n))
65      print('lru_cache 递归：', climb_lru(n))
66      _count_calls(n)
67      print('朴素递归求 f(15) 共调用函数', calls, '次')
68      print('f(1)~f(15)：', [climb_iterative(k) for k in range(1, 16)])
```

5. 递推法（第 12–19 行）与序列解包　只保留最近三项 $a, b, c = f(k-2), f(k-1), f(k)$。第 18 行 `a, b, c = b, c, a + b + c` 的执行顺序是：**先**把右边三个值全部算出组成元组 $(b, c, a+b+c)$，**再**依次赋给左边。若拆成三条普通赋值 `a = b; b = c; c = a + b + c`，第三条用到的 a、b 已经被改掉，结果就错了。

6. 递归法与重复计算　第 22–30 行把递推式直接翻译成函数。正确但慢：画出 $f(5)$ 的调用树（图10）可见 $f(3)$、$f(2)$ 被反复计算。实测求 $f(15)$ 共调用函数 **3313** 次，而真正不同的子问题只有 15 个。第 36–40 行用字典 `cache` 记住算过的结果（**记忆化递归**），每个 n 只算一次；第 44 行的 `functools.lru_cache` 是标准库提供的同一功能。

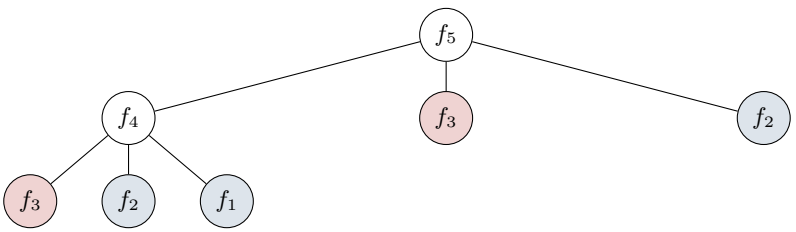


图 10　朴素递归计算 $f(5)$ 的调用树（f_3, f_2, f_1 为递归出口）。看点：红色的 f_3 出现两次，f_2 出现两次；n 越大，重复越多，调用次数近似按 1.84^n 增长。

7. 解法三：组合计数（独立验证）　设用了 a 个 1 级步、b 个 2 级步、c 个 3 级步，则 $a+2b+3c=15$；这些步的不同排列数是多重组合数 $\dfrac{(a+b+c)!}{a!\,b!\,c!}$。对全部 27 组非负整数解求和，程序算得

$$\sum_{a+2b+3c=15} \frac{(a+b+c)!}{a!\,b!\,c!} = 5768.$$

例如 $(a,b,c)=(0,0,5)$ 贡献 1，$(15,0,0)$ 贡献 1，$(1,1,4)$ 贡献 $\frac{6!}{1!1!4!}=30$。这一方法不依赖递推式，与前两种结果吻合，是很强的交叉验证。

最终答案

共有 **5768** 种上楼方法。递推法、朴素递归、字典记忆化递归、lru_cache 递归四种实现输出一致。

8. 结果检验　小规模手工枚举：$n=4$ 时 1+1+1+1、1+1+2、1+2+1、2+1+1、2+2、1+3、3+1 共 7 种，等于 $f(4)=1+2+4$ ✓。另外，第三部分第十章引用的 UC Berkeley CS 61A 公开讨论题给出 `count_k(10, 3) == 274`，与表中 $f(10)=274$ 一致。

易错点

1. **初值写成 $f(3)=3$**：漏掉了“一步上 3 级”这一种。判断方法：初值也要按“最后一步”分类数一遍。
2. **把序列解包拆成顺序赋值**：见第 5 条，会使用被覆盖的旧值。
3. **朴素递归求大 n**：$n=35$ 左右就要等很久；n 很大时还会超过默认递归深度（约 1000）抛 `RecursionError`，此时应改用递推。

【实验九】抓狐狸游戏

1. 考查知识点　用列表对问题建模；随机数；选择与循环；for...else；用异常处理约束输入；（拓展）状态集合与必胜策略。

2. 已知条件　一排 $n=5$ 个洞口（程序编号 $0\sim4$）；狐狸每天恰好移动到一个**相邻**洞口（在两端时只能向内跳）；玩家每天打开一个洞口。

3. 解题目标　编程模拟游戏：抓到则结束；限定天数内没抓到则提示失败。

4. 建模思路　用长度为 5 的列表 `positions` 表示洞口，值为 1 的位置有狐狸。每天的流程：读入合法编号 → 检查 `positions[x]` → 没抓到就让狐狸移动到相邻洞口（更新两个位置的值）。最大天数用 for 循环，失败收尾用 for...else。

```python
# -*- coding: utf-8 -*-
""" 实验九：抓狐狸游戏。
一排 n=5 个洞口，狐狸开始时在某个洞口。玩家每天打开一个洞口，
有狐狸就抓到了；否则第二天再来，但狐狸会在玩家来之前跳到相邻洞口。
涉及：用列表建模、选择与循环、for...else、异常处理约束用户输入。
"""
from random import randrange, choice


def catch_me(n=5, maxStep=10):
    """ 交互式游戏：n 个洞口，最多抓 maxStep 次。"""
    positions = [0] * n                          # 列表建模：1 表示狐狸在这个洞口
    oldPos = randrange(0, n)
```

```
14      positions[oldPos] = 1
15      for day in range(1, maxStep + 1):
16          while True:                                 # 用异常处理约束输入，直到合法为止
17              try:
18                  x = int(input(f'第 {day} 天，请输入要打开的洞口编号（0~{n - 1}）：'))
19                  assert 0 <= x < n, '洞口编号超出范围'
20                  break
21              except ValueError:
22                  print('必须输入整数')
23              except AssertionError as e:
24                  print(e)
25          if positions[x] == 1:
26              print('成功，抓到狐狸了！')
27              break
28          print('今天没有抓到。')
29          # 狐狸跳到相邻洞口：两端只能向内跳
30          if oldPos == 0:
31              newPos = 1
32          elif oldPos == n - 1:
33              newPos = n - 2
34          else:
35              newPos = oldPos + choice((-1, 1))
36          positions[oldPos], positions[newPos] = 0, 1
37          oldPos = newPos
38      else:                                           # 没有 break（没抓到）时执行
39          print('放弃吧，你这样乱试是没有希望的。狐狸最后在', oldPos, '号洞口')
40
41
42  def guaranteed(sequence, n=5):
43      """ 检验一个打开顺序能否保证抓到狐狸：维护狐狸“可能所在位置”的集合。"""
44      possible = set(range(n))
45      for x in sequence:
46          possible.discard(x)                         # 打开 x 没抓到 => 狐狸不在 x
47          if not possible:
48              return True
49          possible = {p + d for p in possible for d in (-1, 1) if 0 <= p + d < n}
50      return False
51
52
53  if __name__ == '__main__':
54      # 先演示必胜策略的验证（0 号编号下即 1,2,3,3,2,1；1 号编号下即 2,3,4,4,3,2）
55      print('策略 [1,2,3,3,2,1] 是否必胜：', guaranteed([1, 2, 3, 3, 2, 1]))
56      print('策略 [1,2,3,1,2,3] 是否必胜：', guaranteed([1, 2, 3, 1, 2, 3]))
57      catch_me()
```

5. 代码要点

- 第 16–24 行：内层 `while True` 不断读取输入，`int()` 失败抛 `ValueError`；`assert 0 <= x < n, '...'` 不成立时抛 `AssertionError` 并携带提示文字；合法时 break 跳出内层循环。这样非法输入**不会**消耗天数。
- 第 29–35 行：狐狸在 0 号只能跳到 1 号，在 4 号只能跳到 3 号，否则随机 `choice((-1, 1))`。

- 第 36 行用序列解包同时修改两个位置：旧位置置 0，新位置置 1。

6. 拓展：有没有一定能抓到狐狸的打法？　换个角度建模：不追踪狐狸“在哪”，而追踪狐狸“**可能在哪**”的集合 P。开始时 $P=\{0,1,2,3,4\}$；打开洞口 x 没抓到，就把 x 从 P 中删去；过夜后 P 变为所有相邻位置的集合。若某天删完后 $P=\varnothing$，说明无论狐狸怎么跳，这天之前一定已被抓到。第 42–50 行的 `guaranteed()` 正是这样检验的。图11展示了打法 1,2,3,3,2,1（程序编号；即人类编号 2,3,4,4,3,2）的检验过程。

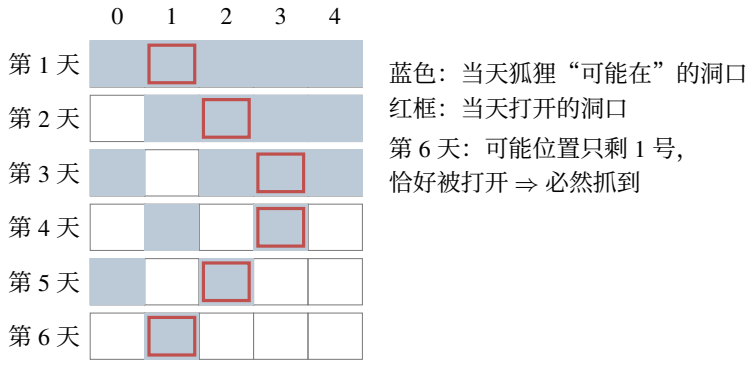


图 11　必胜打法 1,2,3,3,2,1 下“狐狸可能位置”的变化（程序实际计算结果）。看点：每天的可能位置在奇、偶编号间交替，第 3 天后只剩奇数位置，后三天从另一侧把它们“扫”掉。

7. 为什么这个打法一定成功（奇偶性论证）　狐狸每天位置改变 ±1，所以位置的奇偶性每天交替。**情形一**：第 1 天狐狸在奇数洞（1 或 3）。前三天依次打开 1,2,3，打开位置每天右移 1；令 $d_t=p_t-x_t$（狐狸位置减打开位置），则 $d_{t+1}-d_t\in\{0,-2\}$，$d_1\in\{0,2\}$ 为偶数且非负，而狐狸不可能越过右端 4 号逃到更右边，故前三天内必有某天 $d_t=0$，即被抓。**情形二**：第 1 天在偶数洞，则第 4 天必在奇数洞（1 或 3），后三天打开 3,2,1，由对称的论证同样必被抓到。程序穷举还说明：**长度不超过 5 的打法都不能保证成功**，长度为 6 的必胜打法恰有 4 种：1,2,3,1,2,3；1,2,3,3,2,1；3,2,1,1,2,3；3,2,1,3,2,1。

最终答案

按上面的代码实现游戏（列表建模 + 异常处理约束输入 + for...else）。拓展结论：5 个洞口时，按 2,3,4,4,3,2（人类编号）依次打开，最多 6 天必能抓到狐狸，且 6 天是最少的。

8. 结果检验　以测试输入 1,2,3,3,2,1 运行，程序报告“策略必胜 True”；对全部 $5^5=3125$ 种 5 天打法穷举，无一必胜，证明 6 天是下界。

易错点

1. **狐狸在端点越界**：0 号洞若也随机 ±1，可能得到 -1；Python 中 `positions[-1]` 不会报错而是访问最后一个元素，导致“狐狸瞬移到 4 号”这种隐蔽错误。必须单独处理两端。
2. **忘记把旧位置清零**：只写 `positions[newPos] = 1`，列表中会出现两只“狐狸”。
3. **用 assert 做输入校验的局限**：以 `python -O` 运行时 assert 会被整体忽略。课堂练习可以使用，正式程序宜用 `if ...: raise ValueError(...)`。

【实验十】自定义类模拟三维向量及其运算

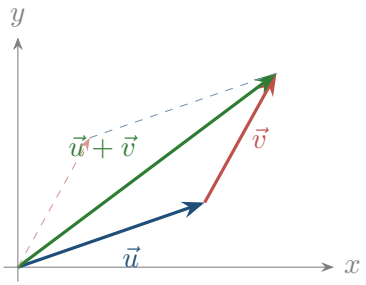
1. 考查知识点　类的定义与实例化；私有数据成员（双下划线与名称改写）；特殊方法（运算符重载）；`@property` 属性；`NotImplemented` 协议。

2. 已知与目标　实现三维向量类：$\vec{u}\pm\vec{v}$、$\vec{v}\cdot k$、$\vec{v}/k$ 用运算符 `+` `-` `*` `/` 完成；长度 $|\vec{v}|$ 用属性访问。

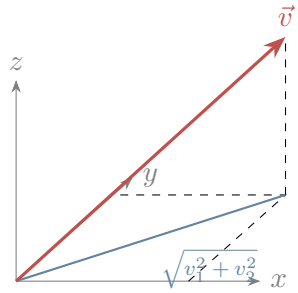
3. 数学定义　设 $\vec{u}=(u_1,u_2,u_3)$，$\vec{v}=(v_1,v_2,v_3)$，$k\in\mathbb{R}$：

$$\vec{u}\pm\vec{v}=(u_1\pm v_1,\ u_2\pm v_2,\ u_3\pm v_3),\qquad k\vec{v}=(kv_1,kv_2,kv_3),$$
$$\frac{\vec{v}}{k}=\left(\frac{v_1}{k},\frac{v_2}{k},\frac{v_3}{k}\right)\ (k\neq 0),\qquad |\vec{v}|=\sqrt{v_1^2+v_2^2+v_3^2}.$$

长度公式来自两次勾股定理：先在 xy 平面求 $\sqrt{v_1^2+v_2^2}$，再与 v_3 组成直角三角形（图12右）。



平行四边形法则（按分量相加）



$|\vec{v}|=\sqrt{v_1^2+v_2^2+v_3^2}$

图 12　左：向量加法即按分量相加；右：三维长度由两次勾股定理得到。看点：运算符重载只是把这些数学定义“翻译”成方法。

4. 解题思路：运算符如何变成方法调用　Python 遇到 `v1 + v2` 时，实际调用 `v1.__add__(v2)`。对应关系如下：

表达式	实际调用	本题含义	说明
`u + v`	`u.__add__(v)`	向量加	返回新向量
`u - v`	`u.__sub__(v)`	向量减	
`v * k`	`v.__mul__(k)`	数乘	
`k * v`	`v.__rmul__(k)`	数乘（左乘）	`int.__mul__` 不认识向量时才轮到它
`v / k`	`v.__truediv__(k)`	数除	Python 3 中不是 `__div__`
`v.length`	`property` 的 getter	长度	像访问数据一样，不加括号

```python
# -*- coding: utf-8 -*-
""" 实验十：自定义类模拟三维向量及其运算。
要求：向量 + 向量、向量 - 向量（运算符 +、-），
      向量 * 标量、向量 / 标量（运算符 *、/），
      向量长度用属性（property）实现。
"""
import math


class Vector3:
```

```
    def __init__(self, x=0, y=0, z=0):
        self.__x = x                        # 双下划线开头：私有数据成员（名称改写为 _Vector3__x）
        self.__y = y
        self.__z = z

    # 只读属性，便于外部读取分量
    @property
    def x(self):
        return self.__x

    @property
    def y(self):
        return self.__y

    @property
    def z(self):
        return self.__z

    def __add__(self, other):               # v1 + v2
        if not isinstance(other, Vector3):
            return NotImplemented
        return Vector3(self.__x + other.__x, self.__y + other.__y, self.__z + other.__z)

    def __sub__(self, other):               # v1 - v2
        if not isinstance(other, Vector3):
            return NotImplemented
        return Vector3(self.__x - other.__x, self.__y - other.__y, self.__z - other.__z)

    def __mul__(self, k):                   # v * k
        if not isinstance(k, (int, float)):
            return NotImplemented
        return Vector3(self.__x * k, self.__y * k, self.__z * k)

    __rmul__ = __mul__                      # 支持 k * v

    def __truediv__(self, k):               # v / k （Python 3 中 / 对应 __truediv__）
        if not isinstance(k, (int, float)):
            return NotImplemented
        if k == 0:
            raise ZeroDivisionError('向量不能除以 0')
        return Vector3(self.__x / k, self.__y / k, self.__z / k)

    @property
    def length(self):                       # 用属性实现：v.length 不加括号
        return math.sqrt(self.__x ** 2 + self.__y ** 2 + self.__z ** 2)

    def __eq__(self, other):
        return isinstance(other, Vector3) and (self.__x, self.__y, self.__z) ==
        ↪  (other.__x, other.__y, other.__z)
```

```
59
60      def __repr__(self):
61          return f'Vector3({self.__x}, {self.__y}, {self.__z})'
62
63
64  if __name__ == '__main__':
65      v1 = Vector3(3, 4, 5)
66      v2 = Vector3(5, 6, 7)
67      print('v1 =', v1, '  v2 =', v2)
68      print('v1 + v2 =', v1 + v2)
69      print('v1 - v2 =', v1 - v2)
70      print('v1 * 3  =', v1 * 3)
71      print('3 * v1  =', 3 * v1)
72      print('v1 / 2  =', v1 / 2)
73      print('|v1| =', v1.length)
74      print('|Vector3(1,2,2)| =', Vector3(1, 2, 2).length)
75      try:
76          v1.length = 10                    # 只读属性不能赋值
77      except AttributeError as e:
78          print('给只读属性赋值会报错：', type(e).__name__)
79      try:
80          print(v1.__x)
81      except AttributeError:
82          print('外部不能直接访问 v1.__x（名称已被改写为 _Vector3__x）')
```

5. 私有成员与属性　第 12–14 行以双下划线开头的 `self.__x` 会被解释器**改写名称**为 `self._Vecto」r3__x`，所以类外写 `v1.__x` 会报 `AttributeError`——这是 Python 实现“私有”的方式（约定加名称改写，并非绝对禁止访问）。第 53–55 行用 `@property` 把 `length` 方法包装成只读属性：读 `v.length` 时自动计算，赋值 `v.length = 10` 会报错，从而保证长度永远与分量一致。

6. 运行结果

```
v1 = Vector3(3, 4, 5)   v2 = Vector3(5, 6, 7)
v1 + v2 = Vector3(8, 10, 12)     v1 - v2 = Vector3(-2, -2, -2)
v1 * 3  = Vector3(9, 12, 15)     3 * v1  = Vector3(9, 12, 15)
v1 / 2  = Vector3(1.5, 2.0, 2.5)
|v1| = 7.0710678118654755        |Vector3(1,2,2)| = 3.0
```

最终答案

按上述 Vector3 类实现：`__add__`/`__sub__` 实现向量加减，`__mul__`（及 `__rmul__`）/`__truediv__` 实现与标量的乘除，`@property length` 实现长度属性。

7. 结果检验　$|(3,4,5)| = \sqrt{9+16+25} = \sqrt{50} = 5\sqrt{2} \approx 7.0711$ ✓；$|(1,2,2)| = \sqrt{1+4+4} = 3$ ✓；检查运算律：$(\vec{u}+\vec{v})-\vec{v}$ 应回到 $\vec{u}$；$(\vec{v}\times 3)/3$ 的各分量应等于原分量（浮点意义下）。

易错点

1. **写成 `__div__`**：这是 Python 2 的名字，Python 3 中 `/` 调用 `__truediv__`，`//` 调用 `__floordiv__`。写错时会报 `TypeError: unsupported operand type(s)`。
2. **在运算方法里修改 self 并返回 self**：`v1 + v2` 之后 `v1` 本身被改变，违背数学含义。运算方法应返回**新对象**（`+=` 这类原地运算才对应 `__iadd__`）。

3. **属性加括号**：`v.length()` 会报 `TypeError: 'float' object is not callable`，因为 `v.length` 已经是一个数。
4. **只写 `__mul__` 却想用 `3 * v`**：左操作数是 int，需要 `__rmul__` 才能支持。

【实验十一】蒙蒂霍尔悖论游戏

1. 考查知识点　条件概率与全概率公式；字典推导式与 `dict.items()`；集合差运算；`assert`；循环结构；随机模拟验证概率。

2. 已知条件（标准规则）　①汽车等可能地在 3 扇门之一后面；②主持人知道答案，**总是**打开一扇玩家没选、且后面是山羊的门；③若这样的门有两扇（玩家首选就是汽车），主持人等可能地任选其一；④主持人总会提供改选机会。

3. 解题目标　编程模拟游戏，并判断“改选”与“坚持”哪个更好。

4. 解题思路　程序部分：用字典 {门号：奖品} 表示三扇门；主持人可开的门 = {山羊门} − {玩家所选门}（集合差）；改选后的门 = {1,2,3} − {首选门, 主持人开的门}。概率部分：关键在于主持人的行为**不是随机的**，他“必定避开汽车”这件事携带了信息。

5. 概率推导（解法一：分情况/全概率公式）　设玩家首选 1 号门，C 为汽车所在门：

- $C=1$（概率 $1/3$）：坚持赢，改选输；
- $C=2$（概率 $1/3$）：主持人只能开 3 号，改选到 2 号，赢；
- $C=3$（概率 $1/3$）：主持人只能开 2 号，改选到 3 号，赢。

故 $P(\text{改选赢})=\frac{1}{3}\cdot 0+\frac{1}{3}\cdot 1+\frac{1}{3}\cdot 1=\frac{2}{3}$，$P(\text{坚持赢})=\frac{1}{3}$。一句话：**改选赢** $\Longleftrightarrow$ **首选是山羊**，而首选是山羊的概率是 $2/3$。

6. 解法二：贝叶斯公式（看到“主持人开了 3 号门”之后）　记 H_3 为“主持人开 3 号门”。由规则 $P(H_3\mid C=1)=\frac{1}{2}$，$P(H_3\mid C=2)=1$，$P(H_3\mid C=3)=0$，全概率公式给出 $P(H_3)=\frac{1}{3}(\frac{1}{2}+1+0)=\frac{1}{2}$，于是

$$P(C=2\mid H_3)=\frac{P(H_3\mid C=2)P(C=2)}{P(H_3)}=\frac{1\cdot\frac{1}{3}}{\frac{1}{2}}=\frac{2}{3},\qquad P(C=1\mid H_3)=\frac{\frac{1}{2}\cdot\frac{1}{3}}{\frac{1}{2}}=\frac{1}{3}.$$

概率树见图13。两种解法的联系：解法一算的是“改选策略”的总体胜率，解法二算的是看到具体开门结果后的条件胜率；在标准规则下两者都是 $2/3$（第三部分第十章会看到规则改变时两者不再相同）。

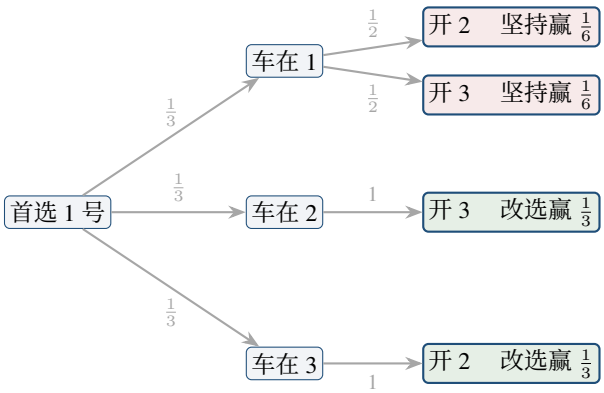


图 13　蒙蒂霍尔问题的概率树（玩家首选 1 号门）。看点：绿色叶子（改选赢）概率合计 $2/3$；只看“开 3 号门”的两片叶子，改选赢 $\frac{1}{3}$ 对坚持赢 $\frac{1}{6}$，条件概率仍是 $2:1$。

```python
# -*- coding: utf-8 -*-
""" 实验十一：蒙蒂霍尔悖论游戏。
3 道门，一辆汽车、两只山羊。玩家选一扇门，主持人（知道门后是什么）
打开另一扇有山羊的门，并问玩家是否改选。
涉及：字典方法、集合运算、assert 断言、循环结构。
"""
from random import randrange, choice


def init():
    """ 返回字典 {门号: 奖品}，门号为 1, 2, 3。 """
    result = {i: 'goat' for i in range(1, 4)}
    result[randrange(1, 4)] = 'car'
    return result


def startGame():
    doors = init()
    while True:
        try:
            firstDoorNum = int(input('请选择一扇门（1、2、3）：'))
            assert 1 <= firstDoorNum <= 3
            break
        except (ValueError, AssertionError):
            print('门号必须是 1、2、3 之一')
    # 主持人要打开的门：不是玩家选的门，且后面是山羊（集合运算 + 字典方法）
    goatDoors = {k for k, v in doors.items() if v == 'goat'}
    hostCandidates = goatDoors - {firstDoorNum}
    goatDoorNum = choice(sorted(hostCandidates))
    assert doors[goatDoorNum] == 'goat' and goatDoorNum != firstDoorNum
    print(f'主持人打开了 {goatDoorNum} 号门，后面是山羊。')
    secondDoorNum = ({1, 2, 3} - {firstDoorNum, goatDoorNum}).pop()
    change = input(f'你想改选 {secondDoorNum} 号门吗？(y/n)：').strip().lower()
    finalDoorNum = secondDoorNum if change == 'y' else firstDoorNum
    if doors[finalDoorNum] == 'goat':
        print('很遗憾，你得到了山羊（输）。')
    else:
        print('恭喜，你得到了汽车（赢）！')
    print('各门后面分别是：', doors)


def simulate(trials=100000):
    """ 统计模拟：比较“坚持”与“改选”两种策略的获胜频率。"""
    stay_win = switch_win = 0
    for _ in range(trials):
        doors = init()
        first = randrange(1, 4)
        host = choice(sorted({k for k, v in doors.items() if v == 'goat'} - {first}))
        other = ({1, 2, 3} - {first, host}).pop()
        stay_win += doors[first] == 'car'
```

```
51          switch_win += doors[other] == 'car'
52      return stay_win / trials, switch_win / trials
53
54
55  if __name__ == '__main__':
56      stay, switch = simulate()
57      print(f'模拟 100000 次：坚持原选择获胜频率 {stay:.4f}，改选获胜频率 {switch:.4f}')
58      while True:
59          startGame()
60          if input('再玩一次吗？(y/n)：').strip().lower() != 'y':
61              break
```

7. 代码要点　第 12 行字典推导式先把三扇门都设为山羊，第 13 行随机放一辆车。第 27 行用 `doors.items()` 找出所有山羊门，第 28 行集合差去掉玩家选的门，得到主持人的候选门（1 扇或 2 扇），第 29 行随机选一扇。第 30 行 `assert` 是“自检”：如果逻辑有错导致主持人开出汽车或开了玩家的门，程序立即报错。第 32 行用集合差求出唯一剩下的门，`pop()` 取出该元素。

8. 运行结果

```
模拟 100000 次：坚持原选择获胜频率 0.3308，改选获胜频率 0.6692
请选择一扇门（1、2、3）：1
主持人打开了 3 号门，后面是山羊。
你想改选 2 号门吗？(y/n)：y
很遗憾，你得到了山羊（输）。
各门后面分别是： {1: 'car', 2: 'goat', 3: 'goat'}
```

最终答案

按上述代码实现游戏。结论：在标准规则下，改选获胜概率为 $\frac{2}{3}$，坚持为 $\frac{1}{3}$，应当改选；10 万次模拟得 0.6692 与 0.3308，与理论一致。

9. 结果检验　两种策略的胜率之和应为 1（每局恰有一种策略赢）：$0.3308 + 0.6692 = 1$ ✓。10^5 次模拟时频率的标准差约 $\sqrt{\frac{(2/3)(1/3)}{10^5}} \approx 0.0015$，实测偏差 $0.6692 - 0.6667 = 0.0025$，在 2 个标准差以内，属于正常。单局“改选却输了”（上面的示例）也完全正常，概率只描述大量重复的规律。

易错点

1. **“只剩两扇门，所以各 50%”**：错在把“剩下两种可能”当成“等可能”。主持人开门受“必须避开汽车”约束，他的行为与汽车位置相关，所以剩下两扇门的概率被重新分配为 $1/3$ 与 $2/3$。
2. **主持人随机开门（可能开出汽车）**：那是另一个问题。若主持人不知情、只是碰巧开出山羊，则条件下改选与坚持都是 $1/2$。编程时第 27–28 行必须排除汽车门，第 30 行断言正是用来守住这个前提。
3. **玩家首选汽车时主持人总开固定的门**：会改变条件概率（见第三部分第十章哈佛 Stat 110 的“偏好开 2 号门”变式），模拟标准问题时应在两扇山羊门中随机选。

【实验十二】凯撒加密算法原理与实现

1. 考查知识点　标准库 string 中的字母表常量；切片；`str.maketrans()` 与 `str.translate()`；取模运算；`ord()`/`chr()`。

2. 已知条件　明文字符串 text、整数密钥 k（可为负或大于 26）。

3. 解题目标　每个英文字母在字母表中循环后移 k 位，大小写各自循环，非字母字符原样保留，输出密文。

4. 数学模型　把字母 a–z 编号为 $0 \sim 25$，加密与解密为

$$E_k(x) = (x + k) \bmod 26, \qquad D_k(y) = (y - k) \bmod 26.$$

因为 $D_k(E_k(x)) = (x + k - k) \bmod 26 = x$，解密就是用 $-k$ 再加密一次。Python 的 `%` 对负数也返回 $[0, 26)$ 内的结果（如 `-3 % 26 == 23`），所以负密钥可以直接处理。

5. 解题思路　**解法一（查表）**：构造“原字母表 → 移位后字母表”的映射表，一次性替换。移位后的字母表用切片得到：`lower[k:] + lower[:k]` 把前 k 个字母挪到末尾（图14）。**解法二（逐字符计算）**：对每个字符按公式计算。

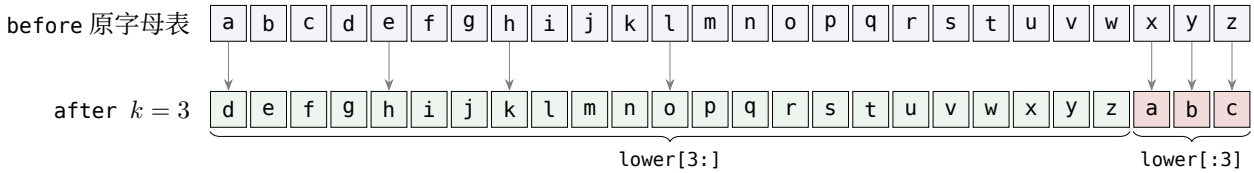


图 14　密钥 $k = 3$ 时的字母映射。看点：用两个切片拼接就实现了“循环”移位，x、y、z 绕回到 a、b、c。

```python
# -*- coding: utf-8 -*-
""" 实验十二：凯撒加密算法原理与实现。
输入字符串和整数密钥 k，把每个英文字母循环后移 k 位（大小写分别循环），
非字母字符保持不变，输出密文。
涉及：标准库 string、切片、str.maketrans() 与 str.translate()。
"""
import string


def caesar(text, k):
    lower = string.ascii_lowercase              # 'abcdefghijklmnopqrstuvwxyz'
    upper = string.ascii_uppercase
    k = k % 26                                  # 密钥可以是负数或大于 26 的数
    before = lower + upper
    after = lower[k:] + lower[:k] + upper[k:] + upper[:k]    # 切片实现循环移位
    table = str.maketrans(before, after)        # 构造映射表
    return text.translate(table)


def caesar_by_ord(text, k):
    """ 对照写法：用 ord/chr 与取模运算逐字符处理。"""
    out = []
    for ch in text:
        if 'a' <= ch <= 'z':
```

```
25              out.append(chr((ord(ch) - ord('a') + k) % 26 + ord('a')))
26          elif 'A' <= ch <= 'Z':
27              out.append(chr((ord(ch) - ord('A') + k) % 26 + ord('A')))
28          else:
29              out.append(ch)
30      return ''.join(out)
31
32
33  if __name__ == '__main__':
34      text = input('请输入要加密的字符串：')
35      try:
36          k = int(input('请输入整数密钥：'))
37      except ValueError:
38          print('密钥必须是整数')
39      else:
40          cipher = caesar(text, k)
41          print('加密结果：', cipher)
42          assert cipher == caesar_by_ord(text, k)
43          print('解密验证：', caesar(cipher, -k))   # 用 -k 再加密一次即为解密
```

6. 代码要点　第 13 行先把 k 化到 $0 \sim 25$；第 15 行一次拼出小写、大写两段移位字母表；第 16 行 `str.maketrans(before, after)` 要求两个字符串等长，生成“字符编码 → 字符编码”的字典；第 17 行 `translate` 按表替换，表中没有的字符（空格、标点、中文）保持不变。第 20–30 行的逐字符写法用于交叉验证。

最终答案

输入 `Hello, World! xyz`，密钥 3，输出 `Khoor, Zruog! abc`；用密钥 -3 再加密即还原明文。

7. 结果检验　①逐个核对：H(7)→(10)K，e(4)→(7)h，x(23)→(26 mod 26=0)a ✓；②查表法与 `ord/chr` 法结果相同（代码中有断言）；③ `caesar(caesar(t, k), -k) == t` 对任意 k 成立。

8. 方法比较　查表法先建表再一次替换，`translate` 在 C 层实现，长文本时更快，且天然不会“重复替换”；逐字符法直接对应数学公式，更容易推广到仿射密码 $E(x) = (ax + b) \bmod 26$ 等变体。

易错点

1. **用 replace 逐个字母替换**：

   ```
   for a, b in zip(before, after):
       text = text.replace(a, b)     # 错误示范
   ```

   $k = 1$ 时先把 a 换成 b，下一轮又把所有 b（包括刚换出来的）换成 c……最终所有字母都被推到同一端。原因是替换是**顺序**进行的，后面的替换作用在前面的结果上。`translate` 是对原文**同时**映射，不存在这个问题。
2. **没有取模**：$k = 29$ 时切片 `lower[29:]` 为空，映射表长度不对，`maketrans` 报错；应先 `k % 26`。
3. **大小写混在一个 52 字母表里循环**：会把 z 移成 A，破坏大小写。大小写应各自循环。

【实验十三】打字练习成绩评定

1. 考查知识点　`zip()` 的配对与截断；生成器表达式；`sum()`；`round()`；`isinstance()`；参数合法性检查。

2. 已知条件　原文 origin、用户输入 userInput；要求 $\text{len}(\text{userInput}) \le \text{len}(\text{origin})$；逐位置比较。

3. 解题目标　成绩 $= \dfrac{\text{对应位置相同的字符数}}{\text{len}(\text{origin})} \times 100$，保留 2 位小数。注意分母是**原文长度**，没打完的部分自然算错。

4. 解题思路　`zip(origin, userInput)` 把两串按位置配成 `(o, u)` 对，较短者结束时停止（图15）；生成器表达式 `1 for o, u in ... if o == u` 对每个相同位置产生一个 1，`sum` 加起来就是正确字符数。生成器表达式不会先建出整张列表，逐个产生、逐个累加，省内存。

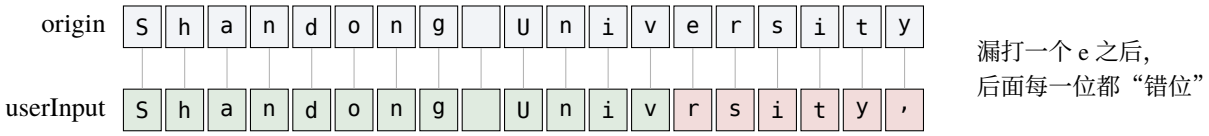


图 15　zip 逐位置配对（绿：相同，红：不同）。看点：按位置比较的规则下，一次漏字会让后面全部判错——这是本题评分规则的固有特点，而不是程序错误。

```python
# -*- coding: utf-8 -*-
""" 实验十三：打字练习成绩评定。
origin 为原始内容，userInput 为用户输入；要求 len(userInput) <= len(origin)。
对应位置字符一致记为正确。成绩 = 正确字符数 / 原始字符串长度，
按百分制输出，保留 2 位小数。
涉及：zip()、sum()、round()、isinstance()、生成器表达式。
"""


def rate(origin, userInput):
    if not (isinstance(origin, str) and isinstance(userInput, str)):
        print('两个参数都必须是字符串')
        return None
    if len(origin) == 0:
        print('原始内容不能为空')
        return None
    if len(userInput) > len(origin):
        print('输入内容长度不能大于原始内容长度')
        return None
    # 生成器表达式：zip 按位置配对，在较短序列结束时停止
    right = sum(1 for o, u in zip(origin, userInput) if o == u)
    return round(right / len(origin) * 100, 2)


if __name__ == '__main__':
    origin = 'Shandong University, School of Control Science and Engineering.'
    print('原始内容：', origin)
    userInput = input('请照着输入：')
    score = rate(origin, userInput)
```

```
    if score is not None:
        print(f'成绩：{score:.2f} 分')
```

5. 推导与运行　原文 `Shandong University, School of Control Science and Engineering.` 共 63 个字符。

- 输入 `Shandong University, School`（27 字符，全对）：$\frac{27}{63} \times 100 = 42.857\ldots \to$ **42.86**。
- 输入 `Shandong Univrsity, School`（漏一个 e，26 字符）：前 13 位相同；从第 14 位起整体错位，之后只有第 25 位碰巧相同（两边都是 o），共 14 位相同：$\frac{14}{63} \times 100 = 22.22$。程序输出 **22.22 分**。

最终答案

核心公式 `round(sum(1 for o, u in zip(origin, userInput) if o == u) / len(origin) * 100, 2)`，并检查：两参数均为字符串、原文非空、输入不长于原文。

6. 结果检验　边界测试：输入为空串 → 0.00；输入与原文完全相同 → 100.00；输入比原文长 → 提示错误并返回 None；参数不是字符串（如整数）→ `isinstance` 检查拦截。

易错点

1. **分母用了 userInput 的长度**：只打 1 个字且打对就得 100 分，违背题意。
2. **以为 round 是“四舍五入”**：Python 的 `round` 对恰好一半的情况采用“**向偶数舍入**”（`round(2.5) == 2`，`round(3.5) == 4`），并且 `round(2.675, 2)` 得 2.67，因为 2.675 在二进制中实际存的是 2.67499999…。对成绩显示通常无伤大雅，严格的十进制舍入要用 `decimal` 模块。
3. **round 后显示位数不足**：`round(50.0, 2)` 显示为 `50.0`；“保留 2 位小数”的**显示**要用格式化 `f'{score:.2f}'`。
4. **原文为空**：分母为 0 会抛 `ZeroDivisionError`，需提前判断。

【实验十四】生成棋盘纹理图片

1. 考查知识点　pillow 的 `Image.new`、`ImageDraw.Draw`、`rectangle`、`putpixel`、`save`；坐标系；奇偶性；整数划分。

2. 已知条件　参数：宽 W、高 H、颜色 c_1, c_2、每行（每列）格子数 m。

3. 解题目标　生成 $m \times m$ 个格子、两色交替的棋盘图片并保存。

4. 解题思路与原理　第 i 行第 j 列（从 0 计）的格子颜色由 $(i+j) \bmod 2$ 决定：为 0 取 c_1，为 1 取 c_2（图16）。上下或左右相邻的两个格子 $i+j$ 恰好相差 1，奇偶性必然不同，所以颜色一定交替——这就是“棋盘纹理”的全部原理。

5. 格子边界的计算　第 j 列的横坐标范围取 $\lfloor jW/m \rfloor$, $\lfloor (j+1)W/m \rfloor$)。相邻两列的边界由同一个公式算出，首尾相接，因此 W 不能被 m 整除时也既无缝隙也不重叠；最后一列右边界正好是 $\lfloor mW/m \rfloor = W$。pillow 的 `rectangle([x0, y0, x1, y1])` **包含**右下角像素，所以传入 $x_1 - 1$, $y_1 - 1$。

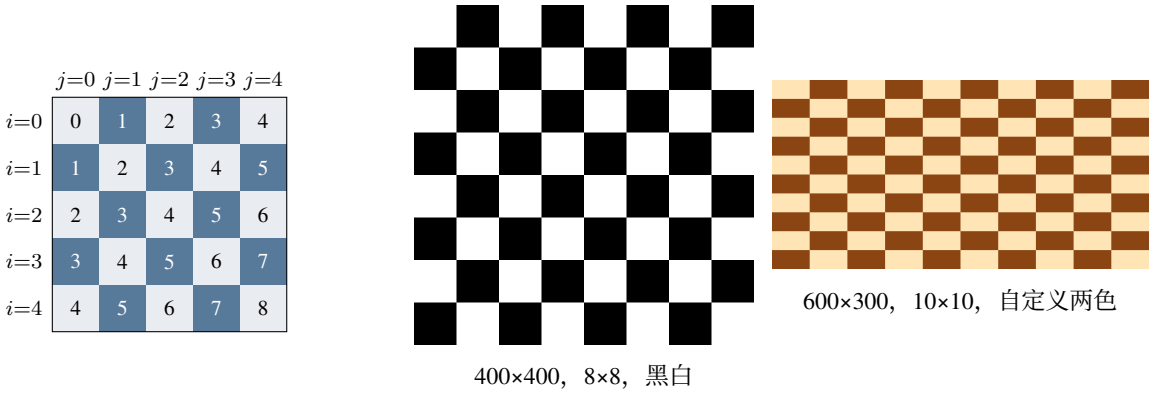


图 16　左：格子中的数字是 $i+j$，偶数为浅色、奇数为深色；中、右：程序实际生成的两张图片。看点：宽高不同时格子变成矩形，但交替规律不变。

```python
# -*- coding: utf-8 -*-
""" 实验十四：生成棋盘纹理图片（需要 pip install pillow）。
棋盘宽度、高度、交替的两种颜色、网格数量都可以通过参数指定，
水平方向和垂直方向的网格数量相同。

原理：第 i 行第 j 列的格子颜色由 (i + j) 的奇偶性决定，
相邻格子 i+j 差 1，奇偶性一定不同，所以颜色交替。
"""
from PIL import Image, ImageDraw


def chessboard(width=400, height=400, color1=(255, 255, 255), color2=(0, 0, 0),
               num=8, filename='chessboard.png'):
    im = Image.new('RGB', (width, height), color1)
    draw = ImageDraw.Draw(im)
    for i in range(num):                        # 行
        for j in range(num):                    # 列
            # 用整数运算计算每个格子的边界，宽高不能整除 num 时也不会留缝
            x0, x1 = j * width // num, (j + 1) * width // num
            y0, y1 = i * height // num, (i + 1) * height // num
            color = color1 if (i + j) % 2 == 0 else color2
```

```
22              draw.rectangle([x0, y0, x1 - 1, y1 - 1], fill=color)   # 右下角坐标包含在内，
                ↪ 故 -1
23      im.save(filename)
24      return im
25
26
27  def chessboard_pixel(width, height, color1, color2, num, filename):
28      """ 对照写法：逐像素判断颜色（速度较慢，但原理最直接）。"""
29      im = Image.new('RGB', (width, height))
30      for x in range(width):
31          for y in range(height):
32              i, j = y * num // height, x * num // width
33              im.putpixel((x, y), color1 if (i + j) % 2 == 0 else color2)
34      im.save(filename)
35      return im
36
37
38  if __name__ == '__main__':
39      chessboard(400, 400, (255, 255, 255), (0, 0, 0), 8, 'chessboard_8x8.png')
40      chessboard(600, 300, (255, 228, 181), (139, 69, 19), 10, 'chessboard_600x300_10.png')
41      a = chessboard(200, 200, 'red', 'blue', 5, 'chessboard_a.png')
42      b = chessboard_pixel(200, 200, (255, 0, 0), (0, 0, 255), 5, 'chessboard_b.png')
43      assert a.tobytes() == b.tobytes()          # 逐字节比较两幅图像
44      print('已生成 chessboard_8x8.png、chessboard_600x300_10.png 等图片；两种写法结果一致')
```

6. 解法二：逐像素着色　第 27–35 行对每个像素 (x, y) 反推它所在的格子：$i = \lfloor ym/H \rfloor$，$j = \lfloor xm/W \rfloor$，再按奇偶着色。它与解法一使用同一套边界公式（x 属于第 j 列 $\iff jW/m \le x < (j+1)W/m$），程序第 43 行逐字节比较两幅图完全一致。比较：解法一每个格子画一次矩形，调用 m^2 次；解法二每个像素调用一次 `putpixel`，$W \times H$ 次，400×400 图就是 16 万次，明显更慢，但原理最直观。

最终答案

调用 `chessboard(width, height, color1, color2, num, filename)` 即可生成任意尺寸、任意两色、num×num 格子的棋盘图片（见图16）。

7. 结果检验　①左上角像素应为 color1（$i = j = 0$）；② 8×8 棋盘中两种颜色格子各 32 个；③两种独立写法逐字节一致。

易错点

1. **格子宽用 `width // num` 再乘 j**：$W = 600, m = 7$ 时格宽 85，$7 \times 85 = 595$，右侧留下 5 像素的底色条。用“先乘后除”的边界公式可避免。
2. **rectangle 右下角不减 1**：相邻格子会重叠 1 像素，后画的覆盖先画的边。
3. **颜色只按 j 的奇偶决定**：得到的是竖条纹而不是棋盘；必须用 $i + j$。
4. **保存 JPG 后颜色边缘发虚**：JPEG 是有损压缩，锐利边缘会出现杂色。棋盘这类纯色图形更适合 PNG。

【实验十五】数据分析与可视化综合实验

1. 考查知识点　csv 模块写文件；pandas 读 CSV、`dropna`、时间索引、`groupby`、`diff`；matplotlib 折线图、柱状图、饼图与保存；文本文件写入；中文显示与文件编码。

2. 已知条件　题目给定的数据生成程序：第 i 天（$i = 0, \ldots, 364$，即 2017-01-01 至 2017-12-31）的营业额

$$A_i = 300 + 5i + U_i, \qquad U_i \in \{0, 1, \ldots, 99\} \text{ 均匀随机.}$$

3. 解题目标　完成 5 个子任务，生成 `first.jpg`、`second.jpg`、`maxMonth.txt`、`third.jpg`。

4. 整体思路　数据处理的固定流水线是“读入 → 清洗 → 变换/聚合 → 输出”（图17）。按月、按季度统计本质上都是“分组求和”：先把日期列转换成真正的日期类型并设为索引，就能直接取 `index.month`、`index.quarter` 作为分组键。

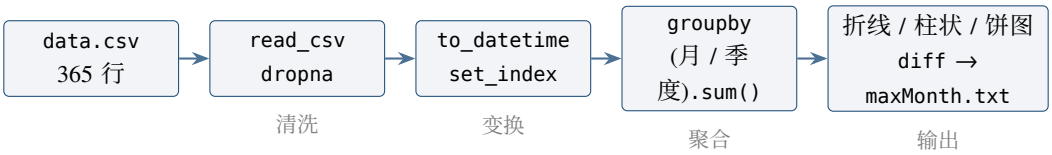


图 17　本实验的数据处理流水线。看点：任务 2 用的是“变换”后的逐日数据，任务 3–5 用的是“聚合”后的月/季度数据。

(1) 生成数据

```python
# -*- coding: utf-8 -*-
""" 实验十五（1）：生成饭店营业额模拟数据文件 data.csv（题目给出的程序）。
与原程序相比只做了两处保护性修改（不改变数据含义）：
  * open(..., newline=''): Windows 下避免 csv 每行之间多出空行；
  * encoding='utf-8-sig'：表头含中文，显式指定编码，pandas 和 Excel 都能正确读取。
"""
import csv
import random
import datetime

random.seed(2017)   # 固定随机种子，便于复现讲义中的结果；删去后每次数据不同

fn = 'data.csv'

with open(fn, 'w', newline='', encoding='utf-8-sig') as fp:
    # 创建 csv 文件写入对象
    wr = csv.writer(fp)
    # 写入表头
    wr.writerow(['日期', '销量'])

    # 生成模拟数据
    startDate = datetime.date(2017, 1, 1)

    # 生成 365 个模拟数据，可以根据需要进行调整
    for i in range(365):
        # 生成一个模拟数据，写入 csv 文件
        amount = 300 + i * 5 + random.randrange(100)
```

```
        wr.writerow([str(startDate), amount])
        # 下一天
        startDate = startDate + datetime.timedelta(days=1)

print('已生成', fn)
```

与原程序相比只改了第 15 行的打开方式：`newline=''` 防止 Windows 下每行之间多出空行（csv 模块自己写 `\r\n`，文本模式又会把 `\n` 转成 `\r\n`）；`encoding='utf-8-sig'` 防止中文 Windows 默认用 GBK 写入、而 pandas 默认按 UTF-8 读取导致的 `UnicodeDecodeError`。第 11 行固定随机种子仅为复现讲义结果。

(2) 分析与可视化

```
# -*- coding: utf-8 -*-
""" 实验十五（2）：数据分析与可视化综合实验。
1) pandas 读取 data.csv，创建 DataFrame，删除所有缺失值；
2) 折线图：每天营业额 -> first.jpg；
3) 按月统计，柱状图 -> second.jpg；
4) 按月统计，找出相邻两个月最大涨幅，把涨幅最大的月份写入 maxMonth.txt；
5) 按季度统计 2017 年营业额，饼状图 -> third.jpg。
运行前先执行 lab15_1_make_data.py 生成 data.csv。
"""
import pandas as pd
import matplotlib
matplotlib.use('Agg')                          # 无窗口环境也能保存图片；在 IDLE/Jupyter 中可删去
import matplotlib.pyplot as plt

# 中文字体：按顺序尝试，Windows 一般有 SimHei / Microsoft YaHei
plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'Noto Sans CJK SC',
                                   'PingFang SC', 'Arial Unicode MS', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False  # 负号正常显示

# ---------- 1) 读取并删除缺失值 ----------
df = pd.read_csv('data.csv', encoding='utf-8-sig')
df = df.dropna()                                # 删除含缺失值的行（返回新对象，需要接收）
df['日期'] = pd.to_datetime(df['日期'])
df = df.set_index('日期')
print(df.head())
print('数据行数：', len(df))

# ---------- 2) 每天营业额折线图 ----------
plt.figure(figsize=(10, 4))
plt.plot(df.index, df['销量'], linewidth=1, color='tab:blue')
plt.title('饭店 2017 年每日营业额')
plt.xlabel('日期')
plt.ylabel('营业额')
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('first.jpg', dpi=150)
plt.close()
```

```
38
39  # ---------- 3) 按月统计柱状图 ----------
40  monthly = df['销量'].groupby(df.index.month).sum()   # 索引 1~12
41  print('每月营业额：')
42  print(monthly)
43  plt.figure(figsize=(9, 4))
44  plt.bar([f'{m}月' for m in monthly.index], monthly.values, color='tab:orange')
45  plt.title('饭店 2017 年每月营业额')
46  plt.xlabel('月份')
47  plt.ylabel('营业额')
48  plt.tight_layout()
49  plt.savefig('second.jpg', dpi=150)
50  plt.close()
51
52  # ---------- 4) 相邻两个月最大涨幅 ----------
53  diff = monthly.diff()                          # 第 m 月 - 第 m-1 月，1 月为 NaN
54  maxMonth = int(diff.idxmax())                  # 涨幅（增加额）最大的月份
55  rate = monthly.pct_change()                    # 增长率口径，作对照
56  maxRateMonth = int(rate.idxmax())
57  with open('maxMonth.txt', 'w', encoding='utf-8') as fp:
58      fp.write(f'涨幅最大的月份：{maxMonth}月（较{maxMonth - 1}月增加 {int(diff[maxMonth])}）
    ↪  \n')
59      fp.write(f'按增长率计算，最大的月份：{maxRateMonth}月（增长率
    ↪  {rate[maxRateMonth]:.2%}）\n')
60  print('相邻月份增加额：')
61  print(diff)
62  print('涨幅最大的月份：', maxMonth)
63
64  # ---------- 5) 按季度统计饼状图 ----------
65  quarterly = df['销量'].groupby(df.index.quarter).sum()
66  print('每季度营业额：')
67  print(quarterly)
68  plt.figure(figsize=(5, 5))
69  plt.pie(quarterly.values, labels=[f'第{q}季度' for q in quarterly.index],
70          autopct='%.1f%%', startangle=90, counterclock=False)
71  plt.title('饭店 2017 年各季度营业额占比')
72  plt.tight_layout()
73  plt.savefig('third.jpg', dpi=150)
74  plt.close()
75  print('已生成 first.jpg、second.jpg、third.jpg、maxMonth.txt')
```

任务 1：读取与删除缺失值（第 21–24 行）　`dropna()` 删除任何含 NaN 的行。它**返回新对象**，必须写 `df = df.dropna()` 接收。随后把“日期”列由字符串转为 datetime 并设为索引，为后续按月、季度分组做准备。本数据由程序生成，实际没有缺失值（仍为 365 行），但在真实数据中这一步必不可少。

任务 2：逐日折线图（第 29–37 行）　横轴为日期索引，纵轴为营业额。图形应呈现一条斜率约为每天 5 的上升带，带宽约 100（来自随机项 U_i）。

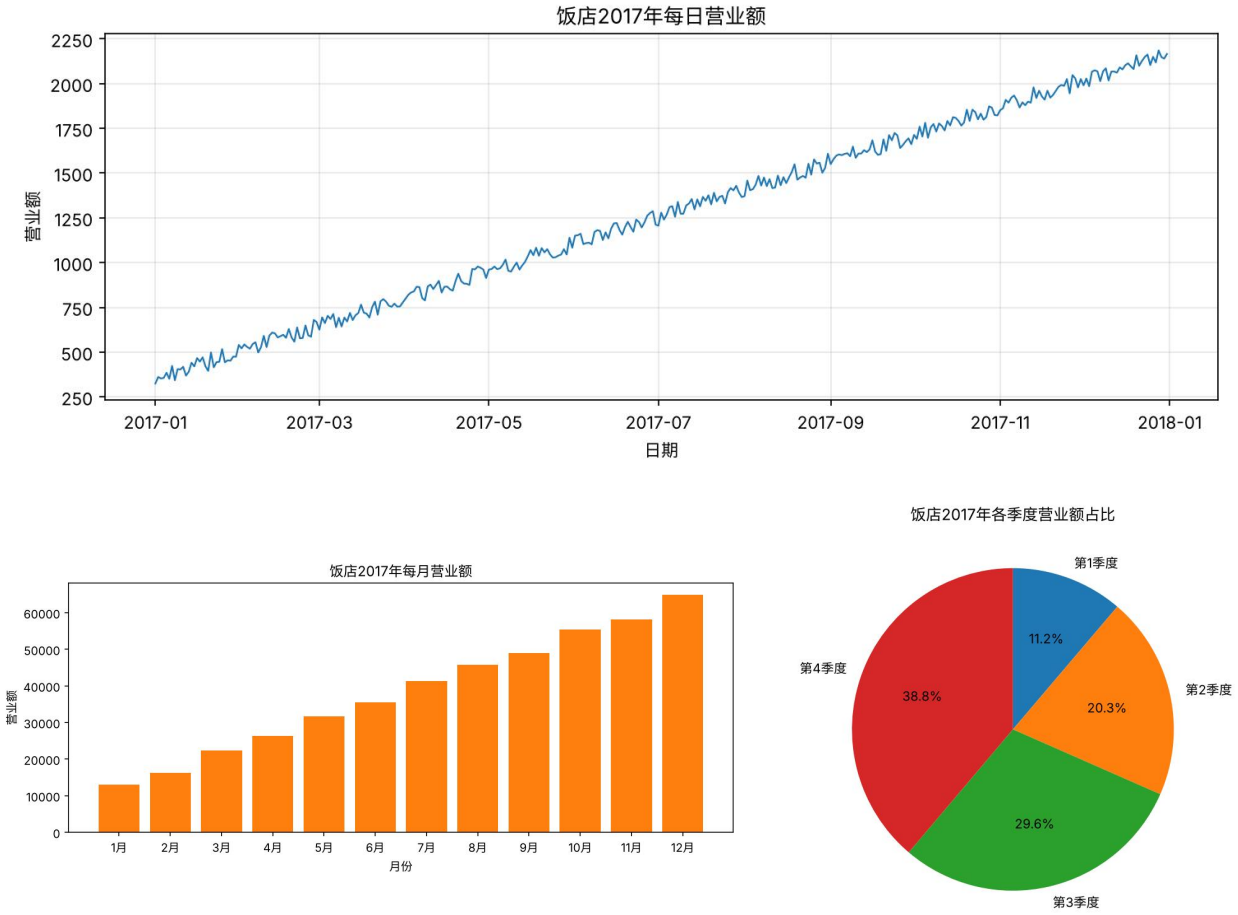


图 18　程序实际生成的 first.jpg（上）、second.jpg（左下）、third.jpg（右下）。看点：逐日数据是带噪声的直线；按月汇总后 2 月因只有 28 天而“矮一截”；四个季度占比逐季增加。

任务 3：按月统计柱状图（第 40–50 行）　`groupby(df.index.month).sum()` 得到 1–12 月的营业额：

月份	1	2	3	4	5	6	7	8	9	10	11	12
营业额	13102	16251	22308	26358	31678	35540	41340	45830	49050	55367	58189	64850
较上月增加	–	3149	6057	4050	5320	3862	5800	4490	3220	6317	2822	**6661**

任务 4：相邻两月最大涨幅（第 53–59 行）　`monthly.diff()` 计算 $\Delta_m = S_m - S_{m-1}$，`idxmax()` 返回最大值对应的月份。本次数据中最大涨幅出现在 **12 月**（比 11 月多 6661），写入 `maxMonth.txt`。“涨幅”也可以理解为增长率 $S_m/S_{m-1} - 1$，按此口径最大的是 3 月（37.27%），程序把两种口径都写入文件并注明。

任务 5：季度饼图（第 65–74 行）　季度营业额为 51661、93576、136220、178406，合计 459863，占比依次为 11.23%、20.35%、29.62%、38.80%。`autopct='%.1f%%'` 让每块扇形显示一位小数的百分比。

最终答案

按上面两个程序完成全部任务。本次数据（随机种子 2017）结果：最大涨幅月份为 **12 月**（较 11 月增加 6661）；季度占比 11.23% / 20.35% / 29.62% / 38.80%。数据随机生成，不固定种子时具体数值会变化。

结果检验（理论期望）　由 $E[U_i] = 49.5$，第 m 月营业额的期望为 $E[S_m] = \sum_{i \in m}(349.5 + 5i)$。

- 12 月 $i = 334 \sim 364$（31 天，i 的平均 349）：$31 \times 349.5 + 5 \times 31 \times 349 = 64929.5$；
- 11 月 $i = 304 \sim 333$（30 天，平均 318.5）：$30 \times 349.5 + 5 \times 30 \times 318.5 = 58260$；
- 期望差 6669.5，实测 6661，非常接近 ✓。同理可算 10 月比 9 月的期望差 6364.5、3 月比 2 月 6288.5，都小于 12 月。

这说明“12 月涨幅最大”主要源于两个确定性因素：日营业额线性增长，以及 12 月比 11 月多一天；随机项的影响（月差的标准差约 $\sqrt{61 \times 833} \approx 225$）不足以通常改变结论，但换随机种子时 10 月偶尔可能反超。

易错点

1. **`resample('M')` 在新版 pandas 中报错**：pandas 3.0 中 `'M'` 已不再作为“月末”频率，需写 `'ME'`；本讲义改用 `groupby(df.index.month)`，对新旧版本都适用。
2. **中文乱码或方块**：matplotlib 默认字体不含中文，需设置 `font.sans-serif`（第 16–17 行），并设 `axes.unicode_minus=False` 让负号正常显示。
3. **只写 `df.dropna()` 不接收**：原 df 不变，等于没删。
4. **保存图片前先 `plt.show()`**：在部分环境中 show 之后图形被清空，再 savefig 得到空白图片；应先 savefig 再 show。
5. **“涨幅”口径不说明**：增加额与增长率可能给出不同月份（本例 12 月 vs 3 月），报告结论时要写清口径。

第三部分　专题拓展：举一反三、知识体系、高校公开题与科学史

开篇　题目分类与筛选

综合知识重要度、典型度、方法代表性、综合性、易错度与可迁移度，把 15 个实验分为四类（同一实验可属多类）：

类别	实验	入选理由
A 重点核心题	三、四、五、八、十、十一	分别是“筛法/素数判定”“随机模拟”“递推与递归”“类与运算符重载”“条件概率建模”五个母题，后续课程反复出现
B 典型基础题	二、六、十二、十三、十四	运算符、内置函数、字符串、循环与异常处理的标准练习
C 易错题	二 (4)、六、七、十二、十三、十五	整除截断、for...else、补零、顺序替换、round、编码与 pandas 版本差异
D 综合题	九、十五	九需建模 + 输入校验 + 策略分析；十五串联文件、数据清洗、聚合与三种图表

实验一为环境配置，属于所有实验的前置条件，不单独归类。

第一章　本次实验的知识体系总览

15 个实验放在一起，本质上在训练一件事：**把一个文字描述的问题，翻译成“数据怎么存 + 步骤怎么走 + 结果怎么验证”**。图19把知识点、对应实验与后续用途连成一张图。

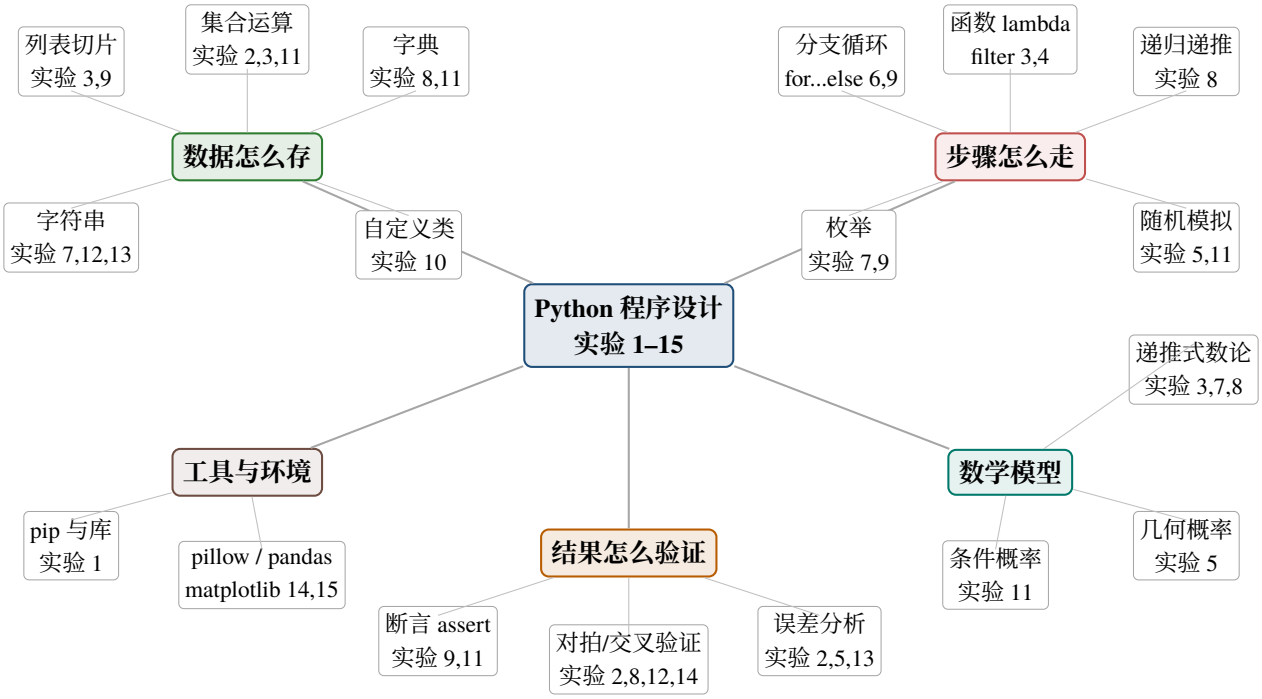


图 19　本次实验的知识地图。看点：几乎每个实验都同时用到“存储结构 + 控制流程 + 验证手段”三类知识，验证手段（断言、对拍、误差估计）是贯穿全部实验的暗线。

知识点	核心概念 / 典型公式	对应实验	常见用途
运算符与数值	`// % divmod`；浮点误差	2,5,13	进制转换、数位处理、计数统计
容器	列表（有序可重复）、集合（无序去重）、字典（键值映射）	2,3,8,9,11	存数据、去重、查表、缓存
函数式工具	`filter / map / lambda`、推导式、生成器	3,4,13	筛选、变换、汇总
控制结构	if–elif–else、for/while...else、try...except...else	6,9	交互程序、输入校验
递推与递归	$f(n)=f(n-1)+f(n-2)+f(n-3)$；记忆化	8	计数问题、动态规划
枚举	搜索空间 + 不变性剪枝（组合代替排列）	7,9	验证猜想、穷举策略
随机模拟	$\hat{p}=h/n$，误差 $\propto 1/\sqrt{n}$	5,11	估计概率、数值积分
面向对象	私有成员、特殊方法、property	10	自定义数据类型
字符串	切片、`maketrans/translate`、`join`	7,12,13	文本处理、简单加密
数据分析	读取、清洗、分组聚合、可视化	15	报表与数据洞察

第二章　重点题深度回顾

母题一：素数（实验三、四）

为什么重要：素数判定与筛法是“循环 + 条件”的标准母题，也是复杂度分析的第一个例子（试除到 $\sqrt{n}$ 与筛法 $O(N \log\log N)$ 的对比）。**训练的能力**：把数学定义翻译成代码，并用定理（“合数必有不超过 $\sqrt{n}$ 的素因子”）缩小循环范围。**母题结构**：“对区间 $[2, N)$ 中每个数判断性质 P”——单个判断用试除，批量判断用筛（先标记、再收集）。**识别信号**：“求 N 以内所有……”、“统计个数”、“质因数”、“互质”。

母题二：随机模拟估计（实验五、十一）

为什么重要：当解析解难求时，“大量随机试验求频率”几乎总是可行的。**训练的能力**：建立“事件 $\leftrightarrow$ 程序中的条件判断”的对应，并估计误差。**母题结构**：“重复 n 次随机试验，统计事件发生次数 h，用 h/n 估计概率，再换算成目标量”。蒙特·卡罗求 π 与模拟蒙蒂霍尔只是“事件”不同。**识别信号**：“模拟”“估计”“近似值”“概率是多少（难以直接计算）”。

母题三：递推与递归（实验八）

为什么重要：它是后续“动态规划”的入门形态，也是差分方程（离散系统）的最简单例子。**训练的能力**：按“最后一步”分类建立递推式；识别重复子问题并缓存。**母题结构**：$f(n) = \sum_{s \in S} f(n-s)$，$S$ 为每步可选的步长集合；换成“硬币面额”“编码方式”“铺砖方式”仍是同一结构。**识别信号**：“有多少种方法”“每次可以……或……”“规模为 n 的问题可由更小规模得到”。

母题四：自定义类型与运算符重载（实验十）

为什么重要：numpy 数组的 `a + b`、pandas 的 `df['a'] * 2` 都是同一机制。**训练的能力**：把数学对象的运算规则映射到特殊方法。**母题结构**：数据（私有成员）+ 运算（特殊方法）+ 派生量（property）。换成复数、分数、矩阵、多项式类仍是同一写法。**识别信号**：“定义一个……类，支持运算符……”“要求使用属性实现”。

母题五：条件概率建模（实验十一）

为什么重要：“新信息如何改变概率”是贝叶斯推断、滤波与机器学习的核心。**训练的能力**：写清规则（谁知道什么、怎么选择），再分情况计算或模拟。**母题结构**：先验 $\to$ 观察到的行为 $\to$ 后验；行为若与未知量相关，就携带信息。**识别信号**：“已知……发生了，求……的概率”“主持人/检测/证人”。

第三章　重点题举一反三

每道变式只改变一个关键条件，并给出完整答案。

变式 3.1（基础迁移，对应实验三、四）　孪生素数

题目：输出所有小于 100 的孪生素数对 $(p, p+2)$（两者都是素数）。
改变了什么：从“输出素数”变为“输出素数之间的关系”。**思路**：先用筛法得到素数**集合** P，再检查 $p+2 \in P$；集合的成员判断平均 $O(1)$。

```
P = set(primes_by_list(100))          # 复用实验三的函数
twins = sorted((p, p + 2) for p in P if p + 2 in P)
```

答案：$(3,5),(5,7),(11,13),(17,19),(29,31),(41,43),(59,61),(71,73)$，共 8 对。**检验**：$p>3$ 时孪生素数对必为 $(6k-1,6k+1)$ 形式，上面除 $(3,5)$ 外都满足（如 $41=6\cdot7-1$）。

变式 3.2（条件变化，对应实验三）　最小素因子筛与质因数分解

题目：对 $2\le n<N$，求每个数的最小素因子 $\mathrm{spf}(n)$，并据此分解 360。
改变了什么：筛法从“记录是否被划掉”变为“记录被**谁**第一个划掉”——这正是图5中颜色所表示的信息。

```
def spf_table(N):
    spf = list(range(N))                  # 先假设每个数的最小素因子是自己
    for p in range(2, int(N ** 0.5) + 1):
        if spf[p] == p:                   # p 是素数
            for m in range(p * p, N, p):
                if spf[m] == m:           # 还没被更小的素数划掉
                    spf[m] = p
    return spf

def factorize(n, spf):
    out = []
    while n > 1:
        out.append(spf[n]); n //= spf[n]
    return out
```

答案：`factorize(360, spf_table(1000))` 得 $[2,2,2,3,3,5]$，即 $360=2^3\cdot3^2\cdot5$。**检验**：$8\times9\times5=360$ ✓。从 p^2 开始划是因为比 p^2 小的 p 的倍数 $kp\,(k<p)$ 已被 k 的素因子划过。

变式 3.3（条件变化，对应实验五）　用随机投点求积分

题目：用蒙特·卡罗方法估计 $I=\int_0^1 e^{-x^2}\,dx$，并说明精确值。
改变了什么：目标从“面积比”变成“函数平均值”。**原理**：若 $X\sim U(0,1)$，则 $E[e^{-X^2}]=\int_0^1 e^{-x^2}\cdot1\,dx=I$，所以 $\hat I=\frac1n\sum e^{-x_k^2}$。

```
import random, math
n = 10**6
I = sum(math.exp(-random.random() ** 2) for _ in range(n)) / n
```

答案：一次运行得 0.74685；精确值 $I=\frac{\sqrt\pi}{2}\,\mathrm{erf}(1)\approx0.746824$。误差约 2×10^{-5}，与理论标准差 $\sigma/\sqrt n\approx0.2/1000$ 同数量级 ✓。

变式 3.4（反求参数，对应实验五）　需要投多少次?

题目：要使 $\hat\pi$ 以 95% 的把握落在 $\pi\pm0.001$ 内，至少要投多少次?
改变了什么：已知精度反求样本量。**推导**：由中心极限定理 $\hat\pi$ 近似正态，95% 区间半宽为 $1.96\sigma_{\hat\pi}=1.96\times\frac{1.642}{\sqrt n}\le0.001$，解得

$$n\ge\left(\frac{1.96\times1.642}{0.001}\right)^2\approx1.04\times10^7.$$

答案：约 1040 万次。**检验**：精度提高 10 倍（0.01→0.001），次数增加 100 倍（约 10.4 万 →1040 万），符合 $1/\sqrt n$ 规律。

变式 3.5（条件变化，对应实验八）　第 7 级台阶坏了

题目：15 级楼梯，每步 1–3 级，但第 7 级台阶坏了不能踩，有多少种方法？
解法一（递推加限制）：令 $g(0)=1$，$g(7)=0$，其余 $g(i)=g(i-1)+g(i-2)+g(i-3)$（负下标取 0）。**解法二（补集计数）**：踩到第 7 级的走法 =（走到 7 级的方法）×（从 7 级走完剩下 8 级的方法）$=f(7)\cdot f(8)=44\times 81=3564$，所以答案 $=5768-3564=2204$。
答案：**2204** 种（两种方法与程序结果一致）。**与原题比较**：递推式不变，只是在某一位置把值强制为 0——这是动态规划中处理“障碍”的标准手法。

变式 3.6（综合提升，对应实验八）　不能连续两步都跨 3 级

题目：同样 15 级、每步 1–3 级，但不允许连续两步都上 3 级，有多少种方法？
改变了什么：方法数不再只取决于“还剩几级”，还取决于“上一步是不是 3 级”，所以状态要扩充为 $g(n,\text{last3})$。

```
from functools import lru_cache
@lru_cache(None)
def g(n, last3):                    # 还剩 n 级，上一步是否为 3 级
    if n == 0:
        return 1
    return sum(g(n - s, s == 3) for s in (1, 2, 3)
               if s <= n and not (s == 3 and last3))
print(g(15, False))
```

答案：**4888** 种。**检验**：小于无限制时的 5768 ✓；$n=6$ 时无限制为 24 种，其中 3+3 这 1 种被禁止，程序给出 23 ✓。

变式 3.7（条件变化，对应实验七）　三位数的“黑洞”

题目：对数字不全相同的三位数做同样的“最大减最小”操作，会得到什么？最多几步？
答案：程序枚举全部 891 个数，都收敛到 **495**（$954-459=495$），最多 **6** 步。**推导**：设数字 $a\ge b\ge c$，差为 $(100a+10b+c)-(100c+10b+a)=99(a-c)$，只可能是 $99,198,\ldots,891$ 这 9 个值之一，问题瞬间缩小到 9 个数的追踪，与原题“结果只与数字集合有关”的思想一致。

变式 3.8（条件变化，对应实验十一）　n 扇门

题目：$n\,(n\ge 3)$ 扇门，1 辆车；主持人开一扇山羊门后允许改选到剩余 $n-2$ 扇中的任意一扇（随机选）。改选赢的概率是多少？若主持人一次打开 $n-2$ 扇山羊门呢？
推导：开一扇门时，改选赢 $=P(\text{首选山羊})\times P(\text{在剩余}n-2\text{ 扇中选中车})=\frac{n-1}{n}\cdot\frac{1}{n-2}$。开 $n-2$ 扇时只剩一扇可改，改选赢 $=\frac{n-1}{n}$。
答案：$P_{\text{改}}=\dfrac{n-1}{n(n-2)}$（$n=4$ 时为 $3/8$，坚持为 $1/4$）；开 $n-2$ 扇时 $P_{\text{改}}=\dfrac{n-1}{n}$。**检验**：$n=3$ 两式都退化为 $2/3$ ✓；且 $\frac{n-1}{n(n-2)}>\frac{1}{n}$ 恒成立，改选总是更优。

变式 3.9（综合提升，对应实验十二）　仿射密码与破解

题目：(1) 用仿射密码 $E(x)=(5x+8)\bmod 26$ 加密 `hello`，并写出解密函数；(2) 截获凯撒密文 `Wfaovu pz mbu`，不知道密钥，如何破解？
(1) h,e,l,l,o 编号 7,4,11,11,14，代入得 $(35+8)\bmod 26=17\to$ r，$28\bmod 26=2\to$ c，$63\bmod 26=11\to$ l，$78\bmod 26=0\to$ a，所以密文 `rclla`。解密需要 5 在模 26 下的逆元：$5\times 21=105=4\times 26+1$，

故 $5^{-1}\equiv 21$，$D(y)=21(y-8) \bmod 26$（Python 中 `pow(5, -1, 26)` 直接得 21）。要求 $\gcd(a,26)=1$，否则不可逆。

(2) 凯撒密钥只有 26 种，逐个尝试 `caesar(c, -k)`，看哪个结果是有意义的英文：$k=7$ 时得到 `Python is fun`。这说明凯撒密码的密钥空间太小，不安全；第三部分第十章 MIT 公开题就是让程序用词典自动判断“有意义”。

变式 3.10（综合提升，对应实验十）　点积、叉积与垂直判断

题目：为 Vector3 增加点积（运算符 `@`）与叉积方法 `cross`，并判断 $\vec{a}=(1,2,3)$ 与 $\vec{b}=(2,-1,0)$ 是否垂直、求 $\vec{a}\times\vec{b}$。

公式：$\vec{a}\cdot\vec{b}=a_1b_1+a_2b_2+a_3b_3$；$\vec{a}\times\vec{b}=(a_2b_3-a_3b_2,\ a_3b_1-a_1b_3,\ a_1b_2-a_2b_1)$。

```
def __matmul__(self, other):          # a @ b
    return self.x * other.x + self.y * other.y + self.z * other.z
def cross(self, o):
    return Vector3(self.y*o.z - self.z*o.y, self.z*o.x - self.x*o.z, self.x*o.y -
    ↪  self.y*o.x)
```

答案：$\vec{a}\cdot\vec{b}=2-2+0=0$，垂直；$\vec{a}\times\vec{b}=(2\cdot0-3\cdot(-1),\ 3\cdot2-1\cdot0,\ 1\cdot(-1)-2\cdot2)=(3,6,-5)$。**检验**：叉积应同时垂直于两向量：$(3,6,-5)\cdot(1,2,3)=3+12-15=0$ ✓，$(3,6,-5)\cdot(2,-1,0)=6-6=0$ ✓。

第四章　易错题专题

难题不等于易错题。下面五个陷阱，代码都不长，但错误隐蔽、运行时往往不报错。

易错 4.1　整除的截断时机（实验二第 4 小题）

错误来源：运算失误 + 概念不清——以为 `a*b//c` 与 `a*(b/c)` 只是写法不同。

错误示范

`result = result * (n-j) // (minNI-j)`，`cni(5,2)` 得 8。

为什么错：整除只有在“被除数恰为除数的倍数”时才不丢信息；中间结果 $5/2$ 不是整数。**正确规则**：边乘边除时，要保证每一步的部分积都是整数——按 $1, 2, \ldots, k$ 的顺序除，部分积恰为 $\binom{n}{j+1}$。**快速自检**：用小数据（$n = 5$）与 `math.comb` 对拍。

易错 4.2　for...else 的归属（实验六、九）

错误来源：思维定势——把 else 一律理解为“if 的否则”。

错误示范

```python
for i in range(3):
    x = int(input())
    if x == value:
        print('对了'); break
    else:
        print('次数用完，答案是', value)   # 每猜错一次就打印一次
```

为什么错：这里 else 与 if 对齐，属于 if。**正确规则**：循环的 else 与 for/while 对齐，含义是“循环没有被 break 打断”。**快速自检**：猜对一次、猜错到底各运行一次，看“答案”是否只在后一种情况出现且只出现一次。

易错 4.3　顺序替换导致的“连锁反应”（实验十二）

错误来源：对字符串替换语义理解不清。错误代码见实验十二的易错点 1。**为什么错**：`str.replace` 返回新串，下一次替换作用在上一次的结果上；映射 a→b、b→c 被串联成 a→c→……。**正确规则**：需要“所有字符同时映射”时用 `translate`，或逐字符构造新串。**快速自检**：$k = 1$ 加密 `ab`，正确结果是 `bc`；若出现 `cc` 或更离谱的结果，就是连锁替换。

易错 4.4　浮点数的“看起来相等”（实验二、五、十三）

错误来源：把十进制小数当作计算机里的精确值。

错误示范

`if 0.1 + 0.2 == 0.3: ...`（永远不成立）；`round(2.675, 2)` 期望 2.68，实际 2.67。

为什么错：0.1、0.2、2.675 在二进制下都是无限循环小数，只能存储最接近的 53 位近似值。**正确规则**：比较浮点数用 `math.isclose(a, b)`；需要精确十进制（如金额）用 `decimal.Decimal('2.675')`；整数问题坚持整数运算。**快速自检**：打印 `f'{x:.20f}'` 看真实存储值。

易错 4.5　数据文件的编码与空行（实验十五）

错误来源：平台差异——Windows 默认编码与换行习惯和 Linux/macOS 不同。

错误示范

中文 Windows 上按原题 `open(fn, 'w')` 写出 CSV，再 `pd.read_csv('data.csv')` 读取，抛出 `UnicodeDecodeError`；用 Excel 打开还能看到每两行之间多一个空行。

为什么错：`open` 未指定编码时使用系统默认编码（中文 Windows 一般为 GBK），pandas 默认按 UTF-8 解码；文本模式还会把 csv 写出的 `\r\n` 再转换一次。**正确规则**：读写文件**永远显式写 encoding**；写 CSV 加 `newline=''`。**快速自检**：用记事本/VS Code 查看文件右下角显示的编码。

第五章　易错题举一反三

变式 5.1（陷阱：整除时机）　卡特兰数

题目：用纯整数运算计算卡特兰数 $C_n = \dfrac{1}{n+1}\binom{2n}{n}$，求 C_{10}。**最易犯的错误**：写成 `1 // (n+1) * comb(2n, n)`，先算 $1//11 = 0$，结果为 0。**原因**：整除先做，信息全丢。**正确解法**：先乘后除 `comb(2*n, n) // (n + 1)`，由于 $\binom{2n}{n}$ 一定是 $n+1$ 的倍数（卡特兰数为整数），整除精确。**答案**：$\binom{20}{10} = 184756$，$184756/11 = 16796$。**防错原则**：整除只放在“已知能整除”的位置。

变式 5.2（陷阱：for...else）　读程序写结果

题目：下面程序输出什么？

```
for n in (9, 7):
    for d in range(2, n):
        if n % d == 0:
            print(n, '是合数'); break
    else:
        print(n, '是素数')
```

最易犯的错误：认为 9 会先打印“是合数”再打印“是素数”。**正确分析**：$n = 9$ 时 $d = 3$ 整除，打印“9 是合数”后 break，else 被跳过；$n = 7$ 时内层循环自然结束，执行 else。**答案**：输出两行“9 是合数”“7 是素数”。**防错原则**：for...else 的 else 读作“如果没有 break”。

变式 5.3（陷阱：顺序替换）　交换两个字母

题目：把字符串中所有 a 换成 b、所有 b 换成 a，例如 abba→baab。**最易犯的错误**：`s.replace('a','b').replace('b','a')` 得到 aaaa。**原因**：第二次替换把第一次产生的 b 也换回 a。**正确解法**：`s.translate(str.maketrans('ab', 'ba'))`，或借助中间占位符。**答案**：baab。**防错原则**：多个替换规则之间若“输出是另一条规则的输入”，就必须同时映射。

变式 5.4（陷阱：浮点比较）　累加 0.1

题目：`x = 0; for _ in range(10): x += 0.1`，问 `x == 1.0` 是否成立？**最易犯的错误**：认为成立。**正确分析**：每次相加都有舍入误差，结果为 `0.9999999999999999`，比较结果为 False。值得注意的是，从 Python 3.12 起内置 `sum()` 对浮点数改用补偿求和，`sum([0.1]*10)` 在本讲义环境（3.13）中恰好得到 1.0——但这只是更精确的求和，不能据此认为浮点运算是精确的。**答案**：逐次 += 时为 False；稳妥写法是 `math.isclose(x, 1.0)`，或用 `math.fsum`。**防错原则**：浮点数只比较“足够接近”，不比较“相等”。

变式 5.5（陷阱：迭代器只能用一次，对应实验四）

题　目：`r = filter(isPrime, [2,3,4,5])`；先执行 `print(list(r))`，再执行 `print(len(list(r)))`，输出什么？**最易犯的错误**：认为第二行输出 3。**正确分析**：filter 对象是迭代器，第一次 `list(r)` 已把它耗尽，第二次得到空列表。**答案**：`[2, 3, 5]` 和 `0`。**防错原则**：需要多次使用结果时，立即 `list()` 保存。

第六章　核心知识系统回顾

6.1　三种除法运算

是什么：`/` 真除法（结果为 float）；`//` 向下取整除法；`%` 取余，满足恒等式 `a == (a // b) * b + a % b`。**为什么这样定义**：Python 的 `//` 是**向负无穷取整**（floor），由恒等式可推出余数与除数同号，所以 `-7 // 2 == -4`、`-7 % 2 == 1`（图20）。这正是实验十二中负密钥能直接用 `%` 处理的原因。**易混淆**：C/C++ 的整数除法向 0 截断，`-7/2==-3`、`-7%2==-1`——在 C++ 课程（如数据结构与算法）中要特别留意这一差异。

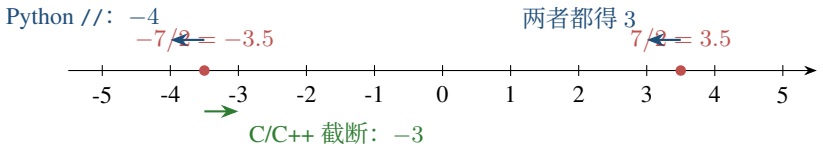


图 20　向下取整（Python）与向零截断（C/C++）。看点：两者只在负数时不同，Python 总是往数轴左边取。

6.2　三种容器：列表、集合、字典

解决什么问题：列表按位置存放有序数据（实验三、九）；集合负责去重和成员判断（实验三、十一）；字典按“键”快速查值（实验八缓存、实验十一门与奖品）。**为什么集合和字典查找快**：它们基于**哈希表**，先由元素算出哈希值直接定位存储位置，平均 $O(1)$；列表的 `in` 需要从头逐个比较，$O(n)$（图21）。**使用条件**：集合元素与字典的键必须**可哈希**（不可变），所以列表不能放进集合。

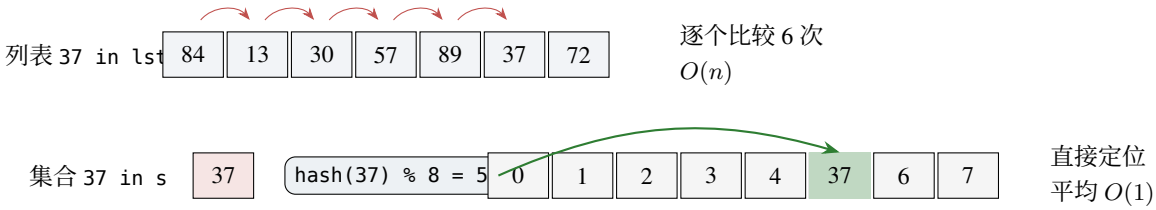


图 21　列表与集合的成员判断。看点：集合（以及字典的键）用哈希值直接算出位置，数据量越大优势越明显（示意图，桶数与真实实现不同）。

6.3　惰性计算：迭代器与生成器

是什么：`filter`、`map`、`zip`、`enumerate`、生成器表达式都返回迭代器，“用到一个才算一个”。**为什么有用**：实验十三中 `sum(1 for ...)` 不必先建出整张列表；处理大文件时可以一行一行读。**代价**：只能遍历一次（变式 5.5）。对比见图22。

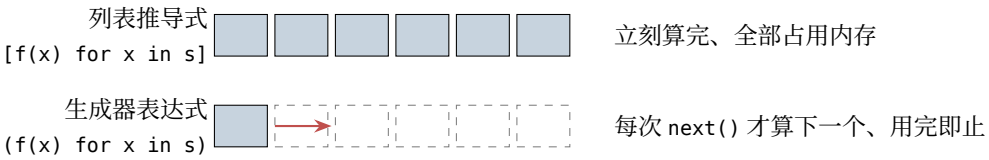


图 22　立即计算与惰性计算。看点：两者结果相同，差别在“何时计算、占多少内存、能否重复遍历”。

6.4　其余核心知识点（配图见第二部分）

知识点	是什么 / 为什么成立	使用条件与联系	图示
筛法	合数必有 $\le \sqrt{n}$ 的素因子，故只需用小素数划去倍数	需预知上界 N，内存 $O(N)$	图5
filter 与 lambda	函数作为参数，按真假放行	返回迭代器	图6
蒙特·卡罗	频率依大数定律收敛于概率，误差 $\propto 1/\sqrt{n}$	随机数需均匀、独立	图7
控制流程	break 跳过循环的 else；try 无异常才执行 else	缩进决定归属	图8
枚举与不变性	只依赖数字集合 $\to$ 用组合代替排列	搜索空间要有限	图9
递推与递归	按最后一步分类；记忆化消除重复子问题	需要初值（递归出口）	图10
状态集合	用“可能状态的集合”分析对手所有行为	状态数有限	图11
运算符重载	运算符即特殊方法调用	返回新对象	图12
条件概率	$P(A \mid B) = P(AB)/P(B)$；行为携带信息	规则必须写清	图13
模运算与映射	$E_k(x) = (x + k) \bmod 26$；translate 同时映射	映射表等长	图14
zip 配对	按位置配对，短者结束	注意截断	图15
奇偶性着色	相邻格 $i + j$ 奇偶不同	边界整数划分	图16
分组聚合	读入 $\to$ 清洗 $\to$ 变换 $\to$ 聚合 $\to$ 输出	日期需转为时间类型	图17

6.5　相似概念辨析

概念	核心区别	常见误区	判断方法
`remove` vs `discard`	元素不存在时前者报 KeyError，后者静默	筛法中用 remove 导致重复删除报错	是否可能删不存在的元素
`/` vs `//`	真除法得 float；整除向下取整	以为 `//` 是四舍五入或向零截断	看负数：`-7//2` 是 -4
round vs 格式化	round 改变数值；`f'{x:.2f}'` 只改变显示	用 round 保证“显示两位小数”	是否需要后续计算
递推 vs 递归	递推自底向上循环；递归自顶向下调用自身	认为递归一定更慢（记忆化后同为 $O(n)$）	是否存在重复子问题
概率 vs 频率	概率是模型中的确定值；频率是试验结果，随机波动	一次模拟 3.24 就认为程序错	误差是否与 $1/\sqrt{n}$ 同量级
assert vs 异常抛出	assert 用于“不应发生”的内部检查，可被 `-O` 关闭	用 assert 做正式的用户输入校验	关闭后程序是否仍正确
属性 vs 方法	属性像数据一样访问、不加括号	`v.length()`	是否带 `@property`

6.6　知识迁移：这些知识以后会在哪里再出现

- **递推式与记忆化** → 《数据结构与算法》的动态规划、递归复杂度分析；线性递推 $f(n) = f(n-1) + f(n-2) + f(n-3)$ 就是一个三阶线性**差分方程**，在《信号与系统》《自动控制原理》的离散系统部分会用 z 变换系统地求解（见第九章示例）。
- **哈希表（集合、字典）** → 数据结构课程中的散列表、冲突处理；数据库索引。
- **蒙特·卡罗与误差分析** → 《概率论与数理统计》的大数定律、中心极限定理；数值积分；强化学习中的蒙特卡罗方法。
- **条件概率** → 贝叶斯公式、贝叶斯滤波与卡尔曼滤波（用新的测量更新状态估计）。
- **运算符重载与类** → numpy 的向量化运算、C++ 中的运算符重载（`operator+`）。
- **分组聚合与可视化** → 数据库 SQL 的 `GROUP BY`；实验数据处理与科研作图。

第七章　学科科普：我们学的“程序设计”究竟是什么

研究什么：程序设计研究如何把问题的解法写成计算机能精确执行的步骤。它真正训练的是“计算思维”——把模糊的问题**分解**成小步骤、找出其中的**模式**、**抽象**出数据和操作、再设计**算法**。本次 15 个实验中，猜数（二分）、筛法（排除）、爬楼梯（递推）、抓狐狸（状态集合）恰好是四种不同的算法思想。

解决什么问题：凡是可以写成明确规则的事，都可以交给程序批量、准确地完成：实验十五用十几行代码完成了手工需要数小时的报表；实验七用不到一秒钟验证了 4536 个数。

为什么本科要学、处在什么位置：Python 程序设计是之后几乎所有“用计算机做事”的课程的工具基础（图23）。对控制、电子、计算机类专业而言，它连接着数学课（概率、线性代数）与专业课（数据结构、信号与系统、控制理论、机器学习）。

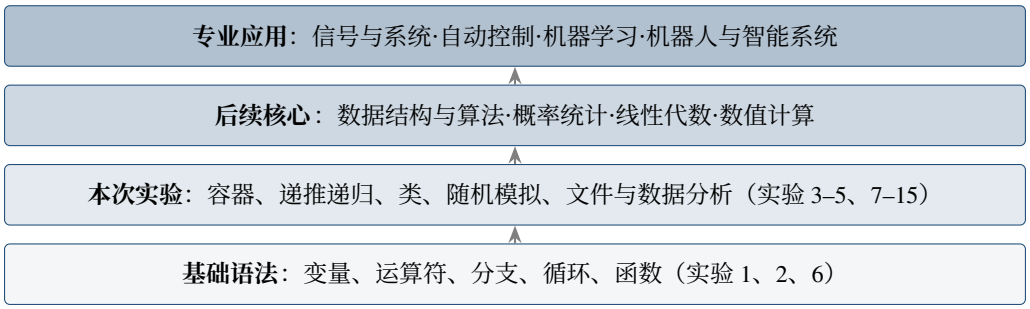


图 23　本次实验在课程体系中的位置。看点：越往上越专业，但每一层都建立在下一层的“会写、会验证”之上。

理论与现实的关系：程序是对现实的**模型**，模型总有简化。实验十五的营业额是“线性趋势 + 均匀噪声”，真实饭店还有周末、节假日效应；实验五假设随机数完全均匀，真实的伪随机数只是“足够好”。理解模型的假设，比写出代码更重要。

进一步会学到什么：用复杂度分析比较算法（如筛法 $O(N \log\log N)$ 对比逐个试除）；用数据结构组织更大规模的数据；用 numpy 向量化代替循环；用面向对象设计更大的程序。

第八章　科学史：这些题目背后的人与事

本次实验中的“小程序”几乎都对应着一段真实的历史。图24按时间排列；正文只陈述有可靠来源的内容，存在不同记载的地方如实说明。

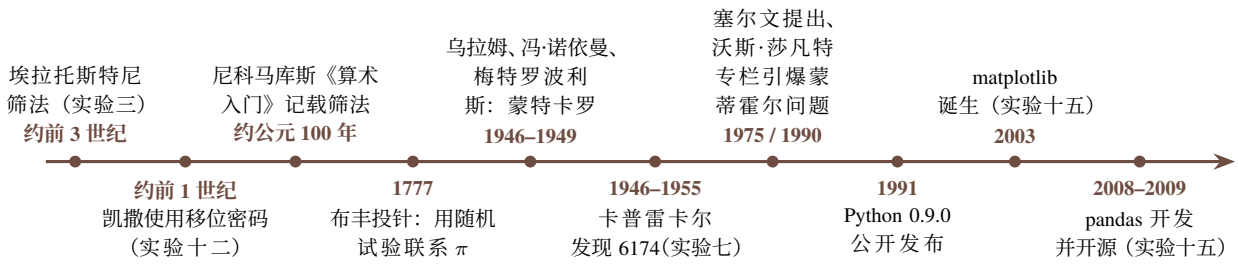


图 24　与本次实验相关的历史时间线。看点：这些想法大多先作为数学或密码学问题出现，几十到上千年后才变成几行代码就能完成的程序。

筛法：两千多年前的“排除法”（实验三）

埃拉托斯特尼（Eratosthenes，约前 276—前 194）是古希腊学者，也以估算地球周长闻名。他本人的著作没有流传下来，关于筛法的最早记载来自约公元 100 年前后尼科马库斯（Nicomachus of Gerasa）的《算术入门》，书中描述了这种方法并归功于埃拉托斯特尼。值得注意的是，尼科马库斯的版本是从奇数 3 开始列表的，与我们今天“从 2 开始”的写法略有不同。**与题目的联系**：我们在实验三中写的“划掉倍数、剩下素数”与两千年前的思想完全相同；唯一的新东西是 `filter` 和集合让“划掉”变得一行就能完成。

凯撒密码：有记载的移位加密（实验十二）

罗马史家苏埃托尼乌斯（Suetonius）在《罗马十二帝王传》中记载，凯撒写机密信件时把每个字母替换为字母表中其后第三个字母，解读时则反向移动三位；同书还记载奥古斯都使用移位一位的变体。史料只说明凯撒**使用**并推广了这种方法，并没有说他**发明**了移位密码。**与题目的联系**：实验十二的密钥 $k = 3$ 正是“凯撒原版”。这种密码在古代有效，很大程度上是因为当时识字的人很少，而不是算法本身安全——变式 3.9 用 26 次尝试就能破解它。

蒙特·卡罗方法：从纸牌到核物理（实验五）

据梅特罗波利斯 1987 年的回忆文章，数学家乌拉姆（Stanislaw Ulam）在病中休养时玩单人纸牌，想到与其推导获胜概率，不如实际玩很多局数一数；他与冯·诺依曼讨论后，把这一思路用于中子扩散计算。“蒙特·卡罗”这个名字由梅特罗波利斯提出，取自摩纳哥的赌城。1949 年，梅特罗波利斯与乌拉姆在《美国统计协会杂志》发表了题为“The Monte Carlo Method”的论文，这是该方法第一篇公开发表的论文。早在 1777 年，布丰就在投针问题中把随机试验与 π 联系起来，但系统地把随机抽样作为计算工具，要等到电子计算机出现之后。**与题目的联系**：原题说“这是蒙特·卡罗发明的方法”并不准确——蒙特·卡罗是地名，不是人名。

6174：一位中学数学教师的发现（实验七）

卡普雷卡尔（D. R. Kaprekar，1905—1986）是印度数学家，长期在中学任教，主要靠自学从事数论研究，当时不少同行认为他的问题过于琐碎。关于 6174 的时间存在不同记载：圣安德鲁斯大学数学史档案称他 1946 年发现这一性质、1949 年在马德拉斯数学会议上公布；而常被引用的论文“An interesting property of the number 6174”标注为 1955 年发表于 *Scripta Mathematica*。原题采用的是“1955 年”这一说法。**与题目的联系**：我们用 210 次循环完成的验证，正是当年需要大量手算才能确认的结论；1981 年还有学者证明，在十进制下这样的“黑洞”只有三位数的 495 和四位数的 6174。

蒙蒂霍尔问题：上万封读者来信（实验十一）

这个问题最早由统计学家塞尔文（Steve Selvin）1975 年在给 *The American Statistician* 的信中提出，并在随后的信中命名为“蒙蒂霍尔问题”（蒙蒂·霍尔是电视游戏节目《Let’s Make a Deal》的主持人）。1990 年，玛丽莲·沃斯·莎凡特在《Parade》杂志专栏回答读者提问时给出“应当改选”的结论，随后收到约一万封来信，其中近千封来自博士，多数认为她错了。据记载，大数学家埃尔德什也是在看到计算机模拟结果之后才接受这一结论。**与题目的联系**：实验十一的模拟程序做的正是当年说服埃尔德什的那件事——当直觉与推导冲突时，模拟是一个有力的旁证。

Python 与数据分析工具（实验一、十五）

Python 由荷兰程序员吉多·范罗苏姆（Guido van Rossum）于 1989 年 12 月开始实现，1991 年 2 月以 0.9.0 版发布到 alt.sources 新闻组；`lambda`、`map`、`filter` 等函数式工具在 1994 年的 1.0 版中加入。matplotlib 由神经科学家约翰·亨特（John Hunter）创建；pandas 由韦斯·麦金尼（Wes McKinney）2008 年在一家投资管理公司工作时开始开发，2009 年前后开源。**与题目的联系**：实验四的 `filter` 与 `lambda` 已有三十多年历史；实验十五几行代码就能画出的柱状图，背后是科学计算社区二十年的积累。

第九章　关联学科知识网络与跨课程迁移

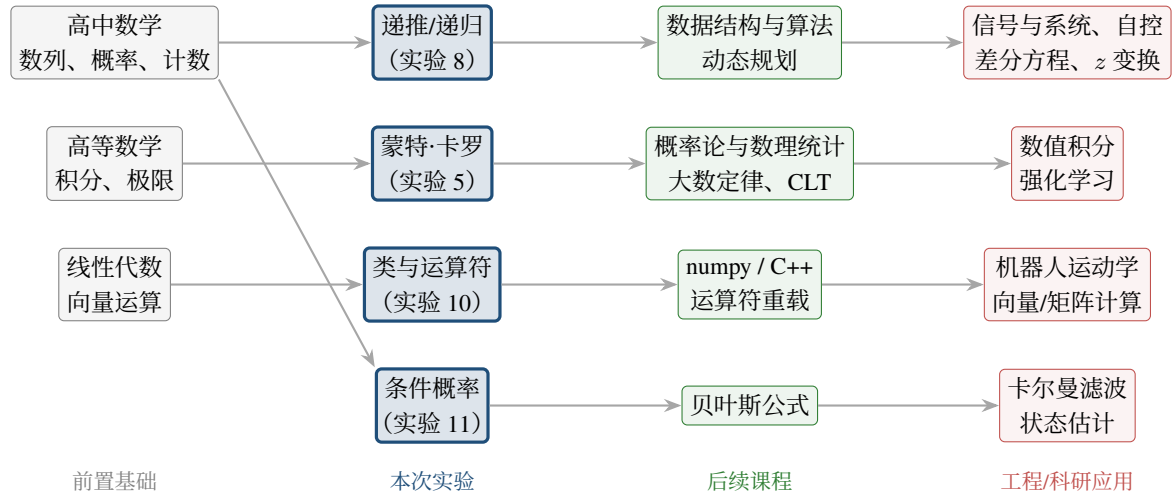


图 25　本次实验的四条主要知识链。看点：每条链从左到右都是真实的课程先修关系，而不是泛泛的“都有关”。

当前知识点	关联课程	为什么有关	关系	学习层级
递推式（实验 8）	数据结构与算法	动态规划就是“递推式 + 表格保存”	后续	必须掌握
递推式（实验 8）	信号与系统 / 自控	线性递推即离散时间系统的差分方程	应用	后续系统学习
筛法、试除（实验 3,4）	数据结构与算法	复杂度分析的经典例子	后续	建议掌握
蒙特·卡罗（实验 5）	概率论与数理统计	误差估计依赖中心极限定理	并行	建议复习
条件概率（实验 11）	概率论；状态估计	贝叶斯更新是滤波算法的核心步骤	后续/应用	拓展了解
向量类（实验 10）	线性代数；C++	向量运算定义；C++ 同样支持运算符重载	前置/并行	必须掌握
分组聚合（实验 15）	数据库；实验数据处理	与 SQL 的 GROUP BY 一一对应	应用	拓展了解

示例 9.1　爬楼梯与离散系统（强关联）

背景：控制与信号课程用差分方程描述离散时间系统。**当前知识**：$f(n) = f(n-1) + f(n-2) + f(n-3)$。**关联知识**：常系数线性齐次差分方程的解由特征方程的根决定。**连接**：设 $f(n) = r^n$ 代入

得特征方程

$$r^3 = r^2 + r + 1.$$

数值求解得一个实根 $r_1 \approx 1.8393$ 与一对共轭复根 $-0.4196 \pm 0.6063\,\mathrm{i}$（模约 $0.737 < 1$）。n 增大时复根项衰减，$f(n) \approx C\, r_1^n$。**验证**：$f(15)/f(14) = 5768/3136 \approx 1.83929$，与 r_1 吻合到第 5 位有效数字；这也解释了实验八中朴素递归调用次数按约 1.84^n 增长。在控制课程中，“特征根的模是否小于 1”正是判断离散系统是否稳定的条件——本例 $|r_1| > 1$，对应一个“不稳定”（发散）的系统。

示例 9.2　蒙蒂霍尔与贝叶斯滤波（拓展关联）

贝叶斯滤波每一步做两件事：**预测**（状态按模型演化）与**更新**（用测量修正概率）。实验十一中“主持人开门”就是一次测量，概率从 $(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$ 更新为 $(\frac{1}{3}, \frac{2}{3}, 0)$；实验九中“狐狸可能位置”的集合每天先按“跳到相邻洞口”演化（预测），再按“打开的洞口没有狐狸”删去（更新）——它正是贝叶斯滤波的“集合版”（只区分可能与不可能，不区分概率大小）。掌握程度：当前只需理解这一对应关系，系统学习在概率论与后续状态估计相关课程中进行。

最小必要知识清单

- **复杂度记号 $O(\cdot)$**：描述运行时间随规模增长的速度。现在需要会区分 $O(1)$、$O(n)$、$O(\sqrt{n})$、指数级；与实验三、四、八直接相关；系统学习在数据结构与算法课程。
- **中心极限定理（会用结论即可）**：大量独立随机变量之和近似正态，因此 95% 区间约为“均值 ± 1.96 倍标准差”；与实验五、十一直接相关；系统学习在概率论与数理统计。

第十章　国内外高校公开课程题典型题专题

本章选取能够核验来源的国内外高校公开课程材料中的题目，用不同学校的考查方式检验本次实验知识的迁移能力。**检索与核验说明**：共核验到 5 所高校（CMU、UC Berkeley、MIT、Harvard、北京理工大学）的 7 份相关公开材料；所选材料均为课程讲义例题、讨论课题目、习题集或公开课实例，**未检索到可核验的、与本次实验直接对应的正式期中/期末考卷原题**，因此不使用“历年真题”的说法。国内高校仅核验到北京理工大学一门公开课的相关实例，资料不足，不再凑数。题目均为摘要或必要摘录，完整题面请访问来源链接。

【公开课程题 1】CMU 15-112 课程讲义：nthPrime

学校 / 课程	卡内基梅隆大学·15-112 Fundamentals of Programming and Computer Science
学期 / 类型	2019 年夏季学期（多个学期讲义内容一致）·课堂讲义例题（Loops 一节）
来源	`https://www.cs.cmu.edu/~112-n19/notes/notes-loops.html`（访问日期 2026-11-03）
对应	素数判定、while 循环·原实验三、四·难度：基础·来源可信度：官方直接来源

题目（摘要）：在已有 `isPrime(n)` 的基础上，编写 `nthPrime(n)`，返回第 n 个素数（从 0 开始计数，即 `nthPrime(0)` 为 2）。**为什么选**：本次实验是“给定范围求素数”，这里反过来“给定个数求素数”，不知道要检查到多大的数，必须用 while 循环。**思路**：维护“已找到的素数个数 found”和“当前猜测 guess”，不断 `guess += 1`，是素数就 `found += 1`，直到找到第 n 个（下标从 0 计）。

```
def nthPrime(n):
    found = 0
    guess = 0
    while found <= n:          # 注意是 <=：要找到下标为 n 的那一个
        guess += 1
```

```
        if isPrime(guess):
            found += 1
    return guess
```

答案与检验：`nthPrime(10)` 为 31（素数序列 2,3,5,7,11,13,17,19,23,29,31 中下标 10 的元素）。**与本次实验的联系**：仍是“试除法判素数”母题，只是循环条件由“范围”改为“计数”。

易错点（与原作业易错点同一机制：边界差 1）

条件写成 `while found < n` 会返回第 $n-1$ 个素数。CMU 2022 年秋季课件在此处特意注明“found == n 时还要再循环一次”。考场自检：代入 $n=0$，应返回 2。

【公开课程题 2】CMU 15-112 课程讲义：Sieve of Eratosthenes 与素数计数函数

学校 / 课程　卡内基梅隆大学·15-112
学期 / 类型　2022 年夏季学期·课堂讲义例题（1d Lists: Worked Examples）
来源　https://www.cs.cmu.edu/~112-n22/notes/notes-1d-lists-examples.html（访问日期 2026-11-03）
对应　筛法、列表·原实验三·难度：基础·来源可信度：官方直接来源

题目（摘要）：用布尔列表 `isPrime = [True]*(n+1)` 实现筛法 `sieve(n)`，返回不超过 n 的全部素数；并据此计算素数计数函数 $\pi(n)$。**新增考查角度**：不是删除元素，而是**标记**元素；且包含 n 本身（$\le n$），与实验三的“$< N$”不同。**解法**：

```
def sieve(n):
    isPrime = [True] * (n + 1)
    isPrime[0] = isPrime[1] = False
    primes = []
    for prime in range(n + 1):
        if isPrime[prime]:
            primes.append(prime)
            for multiple in range(2 * prime, n + 1, prime):
                isPrime[multiple] = False
    return primes
```

答案：$\pi(100) = $ `len(sieve(100))` $= 25$。**比较**：标记法不修改列表长度，下标即数值，逻辑最清晰，且总操作数约 $n\sum_{p\le n}\frac{1}{p} = O(n \log\log n)$；实验三的切片 + filter 写法每次都要重建列表尾部，更慢但更“Python 风格”。

易错点

`[True] * (n+1)` 的长度若写成 `n`，则下标 n 越界或被漏判。自检：n 取素数（如 97），看结果是否包含 97。

【公开课程题 3】UC Berkeley CS 61A 讨论课：count_k

学校 / 课程	加州大学伯克利分校·CS 61A Structure and Interpretation of Computer Programs
学期 / 类型	2018 年秋季 Discussion 3（2017 春、2020 秋、2022 秋、2023 秋等学期讨论课均收录同题）
来源	https://inst.eecs.berkeley.edu/~cs61a/fa18/disc/disc03.pdf（访问日期 2026-11-03）
对应	递归计数·原实验八·难度：中等·来源可信度：官方课程页（访问时归档页面需伯克利账号登录，题面与测试用例依据搜索引擎收录的多个学期公开版本交叉核对，彼此一致）

题目（摘要）：上一个 n 级的楼梯，每次可以上 $1 \sim k$ 级（含 k），编写 `count_k(n, k)` 计算方法数。题目给出的测试：`count_k(3, 3)` 为 4，`count_k(4, 4)` 为 8，`count_k(10, 3)` 为 274，`count_k(300, 1)` 为 1。**与本次实验的联系**：实验八是 $k = 3$ 的特例，$f(10) = 274$ 与本讲义的表格一致。**思路**：最后一步走 i 级（$1 \le i \le k$），方法数为 $\sum_{i=1}^{k}$ count_k$(n-i, k)$；递归出口 $n = 0$ 返回 1（恰好走完是一种方法），$n < 0$ 返回 0（走过头不算）。

```
def count_k(n, k):
    if n == 0:
        return 1
    if n < 0:
        return 0
    return sum(count_k(n - i, k) for i in range(1, k + 1))
```

推导检验：`count_k(4,4)`：4 级楼梯的全部“有序拆分”有 $2^{4-1} = 8$ 种（每两级之间“断开或不断开”），步长上限 4 不构成限制 ✓；`count_k(300,1)` 只能每次 1 级，1 种 ✓。**解法比较**：这里用“$n = 0$ 返回 1、$n < 0$ 返回 0”两个出口，比实验八“$f(1), f(2), f(3)$ 三个初值”更通用，k 改变时不必重算初值——这是伯克利题目比本次实验多出的“推广性”考查。

易错点（与实验八同一机制：初值）

漏写 $n < 0$ 的出口会导致无限递归（n 变为负数后继续减小）。自检：先用最小规模 `count_k(1, 3)` 测试，应为 1。另外 $n = 300$、$k = 1$ 时递归深度约 300，安全；若 n 上千，需改为递推。

【公开课程题 4】MIT 6.00SC 习题集 4：含空格的凯撒密码

学校 / 课程	麻省理工学院·6.00SC Introduction to Computer Science and Programming（OCW）
学期 / 类型	2011 年春季·Problem Set 4（Problem 1: Encryption and Decryption）
来源	https://ocw.mit.edu/courses/6-00sc-introduction-to-computer-science-and-programming-spring-2011/resources/mit6_00scs11_ps4（访问日期 2026-11-03；题面经 OCW 镜像站核对）
对应	凯撒密码、模运算·原实验十二·难度：基础 → 中等·来源可信度：官方直接来源 + 官方镜像

题目（摘要 + 翻译）：该习题集使用凯撒密码的一个变体：**空格也参与移位**，被视为 z 之后的“第 27 个字母”。例如密钥为 2 时，y 变为空格，z 变为 a，空格变为 b。第 1 题要求实现加密与解密；后续题目借助英文词表 `words.txt` 自动找出未知密钥。（另：同校 6.0001 2016 年秋季 OCW 的 Problem Set 4 也包含名为“Cipher Like Caesar”的凯撒密码部分。）**新增考查角度**：字母表长度从 26 变为 27，并要求**自动**破解。**解法**：字母表 `a = 'abcdefghijklmnopqrstuvwxyz '`，$E(x) = (x + k) \bmod 27$。

```
a = 'abcdefghijklmnopqrstuvwxyz '
def enc(t, k):
    return t.translate(str.maketrans(a, a[k % 27:] + a[:k % 27]))
```

答案与检验：`enc('hello world', 2)` 得 `'jgnnqbyqtnf'`（空格变成 b）；`enc('xyz ', 2)` 得 `'z ab'`，与题面所举例子（y→ 空格、z→a、空格 →b）一致 ✓。**自动破解思路**：对 $k = 0 \sim 26$ 逐个解密，统计结果中有多少个词出现在词表里，取最多者——这把变式 3.9 中“人眼判断是否有意义”变成了程序判断。

易错点

把模数仍写成 26：空格无处可去，y、z 的映射全部错位。自检：字母表长度 `len(a)` 必须与模数一致——实验十二中先算 `k % 26` 的写法，背后正是这一条件。

【公开课程题 5】Harvard Stat 110 Strategic Practice 3：偏好开 2 号门的主持人

学校 / 课程　哈佛大学·Statistics 110: Probability（Joe Blitzstein）
学期 / 类型　2011 年秋季·Strategic Practice 3，第 1 节第 1 题（附官方解答）
来源　https://projects.iq.harvard.edu/sites/projects.iq.harvard.edu/files/stat110/files/strategic_practice_and_homework_3.pdf（访问日期 2026-11-03）
对应　条件概率、贝叶斯公式·原实验十一·难度：中等·来源可信度：官方直接来源

题目（摘要 + 翻译）：标准蒙蒂霍尔设定，玩家选 1 号门；但主持人更喜欢开 2 号门：当 2、3 号门后都是山羊时，他以概率 p（$\frac{1}{2} \le p \le 1$）开 2 号门。求：(a) 总是改选的无条件获胜概率；(b) 已知主持人开了 2 号门，改选获胜的概率；(c) 已知开了 3 号门，改选获胜的概率。

解答：(a) 改选赢 $\iff$ 首选是山羊，概率 $\frac{2}{3}$，与主持人的偏好无关。(b) 记 H_2 为“开 2 号门”：$P(H_2 \mid C=1) = p$，$P(H_2 \mid C=2) = 0$，$P(H_2 \mid C=3) = 1$，故 $P(H_2) = \frac{1}{3}(p+0+1)$，

$$P(\text{改选赢} \mid H_2) = P(C=3 \mid H_2) = \frac{1 \cdot \frac{1}{3}}{\frac{1}{3}(1+p)} = \frac{1}{1+p}.$$

(c) 对称地（把 p 换成 $1-p$）得 $\frac{1}{2-p}$。

检验：$p = \frac{1}{2}$ 时 (b)(c) 都是 $\frac{2}{3}$，回到标准问题 ✓；$p = 1$ 时 (b) 为 $\frac{1}{2}$——主持人总开 2 号门，开 2 号门不再提供信息。本讲义用实验十一的代码修改主持人选门规则后模拟 20 万次（$p = 0.8$）：开 2 号门条件下改选胜率 0.5549，理论 $\frac{1}{1.8} = 0.5556$；总体胜率 0.6666，理论 $\frac{2}{3}$ ✓。**与本次实验的联系**：这道题说明“总体胜率”和“看到具体开门后的条件胜率”是两个不同的量，标准问题中恰好相等，规则一变就分开了。

易错点（与实验十一同一机制：主持人行为携带信息）

认为 (b) 仍是 $\frac{2}{3}$。错因：忽略了 $P(H_2 \mid C=1)$ 从 $\frac{1}{2}$ 变成了 p。自检：取极端值 $p = 1$，看结论是否合理。

【真题同型变式 6】基于 MIT 6.0002 蒙特·卡罗与置信区间主题

主题来源　麻省理工学院·6.0002 Introduction to Computational Thinking and Data Science（2016 年秋季 OCW），Lecture 6 “Monte Carlo Simulation”、Lecture 7 “Confidence Intervals”
来源　https://ocw.mit.edu/courses/6-0002-introduction-to-computational-thinking-and-data-science-fall-2016/
说明　**本题为依据该课程公开主题自行编写的同型变式，不声称是 MIT 原题**·对应原实验五·难度：综合

题目：用蒙特·卡罗方法估计 π，要求以约 95% 的置信度使误差不超过 0.01。(1) 理论上至少需要多少次投掷？(2) 不知道 π 的真值时，程序怎样判断“已经足够精确”？

解答：(1) 由实验五，$\sigma_{\hat{\pi}} \approx 1.642/\sqrt{n}$，令 $1.96\sigma \le 0.01$ 得 $n \ge (1.96 \times 1.642/0.01)^2 \approx 1.04 \times 10^5$。(2) 用**样本**估计标准差：重复做 m 次独立试验（每次 n 镖），得到 m 个估计值，计算它们的样本标准差 s；当 $1.96\,s < 0.01$ 时停止，否则加倍 n 再试。这就是“用数据本身评估精度”的思想，不需要知道真值。**与本次实验的联系**：实验五只输出一个数，这里要求同时给出“这个数有多可信”，是从“会算”到“会评估”的升级。

【公开课程题 7】北京理工大学《Python 语言程序设计》：圆周率的计算

学校 / 课程　北京理工大学·Python 语言程序设计（嵩天, 中国大学 MOOC, 课程编号 BIT-268001）
类型　公开课单元实例（第 5 周“圆周率的计算”：问题分析、实例讲解、举一反三）
来源　https://www.icourse163.org/course/BIT-268001（访问日期 2026-11-03）
对应　蒙特·卡罗方法、random 库·原实验五·难度：基础·来源可信度：官方课程页

题目（概述）：该课程以“圆周率的计算”为单元实例，介绍用公式（级数）与用蒙特·卡罗方法两种途径计算圆周率，并要求体会蒙特·卡罗方法的思想。**与本次实验的比较**：本次实验只要求蒙特·卡罗一种方法；该实例把“确定性的公式计算”与“随机模拟”放在一起，引导比较两者的精度与速度。**补充解答（级数法）**：例如 $\pi = 4\sum_{k=0}^{\infty} \frac{(-1)^k}{2k+1}$（莱布尼茨级数），取前 N 项的误差小于 $\frac{4}{2N+1}$，要误差 $< 10^{-6}$ 约需 200 万项；而蒙特·卡罗要达到同样精度约需 $(1.642/10^{-6})^2 \approx 2.7 \times 10^{12}$ 次投掷。**结论**：在一维问题上，确定性方法远比随机模拟高效；蒙特·卡罗的优势在于高维积分和难以写出公式的问题（如实验十一）。

跨校横向比较

核心知识点	学校	材料	考查方式	难点 / 易错点	对应实验
素数	CMU	15-112 讲义	给个数求第 n 个；标记式筛法	边界差 1；含/不含 n	三、四
递归计数	Berkeley	CS 61A 讨论课	步长上限 k 作为参数	两个递归出口	八
凯撒密码	MIT	6.00SC PS4	27 字符表 + 词典自动破解	模数与表长一致	十二
条件概率	Harvard	Stat 110 SP3	改变主持人行为，求条件概率	区分总体与条件胜率	十一
蒙特·卡罗	MIT / 北理工	6.0002 主题 / MOOC 实例	置信区间；与公式法对比	用样本评估精度	五

高频母题结构

母题	识别信号	标准方法	常见变化
批量素数	“N 以内所有素数/个数”	筛法	含不含 N、求最小素因子、求第 n 个
步长计数	“每次可走……有几种方法”	按最后一步分类 + 递推/记忆化	步长集合、障碍、相邻步限制
移位替换	“每个字符按规则映射”	构建映射表 + 同时替换	字母表长度、仿射、自动破解
信息更新	“已知……发生，求……”	列全部情况 / 贝叶斯公式 / 模拟	主持人偏好、门数、检测准确率
随机估计	“模拟、近似、估计”	频率估计 + 误差 $\propto 1/\sqrt{n}$	求样本量、置信区间、积分

本章考试迁移总结

- **所核验材料中，国外课程的常见考法**：把参数显式化（k、p、字母表长度），要求写出**通用**函数并配测试用例；强调“自己验证”（CMU 讲义对比快慢两版 isPrime，Berkeley 给 doctest）。
- **所核验的国内公开课实例**：以完整“实例”组织，强调“问题分析 → 实例讲解 → 举一反三”，侧重方法对比。
- **共同核心**：建模（把文字规则变成条件判断）、边界（初值、含不含端点）、验证（小数据手算、对拍、极端值）。
- **复习优先级**：必会母题——筛法、步长计数、移位映射；必会辨析——总体概率与条件概率、/ 与 //；必查易错——边界差 1、模数与表长、递归出口；值得挑战——Stat 110 偏好主持人、MIT 词典自动破解。

第十一章　生活中的实际应用

【生活应用 1】修图软件里的“透明棋盘格”与相机标定板（实验十四）

场景：在常见的修图或设计软件中删除图片背景，透明区域会显示成灰白相间的棋盘格；手机拍照、自动驾驶等领域给相机做“标定”时，常用打印的黑白棋盘格图案作为标准参照。**作用机制**：修图软件按 $(i+j) \bmod 2$ 交替绘制浅灰与白色小方块，因为“规则的、不可能出现在真实照片里的花纹”最容易让人一眼认出“这里没有像素”；相机标定程序则检测黑白格的交点，因为这些角点的真实间距已知，从而反推镜头畸变（OpenCV 等库提供了专门的棋盘格角点检测函数）。**量化估算**：若格子边长 16 像素，1920×1080 的画布横向 $1920/16 = 120$ 格、纵向 $1080/16 = 67.5$ 格——不能整除，正是实验十四“边界整数划分”要处理的情况（**估算值**，各软件格子大小不同）。**模型与现实**：标定板要求格子物理尺寸精确，打印缩放会直接影响结果。**动手观察**：用实验十四的程序生成 1920×1080、16 像素格的棋盘图，看最下面一行格子是否只有半格高。**类型**：真实使用。

【生活应用 2】从凯撒密码到网上支付的 HTTPS（实验三、四、十二）

场景：浏览器地址栏出现锁形图标时，网页与服务器之间的数据是加密传输的。**作用机制**：凯撒密码与现代加密的共同点只是“用密钥把明文变成密文”（**类比**）；本质区别在于密钥空间大小（图26）。凯撒只有 26 种密钥，变式 3.9 中 26 次尝试就能破解；而常用的 AES-128 分组密码有 $2^{128} \approx 3.4\times10^{38}$ 种密钥。另一方面，RSA 等公钥算法生成密钥时需要随机找出很大的素数（**真实使用**素数判定），工程上常先用小素数试除快速排除大部分候选数，再用概率性素性检验确认。**量化**

估算：即使每秒尝试 10^{12} 个密钥（**估算值**），穷举 2^{128} 个也需约 $2^{128}/10^{12}/(3.15\times10^{7}) \approx 1.1\times10^{19}$ 年。**模型与现实**：现实中的攻击多针对实现漏洞和人为失误（弱口令、钓鱼），而非穷举密钥。

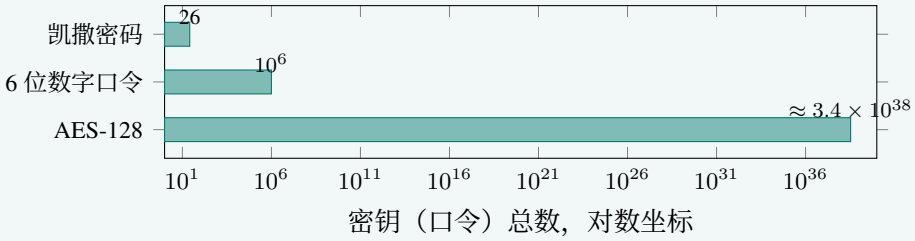


图 26　三种“密钥空间”的大小比较。看点：横轴为对数刻度，凯撒密码与 AES-128 的密钥数相差 37 个数量级。

【生活应用 3】天气预报里的“降水概率”与集合预报（实验五）

场景：天气 App 显示“明天降水概率 70%”。**作用机制**：大气是混沌系统，初始观测的微小误差会被放大。欧洲中期天气预报中心（ECMWF）的集合预报把同一个数值模式从略有不同的初始条件出发运行 51 次（1 个控制预报 + 50 个扰动成员），得到 51 种可能的未来（图27）。这与实验五“重复随机试验、统计频率”的思想一致：例如可以用“预报出降水的成员占比”来估计降水概率。**量化估算**：51 个成员时概率的最小分辨单位约 $1/51 \approx 2\%$；若真实概率为 0.5，把成员当作独立样本时频率的标准差约 $\sqrt{0.5\times0.5/51} \approx 7\%$（**估算值**，实际成员并非完全独立）。**模型与现实**：各国气象部门给出的降水概率还会结合统计后处理和预报员经验，并不只是简单数成员。**类型**：真实使用（集合预报）+ 类比（与投飞镖的对应）。

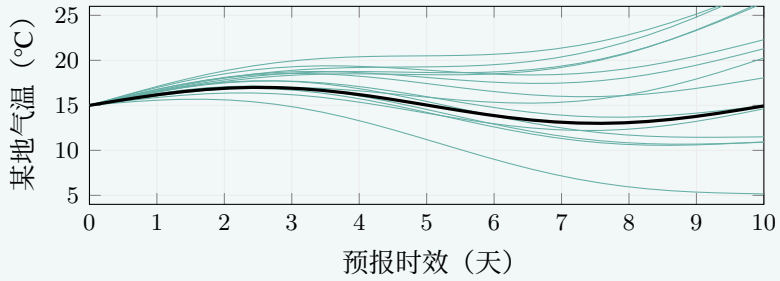


图 27　集合预报“意大利面图”示意（曲线为示意数据，非真实预报）。看点：开始几天各成员几乎重合，越往后越发散——发散程度本身就是“预报有多不确定”的信息。

【生活应用 4】体检“阳性”到底意味着什么（实验十一）

场景：某项筛查检测结果为阳性，是否就说明一定患病？**作用机制**：这与蒙蒂霍尔问题同为“根据新信息更新概率”的贝叶斯问题。设（**自设估算值**）患病率 1%，检测灵敏度 99%（患者检出阳性的概率），特异度 95%（健康者检出阴性的概率），则

$$P(\text{患病}\mid\text{阳性}) = \frac{0.99\times0.01}{0.99\times0.01 + 0.05\times0.99} = \frac{0.0099}{0.0594} \approx 16.7\%.$$

用 10000 人的“自然频率”看更直观（图28）：100 名患者中约 99 人阳性，9900 名健康者中约 495 人假阳性，阳性者中真正患病的只有 $99/594 \approx 1/6$。**模型与现实**：真实的患病率、灵敏度、特异度因疾病、人群和检测方法而异，具体结果的解读应以医生意见为准；这里只说明“患病率很低时，单次阳性的含义比直觉弱得多”这一规律。**类型**：真实使用（医学统计中的阳性预测值概念）。

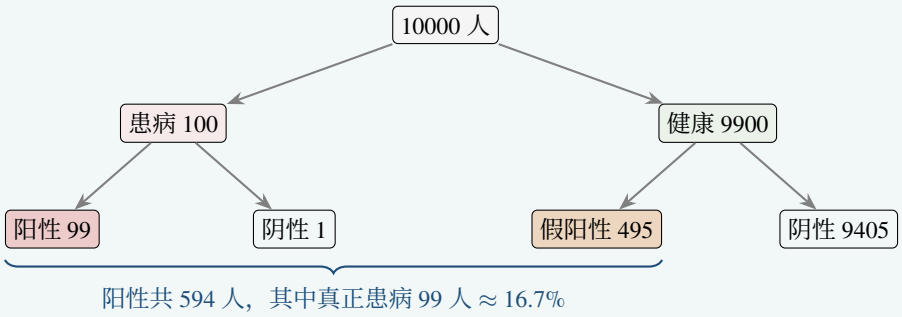


图 28　用“自然频率”理解检测结果（数据为自设估算值）。看点：假阳性来自人数巨大的健康人群，所以数量反而远多于真阳性。

【生活应用 5】打字测速网站与输入法纠错（实验十三）

场景：打字练习网站给出“准确率”；手机输入法把打错的单词自动改正。**作用机制**：实验十三按**位置**逐一比较，漏打一个字母就会让后面全部“错位”；更合理的做法是用**编辑距离**（把一个字符串变成另一个所需的最少插入、删除、替换次数）衡量差异，拼写纠错正是在词典中寻找与输入编辑距离最小的词。**量化对比**：以实验十三的输入 `Shandong Univrsity, School` 为例，与原文对应的 27 个字符相比只差 1 处（漏打 e，编辑距离 1），按“已输入部分”算准确率约 $26/27 \approx 96.3\%$；而按位置比较只得到 14 个字符正确（图29）。**模型与现实**：真实网站各自定义准确率口径，有的按单词计；纠错还会结合词频和上下文。**动手观察**：用标准库 `difflib.SequenceMatcher` 比较两个字符串，看它找到的匹配块。**类型**：真实使用（编辑距离）。

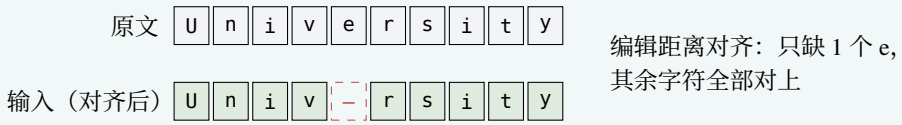


图 29　按编辑距离对齐（对比图15的按位置对齐）。看点：允许“插入一个空位”后，漏字只影响一个位置。

【生活应用 6】用 pandas 分析自己的月度开销（实验十五）

场景：不少银行和支付 App 支持把账单导出为 CSV 或 Excel 文件。**作用机制**：与实验十五完全相同的流水线——`read_csv` 读入、`dropna` 清洗、把日期列转为时间类型、按月份和消费类别 `groupby(...).sum()`、画柱状图和饼图。**量化估算（自设估算值）**：若一个月伙食 1200 元、交通 150 元、学习用品 300 元、娱乐 350 元，合计 2000 元，则伙食占 60%（图30）。**模型与现实**：真实账单常有退款（负数）、转账（不算消费）、同一商户多种名称，需要先清洗——这正是实验十五“删除缺失值”这一步在现实中的放大版。**动手观察**：导出一个月账单，复用实验十五的代码，只改列名。**类型**：真实使用。

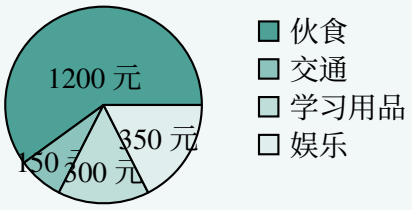


图 30　月度开销结构示意（自设估算值）。看点：与实验十五的季度饼图是同一种“分组求和 → 占比”的表达。

知识点	生活场景	起作用的公式/算法	类型	对应实验
奇偶着色	透明背景棋盘格、相机标定板	$(i+j) \bmod 2$	真实使用	十四
素数、移位加密	HTTPS、网上支付	素性检验；密钥空间	真实使用 / 类比	三、四、十二
随机模拟	集合预报与降水概率	频率估计概率	真实使用 / 类比	五
条件概率	体检结果解读	贝叶斯公式	真实使用	十一
字符串比较	打字测速、输入法纠错	编辑距离	真实使用	十三
分组聚合	个人账单分析	groupby + 可视化	真实使用	十五

第十二章　学习总结

必须掌握：三种容器的选用（列表/集合/字典）；filter、lambda、推导式；for/try 的 else 语义；按“最后一步”建立递推式；特殊方法与 property；随机模拟的写法与误差量级；pandas 的读取—清洗—分组—作图流程。

建议重点复习：实验二第 4 小题（整除时机）、实验八（递推与记忆化）、实验十一（条件概率）——它们分别对应“数值正确性”“算法效率”“建模严谨性”三种能力。

必须避免的错误：整除先除后乘；else 缩进错位；顺序 replace；浮点数用 == 比较；读写文件不写编码；递归缺出口；随机结果“看着不像”就怀疑程序。

做题识别流程（图31）：

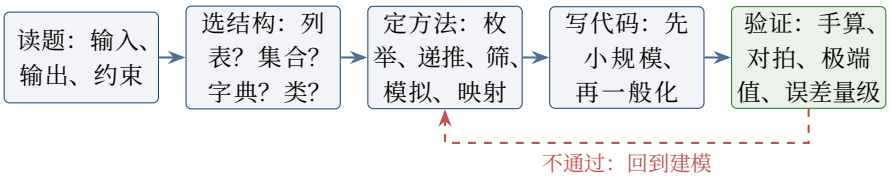


图 31　“看到什么条件 → 想到什么知识 → 选择什么方法 → 如何检查答案”的完整思维链。看点：验证不是最后的装饰，而是决定是否回头修改的关键一步。

看到什么条件	想到什么知识	如何检查答案
“所有……以内的素数”	筛法；合数必有 $\le \sqrt{n}$ 的素因子	取 N 略大于素数平方（如 50），看 49 是否被筛掉
“有多少种方法”“每次可以……”	按最后一步分类的递推	小 n 手工枚举；另一种方法交叉验证
“模拟”“近似”“估计”	频率估计概率，误差 $\approx \sigma/\sqrt{n}$	误差是否与 $1/\sqrt{n}$ 同量级
“已知……发生了”	条件概率；行为是否携带信息	取极端参数看结论是否合理
“定义类，支持运算符”	特殊方法、返回新对象	运算律：$(\vec{u}+\vec{v})-\vec{v}=\vec{u}$
“按月/按类统计”	groupby 聚合	各组之和是否等于总和

第四部分　参考资料与权威来源

以下资料均在编写本讲义时实际查阅或用于核验（网页访问日期均为 2026 年 11 月 3 日）。程序运行结果均由本讲义随附代码在 Python 3.13.18、pandas 3.0.5、matplotlib 3.11.2、pillow 12.3.0 环境中实际得到。

一、Python 官方文档与版本信息（用于实验一、二、四、十、十二、十三）

[1] Python Software Foundation. *Status of Python versions*（Python Developer's Guide）. https://devguide.python.org/versions/ ——各版本生命周期；3.7 于 2023-06-27 终止支持等信息另经 Simon Willison 博客（2023-06-27）交叉核对：https://simonwillison.net/2023/Jun/27/status-of-python-versions/；3.10 于 2026 年 10 月终止支持另见 https://endoflife.ai/article-python-310-eol。

[2] Python Software Foundation. *The Python Standard Library: Built-in Functions*（`filter`、`zip`、`round`、`divmod`、`enumerate`、`isinstance`）. https://docs.python.org/3/library/functions.html

[3] Python Software Foundation. *Built-in Types*（集合运算、`str.maketrans/translate`、浮点数）. https://docs.python.org/3/library/stdtypes.html

[4] Python Software Foundation. *Data model —Emulating numeric types*（`__add__`、`__truediv__`、`__rmul__`、`NotImplemented`）. https://docs.python.org/3/reference/datamodel.html

[5] Python Software Foundation. *Floating Point Arithmetic: Issues and Limitations*（0.1 的二进制表示、`round(2.675, 2)`）. https://docs.python.org/3/tutorial/floatingpoint.html

[6] Python Software Foundation. *itertools —combinations, filterfalse*. https://docs.python.org/3/library/itertools.html

[7] Python Software Foundation. *csv —CSV File Reading and Writing*（`newline=''` 的说明）. https://docs.python.org/3/library/csv.html

[8] H. A. Massa 等. "End Of life: Christoph Gohlke, Unofficial Windows Binaries for Python Extension Packages", PSF community 邮件列表, 2022-06. https://mail.python.org/archives/list/psf-community@python.org/thread/YJGN7MDNHEQQHV5VBMJDMMLT6EI4FSVS/ ——引用该网站 2022-06-12 公告：经费终止，服务于 2022 年 7 月前停止。

二、扩展库文档（用于实验十四、十五）

[9] pandas development team. *pandas documentation*（`read_csv`、`dropna`、`groupby`、`diff`、时间序列频率别名）. https://pandas.pydata.org/docs/ ——"`'M'` 已不再支持、需改用 `'ME'`"一点由本讲义环境（pandas 3.0.5）实际运行报错信息确认。

[10] Matplotlib development team. *Matplotlib documentation*（`plot`、`bar`、`pie`、`savefig`、rcParams 字体设置）. https://matplotlib.org/stable/

[11] Pillow contributors. *Pillow documentation*（`Image.new`、`ImageDraw.rectangle`、`putpixel`）. https://pillow.readthedocs.io/

三、高校公开课程材料（用于第三部分第十章）

[12] Carnegie Mellon University. *15-112: Fundamentals of Programming and Computer Science*, Summer 2019, Class Notes: Loops（isPrime、fasterIsPrime、nthPrime）. https://www.cs.cmu.edu/~112-n19/notes/notes-loops.html；同内容另见 Fall 2022 讲义 https://www.cs.cmu.edu/~112-f22/notes/notes-loops.html 与课件 https://www.cs.cmu.edu/~15112-f22/lecture_sli

des/15112_F22_Lec2_Week2_Tue.pdf。

[13] Carnegie Mellon University. *15-112*, Summer 2022, Class Notes: 1d Lists: Worked Examples（The Sieve of Eratosthenes、The Prime Counting Function）. https://www.cs.cmu.edu/~112-n22/notes/notes-1d-lists-examples.html

[14] University of California, Berkeley. *CS 61A*, Fall 2018 Discussion 3: Recursion & Tree Recursion（count_k）. https://inst.eecs.berkeley.edu/~cs61a/fa18/disc/disc03.pdf；同题另见 Spring 2017 Discussion 2、Fall 2020 / Fall 2022 / Fall 2023 Discussion 4。访问时上述归档页面需伯克利账号登录，题面依据搜索引擎收录的公开版本交叉核对。

[15] Massachusetts Institute of Technology. *6.00SC Introduction to Computer Science and Programming*, Spring 2011, Problem Set 4（Caesar Cipher）. MIT OpenCourseWare. https://ocw.mit.edu/courses/6-00sc-introduction-to-computer-science-and-programming-spring-2011/resources/mit6_00scs11_ps4；题面内容经 OCW 镜像核对：https://ocw.tau.edu.ng/courses/electrical-engineering-and-computer-science/6-00sc-introduction-to-computer-science-and-programming-spring-2011/unit-2/lecture-10-hashing-and-classes/MIT6_00SCS11_ps4.pdf

[16] Massachusetts Institute of Technology. *6.0001 Introduction to Computer Science and Programming in Python*, Fall 2016, Problem Set 4. MIT OpenCourseWare. https://ocw.mit.edu/courses/6-0001-introduction-to-computer-science-and-programming-in-python-fall-2016/resources/ps4

[17] Massachusetts Institute of Technology. *6.0002 Introduction to Computational Thinking and Data Science*, Fall 2016, Lecture 6: Monte Carlo Simulation; Lecture 7: Confidence Intervals. MIT OpenCourseWare. https://ocw.mit.edu/courses/6-0002-introduction-to-computational-thinking-and-data-science-fall-2016/

[18] J. Blitzstein. *Stat 110 Strategic Practice 3*, Fall 2011, Harvard University（含解答）. https://projects.iq.harvard.edu/sites/projects.iq.harvard.edu/files/stat110/files/strategic_practice_and_homework_3.pdf

[19] 北京理工大学，嵩天. 《Python 语言程序设计》，中国大学 MOOC（BIT-268001）. https://www.icourse163.org/course/BIT-268001

四、数学与科学史资料（用于第三部分第八章及各实验背景）

[20] N. Metropolis, S. Ulam.“The Monte Carlo Method”. *Journal of the American Statistical Association*, 44(247): 335–341, 1949.

[21] N. Metropolis. “The Beginning of the Monte Carlo Method”. *Los Alamos Science*, Special Issue, 1987: 125–130. https://mcnpx.lanl.gov/pdf_files/Article_1987_LAS_Metropolis_125--130.pdf

[22] J. J. O’Connor, E. F. Robertson. “Dattatreya Ramachandra Kaprekar”. MacTutor History of Mathematics Archive, University of St Andrews. https://mathshistory.st-andrews.ac.uk/Biographies/Kaprekar/

[23] D. R. Kaprekar. “An interesting property of the number 6174”. *Scripta Mathematica*, 1955（卷号在不同二手资料中记为 15 或 21，本讲义未能核验原刊）。

[24] Wikipedia contributors. “6174”、“Kaprekar’s routine”（含 Prichett 等 1981 年关于 495 与 6174 唯一性的结论及原始文献列表）. https://en.wikipedia.org/wiki/6174

[25] Wikipedia contributors. “Monty Hall problem”（Selvin 1975 年来信、vos Savant 1990 年专栏、

Vazsonyi 1999 关于 Erdős 的记载）. https://en.wikipedia.org/wiki/Monty_Hall_problem

[26] Wolfram Research & MoMath. “The Sieve of Eratosthenes”（Nicomachus《算术入门》的记载）. https://wolframcloud.com/obj/history-of-mathematics/production/artifacts/sieve-of-eratosthenes；另见 R. Gardner, *Notes on Eves' An Introduction to the History of Mathematics*, Section 6.3, East Tennessee State University. https://faculty.etsu.edu/gardnerr/3040/Notes-Eves6/Eves6-6-3.pdf

[27] Omni Calculator. “Who created the Caesar cipher? History of shift ciphers”（引述 Suetonius 的记载）. https://www.omnicalculator.com/who-created-the-caesar-cipher

[28] “History of Python”（Python 0.9.0 于 1991 年 2 月发布于 alt.sources；1.0 版加入 lambda、map、filter、reduce）. 经 EPFL Graph Search 与 python-course.eu 交叉核对：https://graphsearch.epfl.ch/concept/21356332, https://www.python-course.eu/python3_history_and_philosophy.php

[29] H. Bowne-Anderson. “Data Science Tool Building”（对 Wes McKinney 的访谈，pandas 的起源与开源时间）. DataCamp DataFramed, 2018-11-19. https://www.datacamp.com/podcast/data-science-tool-building

五、生活应用相关资料（用于第三部分第十一章）

[30] ECMWF. “Ensemble forecasting”（模式从略有不同的初始条件运行 51 次：1 个控制预报 + 50 个扰动成员）. https://www.ecmwf.int/node/21024

说明

第十一章中标注为“自设估算值”的数字（检测灵敏度与特异度、月度开销、每秒尝试密钥数等）仅用于演示计算方法，不代表任何真实统计数据。